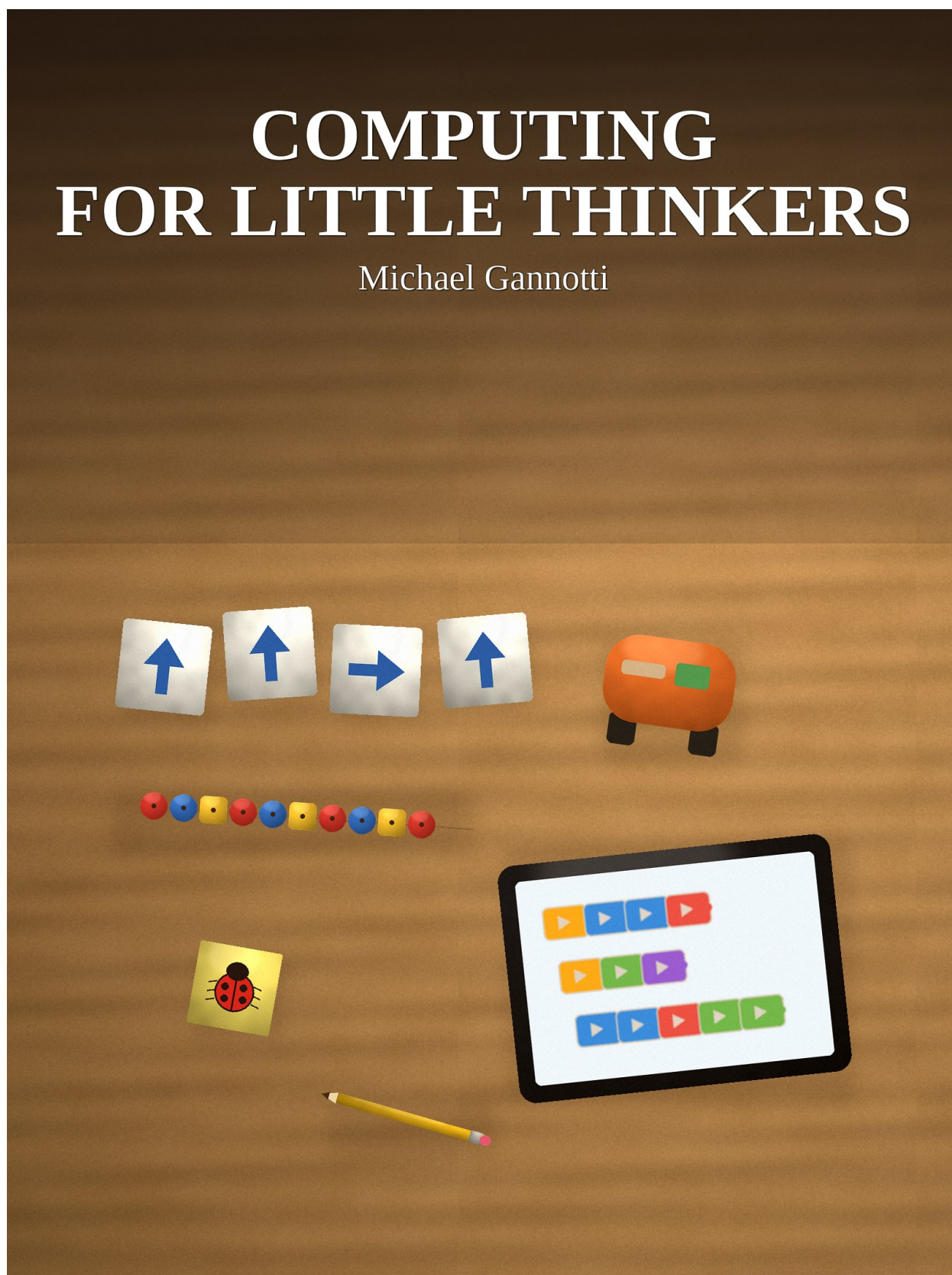


COMPUTING FOR LITTLE THINKERS

Michael Gannotti



Computing for Little Thinkers

Michael Gannotti

A warm homeschool guide to computing for ages 5–10, unplugged first. Book 1 of the SMF WisdomForge Computing series. Illustrated edition, 1 October 2026. Notes run in one series at the back; the superscript numbers link to them. The images are original still lifes, with no people and no brands.

Contents

- Welcome
- How to Use This Book This Week
- If You Only Remember Five Things
- The Computing Hour
- Chapter 1 — What Computing Looks Like at 5–10
- Chapter 2 — Steps and Algorithms
- Chapter 3 — Patterns, Loops, and If-Then
- Chapter 4 — Bugs Are Clues: Debugging as a Habit
- Chapter 5 — First Block Coding — and Robots If You Like
- Chapter 6 — Computers Up Close: Parts, Hands, Keys, and Files
- Chapter 7 — The Internet: How It Works, and How to Be Safe and Kind
- Chapter 8 — AI Explained, with the Grown-Up at the Keyboard
- Chapter 9 — Records
- Chapter 10 — Resources
- A Note on Sources
- Notes

Welcome

You don't need to be a programmer to teach a five- to ten-year-old real beginning computing.

You do need to ask "What do you think will happen?" before anyone presses go, and to leave the child's program in the child's hands.

That is what this book is for.

Why this book exists

Children this age live among computers. There is one in the car, one in the doorbell, one at the supermarket checkout, and one in the tablet that knows exactly which cartoon comes next. Many children can find a favorite video faster than the adults around them.

Very few are ever shown what is going on inside those machines. Fewer still learn the thing that sits underneath all of it: how to tell a computer exactly what to do, and how to fix it calmly when it doesn't do what you meant.

That is computing, and children five to ten can learn it. They can give a "robot" exact steps across the living room. They can lay arrow cards on a tape grid, spot the one card that sends the toy into the wall, and change just that card. They can snap blocks together on a tablet and make a cat dance. They can tell you what goes in, what the machine does, and what comes out. The professional standards that schools use put this kind of work as early as kindergarten, in modest, concrete steps.^{1,2}

Some adults feel unsure about computers, and some feel their children already know more about devices than they do. That is normal. Using a device and telling one what to do are different skills. This book teaches the second one, and it teaches you first, a little at a time, so you can sit beside your child with confidence.

The whole book rests on one line:

Predict before you run; debug before you start over.

Predict means saying what you think will happen before you press go. Debug means finding the mistake and fixing it. Hold those two habits, and the rest of the manual becomes practice.

What a good Computing Hour looks like

Here is a picture. It is an illustration, not a real family.

A six-year-old wants to get her father from the kitchen doorway to the couch. Her father is the robot today. He will do exactly what she says, and nothing else. Before he moves, she draws her plan as arrows on a strip of paper: forward, forward, forward, turn, forward. “What do you think will happen?” he asks. “You’ll sit on the couch,” she says.

He walks, slowly and very literally. On the last step he taps the side of the coffee table with one toe and stops. “Beep. Bump.”

She laughs, then looks at her strip. “You were supposed to turn sooner.” Together they find the first step that went wrong. She moves one arrow. Nothing else. “Run it again,” she says. This time the robot lands on the couch. She takes a photo of the arrow strip for the family folder and tells her brother one sentence about what she fixed.

The screen never came on. That was a full Computing Hour, about twenty minutes long. On other days a tablet will come out for ten or fifteen minutes, and the same habits go with it: guess first, run it, find the first wrong step, change one thing, save it, and tell someone.

You’ll meet the full session shape in **The Computing Hour**, right after the five-things page. Later chapters point back to it, so you won’t rebuild the routine each week.

Two readers at once

This is a teaching manual for the parent or co-teacher at the table, with short student sections a child can actually use.

The parent sections help you understand each week’s idea well enough to hear a wrong turn: a skipped step, a “just press it” with no guess, a child who deletes everything at the first bug, five changes made at once. Then you can ask one good question and wait, without grabbing the mouse or filling the silence with the answer.

The student sections speak to your child directly and respectfully. A five-year-old will hear them read aloud. Many eight- to ten-year-olds will read them alone. Each gives a tiny example, one thing to try today, and a question to answer back.

What this book will not do

It won’t hand you a 180-day worksheet stack. You’ll get a weekly shape and a menu of named activities, and you choose.

It won’t sell you a curriculum, a robot, or an app. Most of the activities use cards, tape, paper cups, socks, and your living-room floor. Where a free program helps, the book says so plainly.

It won’t promise a score, and it won’t tell you that coding makes children smarter. Coding teaches coding. Other good things may come along, and that is a welcome possibility rather than a promise.

It won't give your child accounts or chatbots. At five to ten, the grown-up holds every password and drives any AI tool. Children learn what AI is, mostly through games at the table, with no chatbot involved.

It won't title your child's year with a book name or hand out a certificate. If you keep records, the plain title a stranger can read is **Computer Science**.

It isn't a reprint of any earlier computing book. It was written fresh for this age.

Where this book sits

You are in good company. By the federal count, about 3.4 percent of school-age children in the United States were homeschooled in 2022–23.³

Computing for Little Thinkers is the first title in the SMF WisdomForge Computing series. When your child outgrows it, *Computing for Young Minds* picks up at ages 11–14, followed by *Computing for Emerging Adults* (15–18). *Computing Refresh for Adults* is for grown-ups who want their own turn.

If your family uses other Little Thinkers books, two are natural neighbors. *Critical Thinking Skills for Little Thinkers* practices noticing and reasoning that pair well with predicting and debugging. *Math for Little Thinkers* gives you counting, patterns, and number sense that show up in loops and binary cards. Here, we teach computing.

How the book is built

The front matter gives you how to use this week, five things to remember, and **The Computing Hour**, which also holds Family safety basics, a short note on screen time, and the one box on AI for this age. Each is said carefully, once.

Eight teaching chapters follow: what computing looks like at 5–10; steps and algorithms; patterns, loops, and if-then; bugs as clues; first block coding (and robots, if you like); computers up close; the internet, safe and kind; and AI explained, with the grown-up at the keyboard. Then come records and resources.

Every teaching chapter uses the same eight headings, a talk box with an exact opening question, and named activities inside the home and outside it. Inside each chapter you'll find two planning bands, **5–7** and **8–10**. Place your child by skill, not birthday. A careful six-year-old and a brand-new nine-year-old may start in the same place, and that's fine.

Where to begin

This week you can learn one sentence, "What do you think will happen?", and practice waiting after you say it.

Today your child can be the programmer, and you can be a very literal robot. The robot will do exactly what it's told. Exactly.

You don't need to be a programmer. You need a clear floor, a few arrows on paper, and a question you are willing to wait for.

That is enough to begin.

— Michael Gannotti

How to Use This Book This Week

Pick one idea. Run a few short Computing Hours. Take one outing. On Friday, do a small check and stop.

That is the honest lesson plan: a weekly shape you can repeat all year, not a 180-day calendar.

This week's shape

1. **Choose one focus.** In week one, it's the focus line itself: predict before you run; debug before you start over. Later weeks take one idea from a chapter: exact steps, a repeat, an if-then rule, a first ScratchJr script.
2. **Run three to five Computing Hours** of about 20–40 minutes each, using the five steps in **The Computing Hour**. At 5–7, nearer 20–30 minutes is plenty.
3. **Do one out-of-home try-it** when you can: a traffic-light walk, a barcode hunt, a library visit, a playground program, or an automatic-door detective errand.
4. **On Friday, do a short check** and read the chapter's "done enough" list. Ten minutes is enough.

If a day only holds ten minutes, do the unplugged warm-up and one prediction. That counts.

A first week you can start on Monday

- **Day 1 (about 20 minutes, no screen).** STEP-02 Robot Grown-Up. Your child gives you spoken steps to reach the couch. Ask "How many steps to the couch?" before you move. Follow the steps exactly. When you bump into something, find the wrong step together. Photograph the drawn steps, and ask for one sentence: "I predicted... It did... Next I'd change..." (AS-01).
- **Day 2 (20–25 minutes).** STEP-03 Arrow Card Path on a masking-tape grid, then BUG-01 Spot the Wrong Arrow. Start a Bug Jar (BUG-08): one token for every bug found.
- **Day 3 (25–30 minutes).** For 5–7, JR-01 Move the Cat in ScratchJr, a free tablet app for ages 5–7, for about ten minutes of screen.⁴ For 8–10, SCR-01 Tour Before You Touch in Scratch. Either way, the Expected/Actual card comes first, and screens go off at Save and Tell. No tablet? Repeat Day 2 with a longer path.
- **One errand.** OUT-05 Automatic Door Detective.

- **Friday (10 minutes).** AS-03 Debug on the Spot: you lay out a short card program with one wrong card, and your child finds it. Then AS-05 Portfolio Pick: your child chooses one photo for the folder and says why. Read the Chapter 1 “done enough” list.

Your own job this week is small and specific. Learn to ask “What do you think will happen?” and then wait while you count to five in your head. Learn the five step names of the Hour: Unplug First, Predict, Build and Run, Bug Hunt, Save and Tell.

Placement by skill, not birthday

The bands 5–7 and 8–10 are planning help, not birthday rules.

- **Reading.** A child who can’t yet read short instructions works with pictures, arrows, and spoken steps. That’s most 5–7s, and some older children too.
- **Hands.** Touch and drag-and-drop come before a mouse. Typing waits until about 8–10.
- **Experience.** A nine-year-old who is new to all of this starts with the 5–7 cards for a week or two, then moves up quickly. A six-year-old reader may borrow an 8–10 card.

Move a child between bands based on what they can show you, not on how old they are.

How to read a teaching chapter

Read **Why this matters** and **For the parent** first, alone if you can. It takes about twenty minutes. Finish with the five-minute parent warm-up.

How to teach it this week is your script for the table. The exact wording is there on purpose; use it until you have your own. The 5–7 and 8–10 moves live inside that heading.

Practice that actually builds learning holds the named try-its, each with time, materials, safety, the fun, the skill, and what to save. The **talk box** gives you the exact opening question, a few follow-ups, and what to do when the room goes quiet.

Read **For the student** with your child. Use **If it isn’t clicking** when something stalls. **What “done enough” looks like** tells you when to move on.

The activity banks are a menu

The book draws on 96 named activity cards, each with a short code such as STEP-02 or OUT-05. Sixty-three of them need no screen at all. Each chapter uses a handful. Pick one or two cards per Hour. Nobody does all 96, and nobody should try.

Low-device and no-device weeks

An unplugged week is a full week. Cards, tape, cups, and the library can carry most of a year. Once your child is ready, try to let them meet a real program most weeks, even if that happens at

the library or on a borrowed tablet.

Siblings and mixed ages

Use **driver and navigator**. The driver's hands are on the cards or the tablet. The navigator holds the Expected/Actual card, reads the plan, and points. Swap on a timer, every five minutes or so. The older child is a programmer too, not the unpaid tutor. Everyone joins the unplugged warm-up.

Multilingual homes

Steps can be said in any language, and bug words can be in the home language too. ScratchJr lists more than twenty interface languages.⁴

Safety first, even before Chapter 7

If your child already uses a tablet or phone, teach two short safety cards in the first week, even though Chapter 7 comes later: SAFE-02 Passwords Are Like Toothbrushes (the grown-up keeps the list) and SAFE-03 Stop, Tell, Close (for anything weird, scary, or asking for a name). A family device agreement (SAFE-06) can follow. **The Computing Hour** holds Family safety basics.

What to skip for now

- Scratch for a child who can't read yet. ScratchJr and picture cards do the same job.
- Typing drills at five.
- Any account for your child.
- Robot shopping. A person makes a fine robot. One floor robot is plenty later if you want one, and none is fine.
- AI tools, until you've read the AI box in **The Computing Hour**.

A year in outline (a suggestion)

Weeks	Chapters	What it feels like
1–5	1–2	Robot grown-up, card paths, big jobs into small jobs
6–10	3	Patterns, loops, if-then, events
11–13, then always	4	Bug Jar, Expected/Actual, one change at a time
14–20	5	ScratchJr or Scratch; one paper-planned project

Weeks	Chapters	What it feels like
21–25	6	Be the Computer, binary cards, keys, files
26–30	7	Messages in pieces, Stop-Tell-Close, family agreement
31–34	8	Sorting machine, bad examples, next-word guessing
35–36	9	One project shown to family; year summary

The order is a map, not a race. The bug habit from Chapter 4 runs from week one.

Records without drama

Date every photo. Crop it to the cards or the screen, not your child's face. Keep one folder, on paper or a family drive. That folder is the record. Chapter 9 turns it into a plain year summary.

If You Only Remember Five Things

Pin this page where you plan the week. When an Hour feels scattered, come back here before you add anything new.

1. Unplug First

Every Computing Hour starts away from the screen: a body, a card, a cup, a strip of tape. A child who has been the robot understands the cat on the tablet much faster. At 5–7, many Hours never need a screen at all. Once your child is ready, they should still meet a real program most weeks. Unplugged is the warm-up and the explanation, not the whole story.

2. Predict before you run

Before anything moves, ask: “What do you think will happen?” Then wait. Count to five in your head. A guess turns a program into an experiment, and it gives you the clearest window you’ll ever get into your child’s thinking. At 5–7 they say it or draw it. At 8–10 they write it on the Expected/Actual card. “I don’t know” can become a choice of two: “Will it stop here, or here?”

3. Debug before you start over

A bug is a mistake in the instructions. It is a clue, not a failure. When the robot bumps the table or the cat walks off the screen, the move is always the same: What did you expect? What did it do? Where did it first go wrong? Change one thing. Run it again. Celebrate the bug found with a token in the Bug Jar. Starting over from nothing teaches less than fixing one card. The standards schools use expect even kindergartners to spot the step that doesn’t work, so your five-year-old is doing real computer science when they point to the wrong arrow.²

4. Child’s hands, child’s program

The child predicts, builds, and debugs with their own hands. Yours stay in your lap. If you need to show something, show it on your own cards or your own project, then put it away and hand the turn back. When you feel the itch to grab the mouse, point with words instead: “Look near the top.”

The rescue order is ask, wait, hint, a short demo on your own cards, then hand it back. A program you fixed is your program. A program they fixed is theirs.

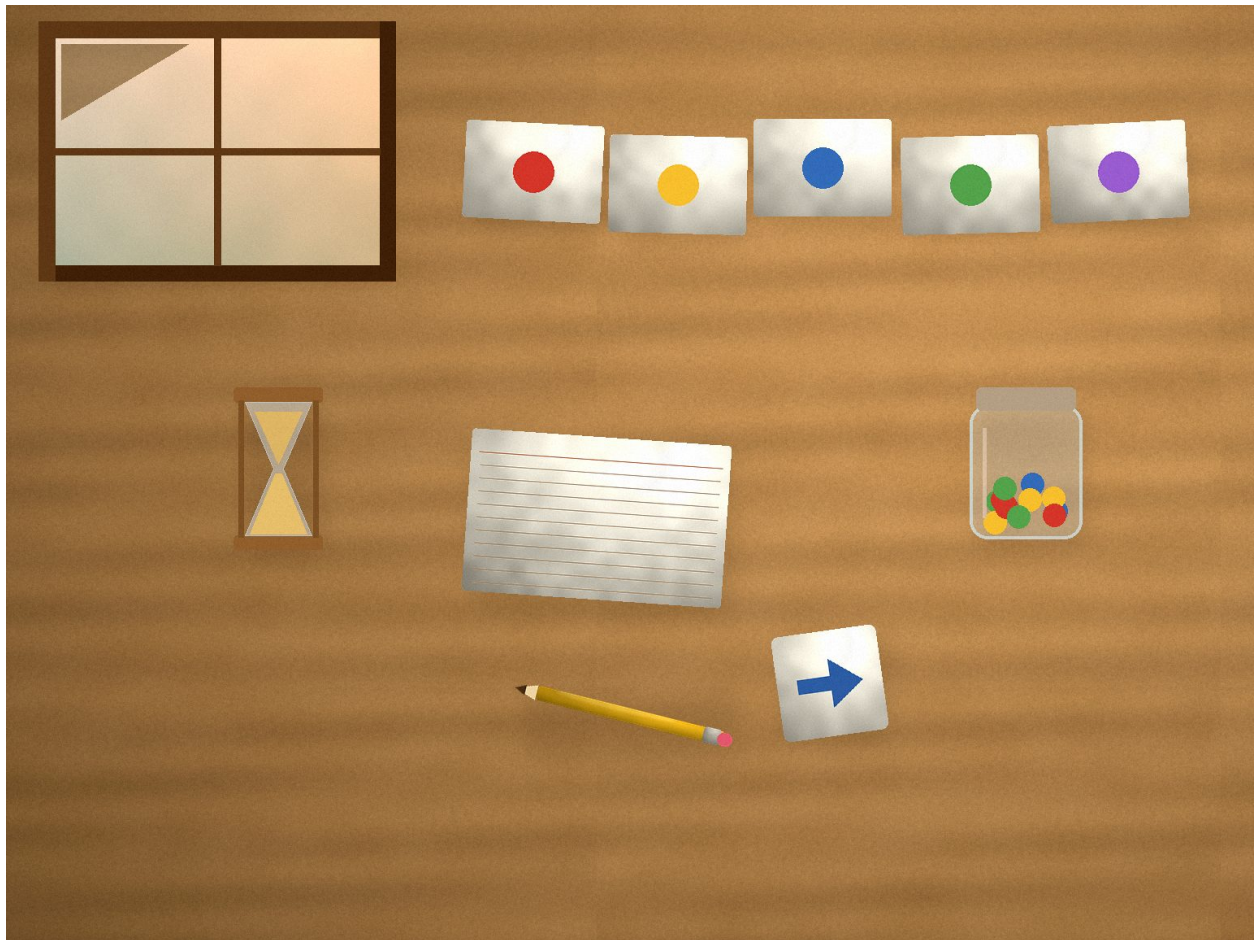
5. The grown-up holds the passwords and drives the AI

At 5–10, you create and keep every account and password. Most of this book needs no account at all. Devices live in a shared room, and screens go off about an hour before bed. Your child’s script for anything weird, scary, or asking for a name is three words: **Stop, tell, close**. AI tools stay on your account, with your hands on the keyboard. Children learn what AI is through games at the table, not through a chatbot of their own. Photos, voices, and names of your children stay out of AI tools. Anything a tool makes is “made by a tool,” never your child’s work.

These five are the whole book in miniature. Every chapter adds an idea, and every idea runs on these five habits.

The details live in one place: **The Computing Hour**, next, holds the session shape, the talk-box shape, Family safety basics, the screen-time note, and the AI box. Later chapters point back there in a sentence instead of repeating it.

The Computing Hour



Twenty to forty minutes. Five steps. Stop while it's still fun.

“Hour” is the family word. A real session runs about 20–40 minutes, and for 5–7 nearer 20–30. This is the session shape for the whole book. Later chapters say “run the Computing Hour” and point back here, so you only learn it once.

Parent warm-up (about three minutes, before the child sits)

Read today's talk-box opening aloud once. Set out the cards, tape, or cups, or open the app and check where the project will be saved. Check that the device is charged and in the shared room. Know which one or two activity cards you'll use. Then put your own phone face down.

The five steps

Step	Minutes (5–7 / 8–10)	What happens	What you say first
1. Unplug First	3–8 / 3–8	A body, card, cup, or tape warm-up on today's idea. The screen stays dark.	"Today you're the robot."
2. Predict	2–5 / 2–5	Before anything runs, the child says, draws, or writes what will happen.	"What do you think will happen — before we press go?"
3. Build and Run	8–12 / 10–15	The child builds a card program, a person-robot path, a ScratchJr or Scratch script, or a robot route, and runs it.	"Your hands, your program. I'm right here."
4. Bug Hunt	3–8 / 3–8	Compare the guess with what happened. Find the first wrong step, change one thing, run again.	"What did you expect it to do — and what did it do instead?"
5. Save and Tell	2–5 / 2–5	Save and name the file, or photograph the card program. Screens off. One short exit sentence, then tell someone else.	"Save it, name it, screen off — then tell me one thing."

1. Unplug First

Start with the body or the table. Be the robot (STEP-02), clap a pattern (PAT-01), follow a "Simon says if" rule (IF-01), or hunt the wrong arrow in a strip (BUG-01). Use it to bring back last week's idea too: "Show me last week's turn card." A child who has done the idea with their body understands it on a screen far faster.

2. Predict

The child says what will happen before anything runs. At 5–7 they say it or draw it on the Expected/Actual card (BUG-02): an index card with two boxes, one filled in before the run and one after. At 8–10 they write a line or two. A guess is a ticket to play. Wrong guesses are welcome; we’re testing ideas, not children.

3. Build and Run

The child’s hands are on the cards, the robot, or the blocks. Yours are in your lap. Run it. If the child wants to change things before the first run, let them finish and run it anyway. The run is the experiment.

4. Bug Hunt

The talk box lives here. Ask what they expected and what happened. Ask where it first went wrong. Change one thing. Run again. Every bug found earns a token in the Bug Jar (BUG-08): you celebrate the bug found, not the bug-free run. At 8–10, add a line to the Bug Log (BUG-06) and try Read Before You Run (BUG-07): trace the program with a finger before pressing go. Chapter 4 is all about bugs.

5. Save and Tell

Save the file with a name you’ll find again, today’s date first, or photograph the card program cropped to the cards. Screens off. Then the exit crumb (AS-01), said at 5–7 and written at 8–10: “I predicted... It did... Next I’d change...” Last, the child tells one sentence to someone else: a sibling, a grandparent, the cat.

Bands inside the Hour

5–7. Spoken predictions and pictures on the Expected/Actual card. One change per run. If there is a screen block, it sits inside Build and Run and lasts about 10–15 minutes. Many Hours have no screen at all. Stop while it’s still fun.

8–10. Written predictions, a Bug Log, and Read Before You Run. A project Hour in Scratch (SCR-05) may run about 25 minutes of screen time, with you nearby and a firm stop at Save and Tell. A project can stretch across two Hours.

Place by skill, not birthday.

The talk-box shape

Every teaching chapter has a talk box. The shape never changes.

The opening question is real. It asks about a prediction, a first step, what repeats, what was expected and what happened, which block does it, what went in and out, who made it, or what examples taught a machine. It has no single “right” recitation. Each chapter gives its exact opening.

Three to five follow-ups, one for each of these: - prediction: “What will it do?” - evidence: “What did you see that makes you say that?” - location: “Where did it first go wrong? Point to it.” - change: “What’s one thing we could change?” - revision: “You can change your guess. Let’s run it again.”

How to wait. After you ask, count to five in your head. That comes from classroom studies of “wait time,” used here as a pause habit, not a precise rule for kitchen tables.⁵ After the child answers, wait again. If they’re mid-build, don’t cut in. If the silence feels hard, look at the cards or the screen, not at the child.

What a stuck silence usually means: the question was vague; there was no wait; they’re afraid the bug means they’re “bad at it”; they’re waiting for you to fix it, because you have before; they skipped the prediction, so they have nothing to compare; or the screen is more interesting than the question. Offer a choice of two (“Is it this card or this card?”), ask them to point, or turn the tablet face-down for a moment. Your job is to ask, wait, and point back at the program.

The try-it shapes

Every named try-it in this book, inside or outside the home, gives you six things: **time, materials, safety and privacy, the fun, the skill**, and **what to save**. Each teaching chapter has at least one in-home try-it and at least one out-of-home try-it.

In the home: robot grown-up, arrow cards, a cup tower, clap-stomp loops, a sock-sorting machine, the Bug Jar, a ScratchJr cat, binary dot cards, being the computer, a message cut into pieces, Stop-Tell-Close role-play.

Outside: a traffic-light walk, a barcode hunt, a library visit, a playground program, an automatic-door detective errand. Hold hands near roads and follow the usual courtesy of the place. Outings are practice. They don’t replace predict, run, and debug at home.

When the Hour goes wrong

If the Hour looks like...	Try this
The child wants to skip to the screen	Keep Unplug First short and physical. Promise the screen and keep the promise.
The child presses go before saying anything	Hands on the table: “Tell me what it will do first.”
The child deletes everything at the first bug	“Bugs are clues.” Undo. Find the first wrong step together.
Five changes at once	Undo back to the last run. “Change one thing, then run it.”

If the Hour looks like...	Try this
You grab the mouse “just to show”	Hands back in your lap. Point with words. Demo on your own cards, then put them away.
A sibling takes over	Driver and navigator, swapping on a timer.
Tears at a bug	Name it a clue, add a Bug Jar token, and stop while it’s fixable.
“The robot won’t listen!”	“It listens perfectly. It does exactly what we said. That’s the lesson.”
You don’t know the answer	Say so. Predict together. Run it. That’s the method.
The device dies, or there’s no Wi-Fi	It’s an unplugged day. The Hour isn’t lost.
A login box appears	You handle it. Your child never types a password into a new site.
An AI helper pops up, or the child wants to “ask the AI”	Close it. “That’s a grown-up tool. We’ll look together at a time I choose.”

Family safety basics (said once)

These are ordinary and non-negotiable. Chapters repeat them in a line.

- **You hold every account and password.** Children under 13 don’t get accounts unless you create and keep them. Most of this book needs none. A federal law called COPPA requires services aimed at children under 13 to get a parent’s consent before collecting a child’s personal information, which includes photos, videos, and voice recordings.^{6,7} The law governs companies; it doesn’t make a site safe. Chapter 7 says more.
- **One shared device in a common room.** Screens go off about an hour before bed.
- **Stop, tell, close.** For anything weird, scary, or asking for a name, your child stops, tells a grown-up, and closes the lid or the app. No trouble, no shame.
- **First names or nicknames** in projects. No school names, addresses, or faces shared.
- **Photos cropped to the work.** Record the cards or the screen, not your child’s face.
- **Bodies and things.** Clear the floor and take slow steps in robot games. No knives and no stove in food activities, and check allergies first. An adult handles old devices; nobody opens batteries or power supplies. Small robot parts are a choking risk at 5–7, so check the age label. Hold hands and follow road rules on walks.

A note on screen time (said once)

The American Academy of Pediatrics replaced its older media guidance in 2026. Its new policy statement puts the weight on what children see and do, on using media together, and on protecting sleep, meals, movement, homework, and bedrooms. It suggests family-tailored limits, roughly “1 to 2 hours/d or more of entertainment (not school-related) media” for school-age children, and screen-free times such as the hour before bed.⁸

For five-year-olds, the CDC offers a simpler tip: no more than an hour of screen time a day, and no screens for an hour or two before bed.⁹ The World Health Organization asks families to limit recreational screen time for children 5–17 and to keep them moving at least an hour a day. It gives no number for screens.¹⁰

These are guidelines for families, not a stopwatch rule, and they change as the research does. The Computing Hour is school, mostly unplugged, and short. At 5–7 the screen block is 10–15 minutes at most, with you beside your child, and it ends with screens off.

Dose honesty

Three to five short Hours a week, across a year, is a real course at this age. A ten-minute unplugged Hour on a hard day counts. There is no evidence that more screen minutes make better young programmers. Short and frequent beats long and rare.

The life of the habit

Predict, run, debug, save, and tell spill out of the Hour on their own. “What do you think will happen if we add more water?” “Where did the brick tower first go wrong?” Notice it when it happens and say so in a sentence. That is the whole habit. It lives here, in each chapter’s “done enough” list, and in your records.

The AI box (ages 5–10, said once)

At 5–10, the grown-up drives. The child does the thinking, predicting, building, and debugging. You may use an AI tool on your own account, with the child beside you or after the Hour. The child never sits alone with a chatbot, an AI companion, a voice-assistant “friend,” or an AI toy. Independent testers have found that “kids modes” and talking toys can fail, so your hands on the device are the safety feature.^{11,12}

Ages 5–7. No AI chat for the child, even with you nearby. You may use a tool to prepare: have it explain today’s idea to you, invent unplugged puzzles, or write a hint script for after the try. Your child plays the unplugged AI games in Chapter 8.

Ages 8–10. Still your account and your hands. Your child may sit beside you, say a question aloud, and check the answer against a book. No account of their own. No AI helper inside their coding tool. No “make my game” prompts.

You may, on your account: - have it explain today’s idea to you, from a page you can open and check; - make extra puzzles, with the answers hidden for you; - write a hint ladder you’ll use after your child has tried; - look at a screenshot of a saved project’s code (the code, not the child) so you understand a bug before you ask questions; - brainstorm unplugged activities; - make a picture of an object or animal for the “spot the made-up picture” game in Chapter 8. Never a person, and never a child.

Please keep these off the table: - having it write your child’s program, game, or story; - pasting a puzzle in for the answer while your child is still trying; - turning on an AI helper inside your child’s coding tool; - letting your child chat alone with any chatbot, companion, voice “friend,” or AI toy, kids mode or not; - uploading your child’s photo, voice, or name; - filing generated code, pictures, or stories as your child’s work (they are “made by a tool”); - using an AI “detector” to judge your child; detectors make mistakes, so they have no place here;¹³ - trusting a study a tool cites before you’ve opened it yourself, because tools sometimes invent citations;¹⁴ - handing your child an agent, a tool that can act on a computer by itself.

One caution from a different age. In a study of high-school math students, those given an unrestricted chatbot did better on practice problems and worse on a later test without it.¹⁵ That is a different age and a different subject, so it isn’t evidence about 5–10 or about computing. The rule it supports is the same one this book uses anyway: the child builds and debugs first.

Accounts and terms. Ages 5–10 are under 13. Most consumer AI services set 13 as their minimum age, and some set 18.¹⁶ Google lets a parent turn on Gemini for a supervised under-13 account.¹⁷ This book still keeps children 5–10 off chatbots. Terms change, so check the current ones before you rely on them.

The Grok Bot caution. If you use Grok Bot, all of your bots share one computer, so keep photos of your children and family off it. Keep your child’s records on a family drive, not on that computer. Its documentation describes no education version and no child account.¹⁸

Two more names. An AI helper inside a coding course is still an AI tool; it stays off or parent-run. The Hour of AI, from CodeAI (formerly Code.org) and its partners, is a set of activities, not a chatbot for children. Some activities include an AI helper, so preview them and pick unplugged or non-chat options.^{19,20,21}

Later chapters point back to this box in one sentence.

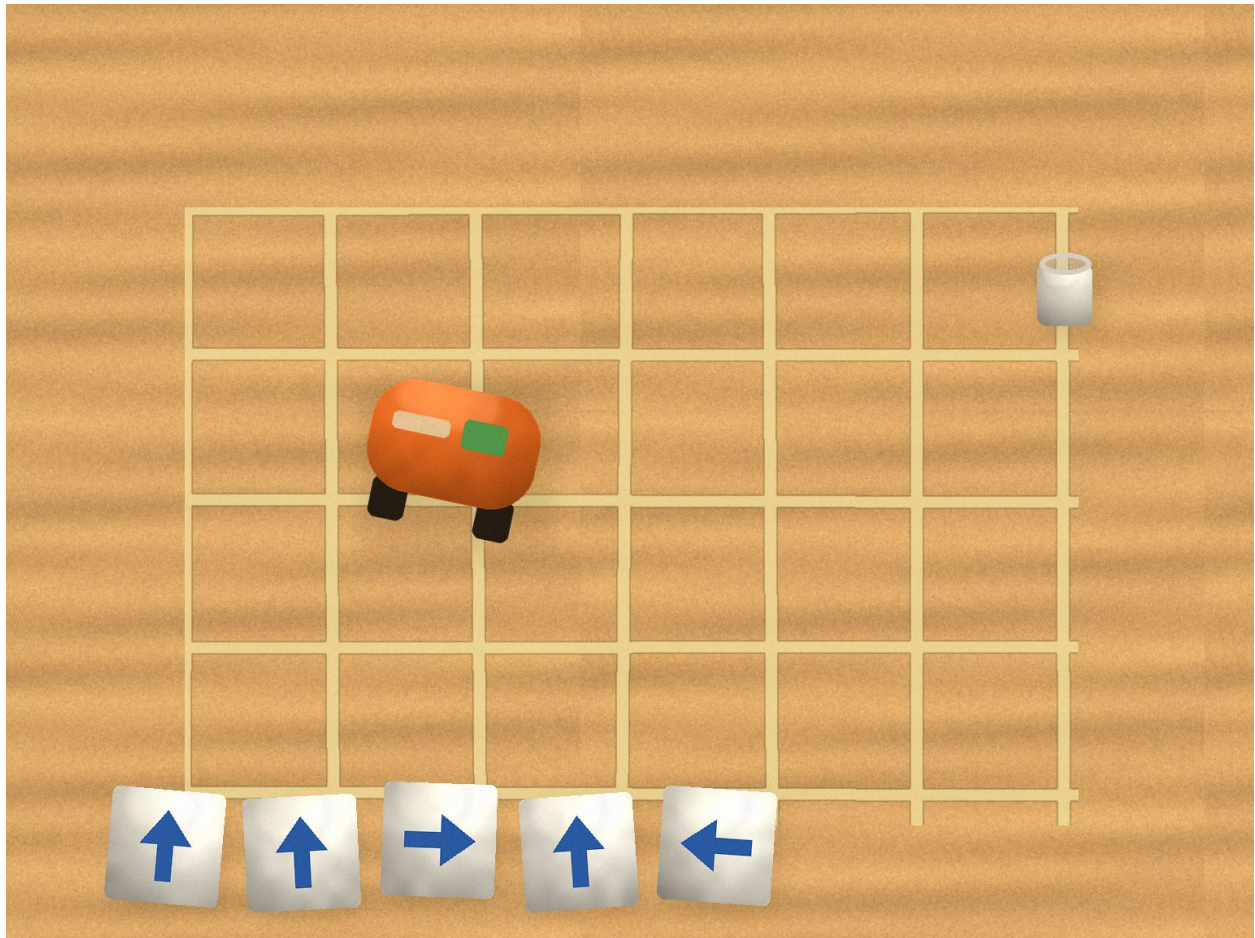
Five things to say

1. **Opening the Hour.** “It’s Computing Hour. Today we’re going to be robots. First, no screens — just us.”
2. **The predict pause.** “Before we press go — what do you think will happen?” (Count to five in your head.) “Take your time. A guess is fine. Guesses are how programmers start.”
3. **The bug moment.** “Oh, interesting. That’s a bug. What did you expect it to do? What did it do instead? Let’s find the very first place it went wrong.”
4. **Save and stop.** “Let’s save it with a name we’ll find again: today’s date, then what it is. Screen off. Now tell me one thing you figured out.”
5. **The grown-up tool line.** “That’s a grown-up tool, like the oven. I use it sometimes, and when we look together, my hands are on the keyboard. Your ideas stay yours.”

Ready to begin

You have the five steps, the talk-box shape, the try-it shapes, Family safety basics, the screen-time note, and the AI box. Chapter 1 puts the focus line to work. Clear a path to the couch, practice “What do you think will happen?” once, and sit down.

Chapter 1



What Computing Looks Like at 5–10

Why this matters

Here is a picture. It is an illustration, not a real family.

A seven-year-old can find any video on the family tablet in about four seconds. One morning her mother asks her to tell her little brother, step by step, how to pour a bowl of cereal. He has agreed

to do exactly what she says and nothing else.

“Get the cereal and pour it in the bowl,” she says.

He picks up the box and holds it over the bowl. Nothing comes out. “You didn’t say open it.”

She laughs, and then she thinks. “Open the top. Then tip it. No, *slowly*.” Three tries later there is cereal in the bowl and a small pile on the table, and she has learned something that no amount of swiping taught her: a machine does what you say, not what you mean.

That gap is where this book lives. Using a computer and telling one what to do are two different skills. Children pick up the first one by being around devices. Almost nobody picks up the second one by accident. It has to be taught, and it can be taught at five.

At this age, computing means two habits, and they are the spine of the whole book:

Predict before you run. Before anything moves, say what you think will happen. A guess turns a program into an experiment. Without a guess, there is nothing to compare, and a mistake is just a surprise.

Debug before you start over. When it goes wrong, and it will, find the first step that went wrong and change that one thing. A bug is a mistake in the instructions. It is a clue, not a failure. Sweeping every card off the table teaches less than fixing one.

Those two habits unlock everything that follows. A child who predicts and debugs can meet a floor robot, a ScratchJr cat, a traffic light, or a voice assistant with the same calm question: what is it supposed to do, and what is it actually doing?

Across the year, your child will meet a small set of big ideas, each in its own chapter: exact steps, patterns and repeats, if-then decisions, bugs as clues, first block programs, what’s inside the machine, how the internet carries messages, and what AI is. This chapter gives you the whole year in miniature, and it gives your child the first taste of being the boss of a very literal machine.

Why is it worth the struggle? Because the struggle is the skill. Waiting to hear a guess is hard. Watching the robot walk into the wrong room without jumping in is hard. Finding the first wrong step instead of the most obvious one is hard. Each of those is a small act of patience and attention, and your child gets to practice it in a game where mistakes are funny instead of frightening.

You don’t need to be a programmer. You do need to ask what will happen before anyone presses go, and to leave the child’s program in the child’s hands.

For the parent: understand it yourself

Many adults feel unsure about computers, and some feel their children already know more about devices than they do. That’s normal. It doesn’t stand in your way here. Your job this week isn’t to learn to code. It is to understand one idea well enough to hear a wrong turn, and to ask one good question without grabbing the mouse or filling the silence with the answer.

An everyday picture

Picture a babysitter who follows your note to the letter and does nothing else. The note says, “Bath at 6:30. Bed at 7.” At 6:30 the babysitter runs a bath. Nobody said to put the child in it. At 7 the child is in bed in the day’s clothes, because nobody mentioned pajamas.

That babysitter is a computer. Every instruction is followed exactly, in order, and anything you left out simply doesn’t happen. The note isn’t wrong, exactly. It is missing steps you thought were obvious. Most bugs are like that.

A precise picture

A **program** is a list of instructions written in a language a machine can follow. An arrow card is a tiny language. So is a ScratchJr block. The machine runs the instructions in order, exactly as written, and very fast. It doesn’t know what you meant. It only knows what you said.

When the machine does something you didn’t expect, there is a mismatch between your instructions and your intention. That mismatch is the **bug**. A prediction makes the mismatch visible: you said the robot would reach the couch, and it reached the coffee table. Debugging means finding the first place where what happened stopped matching what you predicted. That’s where the wrong instruction usually lives. Change that one thing, run it again, and compare.

That’s the whole method, and it doesn’t get more complicated at ten. The programs get longer. The habit stays the same.

Computing is not the same as using a device

The Computer Science Teachers Association, which writes the standards many schools use, says it plainly. Computer science “is not” using technology tools “like word processors, slide decks, or generative AI tools.” It is also “not limited to coding or programming.”¹

That helps you sort your child’s week honestly. A child who swipes quickly through a tablet is using a device. A child who sorts socks by a rule, lays arrow cards to a goal, and finds the wrong arrow is doing computer science, even with the tablet in a drawer.

Typing, saving files, and staying safe online are real skills too. Your child needs them, and Chapters 6 and 7 teach them. They are just a different kind of skill from telling a computer what to do. Keeping that straight now keeps your records honest later.

You may hear that children pick up computers on their own. Swiping is a real skill, and many children are good at it. It isn’t the same as understanding what the machine is doing, and it doesn’t tell a computer anything new. That part your child learns from you, a little at a time.

The big ideas, in plain words

You may see the phrase *computational thinking* on program websites. It means the habits underneath computing: breaking a big problem into smaller parts, writing exact steps, noticing patterns, and finding and fixing mistakes.^{22,23,24} In this book we just use those plain words.

Here is the year, idea by idea:

- **Steps and algorithms.** An algorithm is exact steps to reach a goal. Order matters, and nothing can be skipped. (Chapter 2)
- **Patterns, loops, and if-then.** A loop means “do it again.” If-then means “only when this is true.” Events mean “when this happens, start.” (Chapter 3)
- **Bugs and debugging.** Expected versus actual, the first wrong step, one change at a time. (Chapter 4)
- **First programs.** Snapping blocks together on a tablet or computer, and optional robots. (Chapter 5)
- **Computers up close.** Input is what goes in, output is what comes out, and storage is where it’s kept. Plus mouse, keys, and files. (Chapter 6)
- **The internet.** How a message travels in pieces, and how to stay safe and kind. (Chapter 7)
- **AI.** A computer program that learned patterns from lots of examples, and why it guesses wrong. (Chapter 8)

What children can do at 5–10

The 2026 CSTA standards lay out a ladder, with a combined pre-K and kindergarten band and then one grade at a time through fifth grade.¹ Three rungs show how modest and concrete it is.

For pre-K and kindergarten, children “carry out algorithms in daily activities” and “identify a step in a sequence of commands that does not work as expected.”² That is debugging at five: pointing to the card that sent the toy the wrong way.

By second grade, children “debug a program that includes sequence, events, and iteration,” which means steps in order, things that start when something happens, and repeats.²⁵

By fourth grade, they “debug a program incrementally and repeatedly throughout the development process.”²⁶ In plain words: test a little, fix a little, test again, all the way through.

Read this as a map, not a checklist. It tells you what is reasonable to expect, so you can stop worrying about whether your five-year-old is “ready.” If they can follow a game’s rules and tell a story with a beginning and an end, they’re ready to be the programmer while you play the robot.

Unplugged first, then a real program

Every Computing Hour starts away from the screen, and there are good reasons for that.

The CDC’s milestone checklist says that by five, most children can pay attention to an activity for 5 to 10 minutes, during things like story time or crafts. It adds a note in parentheses: “screen time does not count.”⁹ A five-year-old’s best attention lives in their hands and body, so the Hour is built from short, hands-on blocks.

A review of coding studies with children from 4 to 16 found the clearest benefits for the youngest children came from structured activities and hands-on ones, such as programmable floor robots. Open-ended block coding showed more benefit with older children.²² One classroom study with seven-year-olds found that starting with unplugged activities and then moving to the screen worked

well.²⁷ Neither is a promise about your kitchen table, but both point the same way: body, then symbols, then screen.

Unplugged forever would miss something, though. A child should feel the real thing: a program that runs exactly as written, on a machine that doesn't care how hard you tried. So once your child is ready, they meet a real program most weeks.

How do you know they're ready? When your child can predict what a short strip of four or five arrow cards will do, and can point to the wrong card when it goes astray, they're ready for ten minutes with a tablet. At 5–7 that usually means ScratchJr, a free tablet app built for ages 5–7.^{4,28} At 8–10 it may mean Scratch. Chapter 5 walks you through both. The screen is the third step, not the reward.

What the evidence says, honestly

You may have heard that coding makes children smarter, or better at reading and math. Here is what the research behind this book actually found.

Coding teaches coding. In a large randomized trial with 1,057 children in kindergarten through second grade, the ScratchJr-based curriculum Coding as Another Language improved children's programming skills. The same trial found no difference on a broader test of thinking skills.²⁹ A related second-grade study found the same pattern for programming and thinking, and no difference in reading scores either.³⁰ The researchers who built the curriculum ran both studies.

Wider benefits are possible, and smaller than the headlines. A large review of programming studies across many ages found some spillover into skills like problem solving and creative thinking. The effects shrank when the comparison group was also doing something worthwhile, and school achievement and literacy benefited least.³¹ This is a useful study, not a promise that every home will see the same result. Older research found the same caution: planning skills didn't carry over from programming on their own.³²

So teach computing because it is worth knowing. Children live among machines that follow instructions, and they deserve to understand them. If patience, planning, or a calmer response to mistakes grows along the way, welcome it as a bonus, not as the reason.

There is also no deadline. Nothing in the research behind this book points to a window for programming that closes at five or seven. A nine-year-old who starts this year with the 5–7 cards will move up quickly. Starting late is just starting.

Wrong answers you should be able to hear

1. **"It's broken."** Usually means the child expected the machine to know what they meant. Reply: "It's doing exactly what we told it. Let's find the step."
2. **"Just press it!"** The prediction got skipped. Without a guess, a bug is only a surprise. Reply: "Guess first. A guess is your ticket."
3. **Sweeping every card away at the first bug.** Usually means the bug felt like failure. Reply: "Bugs are clues. Where did it *first* go wrong?" Put the cards back and point at the top.

4. **“The robot doesn’t listen to me.”** Usually means the child is blaming the machine for a missing step. Reply: “It listens perfectly. It just can’t guess.”
5. **“I’m not good at computers.”** Usually means a bug has started to feel like a verdict. Reply: “Nobody’s born knowing this. You found a bug today. That’s what programmers do.”

You’ll meet a few more later: five changes at once (Chapter 4), steps in favorite order instead of time order (Chapter 2), a name typed into a pop-up (Chapter 7), and a generated picture called “mine” (Chapter 8). Hear them early so they don’t surprise you.

Your five-minute warm-up

Before tomorrow’s Hour, alone, walk the path from the kitchen doorway to the couch. Count your steps and notice every turn. Then say aloud, once: “What do you think will happen — before we press go?” Count to five in your head after you say it. That silence is the hardest part of the Hour, so practice it while nobody’s watching.

How to teach it this week

This week’s Hours all run on the five steps from **The Computing Hour**. Here is how they fit this chapter’s first activity:

1. **Unplug First** (3–8 minutes). You become the robot. Clear the floor.
2. **Predict** (2–5 minutes). Your child says or draws the steps and says where you’ll end up.
3. **Build and Run** (8–12 minutes). Your child gives the steps. You follow them exactly.
4. **Bug Hunt** (3–8 minutes). Where did the robot first go wrong? Change one step. Run again.
5. **Save and Tell** (2–5 minutes). Photograph the drawn steps. Your child tells someone one thing they fixed.

Exact words you can say

- Opening: “Today you’re the programmer and I’m the robot. I’ll do exactly what you say. Exactly.”
- Before the run: “What do you think will happen — before we press go?”
- If the child says “I don’t know”: “Will I end up on the couch, or somewhere else?”
- During the run, when an instruction is unclear: “Robot does not understand. Please give one step.”
- After a bump: “Beep. Bump. What did you expect to happen? What happened instead?”
- At the bug: “Point to the step where I first went the wrong way.”
- After the fix: “Shall we run it again?”
- At the end: “Tell me three things: I predicted... It did... Next I’d change...”

In-home — STEP-02 Robot Grown-Up

The signature first Hour of the book.

- **Time:** about 10 minutes of play inside a 20-minute Hour.
- **Materials:** a clear path across a room; paper and a crayon for drawn arrows; optional three paper cups as a target.
- **Safety and privacy:** clear the floor of toys and cords first. The robot takes small, slow steps and stops before anything hard or sharp. If a photo is taken, crop it to the paper.
- **The fun:** a grown-up who has to do exactly what a child says, including the silly parts.
- **The skill:** giving precise instructions in order.
- **What to save:** a photo of the drawn steps, and one instruction your child fixed.

How it goes. Stand in the doorway. Your child's goal is to get you to the couch (or to the three cups on the floor). Ask for a prediction first: "How many steps to the couch?" Then your child gives instructions, one at a time, and you follow them with kind, literal precision. "Walk" means walk until told to stop, slowly. "Turn" with no direction means you stand there blinking: "Which way?" "Go to the couch" gets "Robot does not understand. Please give one step."

When you go astray, stop, and leave the fixing to your child. Ask where you first went the wrong way, and let your child change that one instruction. Then go back to the doorway and run the whole program again from the start. Programs run from the beginning every time, and that small ritual teaches it.

Stuck pattern. The child giggles so hard they can't give instructions. That's fine. The giggles are the lesson: they've noticed the robot can't guess. Let it run its course, then say, "Robot is ready. First step, please."

How to fade help

- **Day 1.** You may hint at the type of step that's missing: "I think the robot needs a turn somewhere."
- **Day 2.** Ask only where it first went wrong. Let your child find the type of step.
- **Day 3.** Ask only the opening question and "Shall we run it again?" Your child runs the Bug Hunt.
- **Any day.** Hands stay off the cards and the paper. If you must show something, use your own paper, then put it away. The rescue order is ask, wait, hint, a short demo on your own cards, then hand it back.

When to stop talking

After you ask for a prediction, wait. After your child answers, wait again. During the run, the robot speaks only in robot. During the Bug Hunt, one question at a time. If your child is studying the arrows, that silence is work, so let it be.

Notice specifically instead of praising generally. "You added a turn before the last step, and that's exactly where I went wrong" teaches more than "Good job."

Age-band moves: 5–7 and 8–10

5–7. Predictions are spoken or drawn. Keep programs short: three to six steps. Your child may point and say the steps while you draw the arrows. Use one change per run. Stop the Hour while they still want one more go. A five-year-old who can direct you to the couch and fix one step has had a complete Hour.

8–10. Predictions are written before the run on the Expected/Actual card. Programs can be longer, with obstacles. Add a twist: your child writes the whole program first, then you run it without any talking. Later in the week, preview Chapter 2’s Two Ways to the Door (STEP-07): your child writes two different programs to reach the same goal, and you both decide which is clearer. If the habits are shaky, stay with the 5–7 moves for a week. That’s placement by skill, not a step backward.

This week at a glance

Day	Focus	Try-it
Mon	Precise instructions	STEP-02 Robot Grown-Up
Tue	Find the first wrong step	BUG-01 Spot the Wrong Arrow on a short tape path; start the Bug Jar (BUG-08)
Wed	Guess, then check	BUG-02 Expected / Actual Card before a first ScratchJr or Scratch look (JR-01 or SCR-01, from Chapter 5), or before a new robot path if you have no tablet
Thu	Machines follow instructions	HW-01 Input–Output Hunt
Errand	Predict the machine	OUT-05 Automatic Door Detective
Fri	Short check	AS-01 Exit Crumb Trio, AS-03 Debug on the Spot, AS-05 Portfolio Pick, and the “done enough” list

Practice that actually builds learning

The try-its below run inside and outside your home. Each lists time, materials, safety and privacy, the fun, the skill, and what to save. Pick one or two per Hour. Errands and walks *are* practice, and they never replace the predict-run-debug move at home.

In-home — BUG-02 Expected / Actual Card

- **Time:** about 5 minutes, inside any Hour.

- **Materials:** an index card divided into two boxes labeled “Expected” and “Actual”; a pencil or crayon.
- **Safety and privacy:** none beyond the activity it’s used with. Write a first name only.
- **The fun:** a secret guess written down before anyone knows the answer, then revealed.
- **The skill:** making a prediction concrete, so there’s something to compare.
- **What to save:** the card itself, dated.

How it goes. Before the run, your child fills in the Expected box: a drawing of where the robot will end up (5–7) or a sentence (8–10). After the run, they fill in Actual. If the two boxes match, celebrate the prediction. If they don’t, you have a bug to hunt, and you know exactly where to start: the difference between the boxes. This card shows up in every chapter of the book.

In-home — BUG-01 Spot the Wrong Arrow

- **Time:** about 5 minutes.
- **Materials:** five or six paper arrow cards (forward, turn left, turn right), a small toy, and a short strip of masking tape on the floor as a path.
- **Safety and privacy:** keep the toy and cards out of walkways.
- **The fun:** you, the grown-up, made a mistake on purpose, and your child gets to catch it.
- **The skill:** finding the first wrong step.
- **What to save:** a photo of the strip with the wrong card circled or flipped.

How it goes. Lay out a short arrow strip that should take the toy along the path, with one wrong card in it. Ask for a prediction: “Where will the toy end up?” Your child moves the toy one card at a time. When it leaves the path, ask, “Which card sent it the wrong way?” Your child points, and swaps that card. Run it again from the start. Chapter 2 builds full arrow-card programs. This week, just hunt.

Stuck pattern. The child picks the last card instead of the first wrong one. Run it again, slowly, and stop after each card: “Is the toy still on the path?”

In-home — BUG-08 Bug Jar

- **Time:** ongoing, a few seconds per bug.
- **Materials:** a jar and some tokens, such as buttons, dried beans, or paper squares.
- **Safety and privacy:** at 5–7, use tokens too big to swallow.
- **The fun:** the jar fills up because of mistakes.
- **The skill:** treating a bug found as a win.
- **What to save:** at the end of the week, count the jar and name one bug out loud.

How it goes. Every time anyone finds a bug, child or grown-up, a token goes in the jar. You celebrate the bug found, not the bug-free run. When you find your own mistake in front of your child, say “Oh, a clue!” and drop a token in. Children believe what you do far more than what you say.

In-home — HW-01 Input–Output Hunt

- **Time:** about 10 minutes.
- **Materials:** the household: a doorbell, a light switch, a microwave, a tablet, a toaster.
- **Safety and privacy:** look and talk only. An adult operates anything that heats; nobody touches the stove or opens a device.
- **The fun:** a scavenger hunt for machines hiding in plain sight.
- **The skill:** noticing that machines take something in and give something back. Input is what goes in; output is what comes out.
- **What to save:** two input-output pairs, said aloud or drawn (“button in, ding out”).

How it goes. Walk the house together. At each machine, ask what goes in and what comes out. Press the doorbell: finger in, sound out. Then the key question for this chapter: “Who told it to do that?” A person wrote the instructions. Machines follow instructions that people wrote. Chapter 6 goes much further with this.

Out-of-home — OUT-05 Automatic Door Detective

- **Time:** a few minutes during an ordinary errand.
- **Materials:** a store, library, or clinic with automatic doors, hand dryers, or an elevator.
- **Safety and privacy:** hold hands in the parking lot. Stand to the side of the doorway so others can pass. Test the door once or twice, not ten times. Keep fingers clear of closing doors.
- **The fun:** being a detective who figures out what makes the door “decide.”
- **The skill:** predicting what a machine will do, and noticing a simple if-then rule.
- **What to save:** one sentence in the form “If , *then* ,” said in the car or written at home.

How it goes. Before you reach the door, stop and ask for a prediction: “What will make the door open? How close do we have to get?” Walk up slowly and watch. Did it open when your child expected? Too early? Too late? In the car, put it into words: “If someone comes close, then the door opens.” Try a hand dryer the same way. Chapter 8 comes back to these machines to ask which ones follow a rule and which ones learned from examples.

Talk box (this chapter)

Opening: “What do you think will happen — before we press go?”

Follow-ups: 1. Prediction: “What will the robot do after this step?” 2. Evidence: “What did you see that makes you say that?” 3. Location: “Where did it first go wrong? Point to it.” 4. Change: “What’s one thing we could change?” 5. Revision: “You can change your guess. Shall we run it again?”

How to wait. Count to five in your head after you ask. Here is a move for this chapter: let your child hold the “go” card, or stand ready to say “Go,” but they can’t use it until they’ve said a guess. The guess becomes their ticket. If the silence feels long, look at the arrows, not at your child.

What a stuck silence usually means. The question was too big (“Why didn’t it work?”), so shrink it: “Is it this step or this one?” Your child may be afraid a wrong guess is a failure, so make

a deliberately silly guess yourself first. They may be waiting for you to fix it, because you have before; put your hands in your lap and wait. Or the guess was skipped, so there's nothing to compare. Go back and make one.

Blocked and mixed practice

Blocked practice for the new move. This week, every Hour includes a prediction before the run and a first-wrong-step hunt after it. Same move, different programs: a robot path on Monday, an arrow strip on Wednesday, a door on the errand.

Mixed practice. By Friday, mix it up. Ask for a prediction about something that isn't a program at all: "What will happen to this ice cube on the windowsill?" Or plant a bug in an ordinary routine, like a list of bedtime steps with "brush teeth" after "lights out." Your child is learning when to use the move, not just how.

One incorrect example to diagnose

Here's an illustration, not a real family. A parent sets up Robot Grown-Up. The child shouts steps without stopping; the parent, wanting to help, quietly steers toward the couch. The robot "arrives." Nobody predicted where it would end up, and nobody looks at the steps. The child says the robot "just knew."

Diagnosis. No prediction, a robot that guessed, and no Bug Hunt. The child learned that machines know what you mean, which is the opposite of the lesson.

Rescue. Next Hour, draw the steps first. Ask for a prediction. Be truly literal, even if that means walking into the wall (gently). When you go wrong, ask where. The bump is the best part.

For the student

You are going to learn to boss a computer around, politely and exactly.

Here's the secret: computers do what you *say*, not what you *mean*. If you leave out a step, the computer leaves it out too. That isn't the computer being silly. That's how computers work.

A tiny example. You want your robot (a grown-up) to reach the couch. You say: "Walk forward. Stop. Sit." The robot walks forward, stops, and sits on the floor, because the couch is around the corner. You forgot the turn! So you find the place where it went wrong. You add one step: "Turn right." You run it again from the start. This time the robot lands on the couch.

That missing turn was a **bug**, which means a mistake in the instructions. Finding it made you a **debugger**. Programmers find bugs every day. A bug is a clue, not a failure.

Try this today. Be the programmer. Get a grown-up to be your robot. Before they move, say what you think will happen: "You'll end up on the couch." Then give your steps, one at a time. If the robot goes the wrong way, don't start over. Find the *first* wrong step and fix just that one.

Explain it back. Tell someone: Why did the robot sit on the floor instead of the couch? What did you change?

Challenge. Write or draw your steps on paper first, then hand the paper to your robot. No talking allowed while the robot follows it! Did your paper program work the first time? If not, where did it first go wrong?

You're allowed to guess wrong. Guessing is how programmers start.

If it isn't clicking

Your child won't predict. "I don't know." A guess may feel like a test they could fail. Next move: offer two choices ("Couch or coffee table?"), or make a silly guess yourself first ("I think the robot will end up in the bathtub"). Any guess is a ticket to play. If guessing still feels too big, ask them to point to where they think the robot will stop.

The child gets upset when the robot goes wrong. The bug may feel like a verdict on them. Next move: make the robot the funny one. "Beep. Robot confused. Robot needs help." Drop a token in the Bug Jar right away. Keep programs very short, three steps, so the fix is easy and quick. If tears come, stop and come back tomorrow. Stopping isn't failing.

You feel unsure you're doing it right. That's common in week one, and it usually means you're trying to have the answers. Next move: remember that your job is the questions. Ask for the guess, wait, run it, ask where it went wrong. That's the whole method. If you don't know the answer, say so and run it to find out together.

When to slow down. Your child skips the guess every time, or sweeps the cards away at every bug. Stay with Robot Grown-Up and very short programs for another week.

When to go ahead. Your child guesses before the run without being asked, and points to the first wrong step on their own. Move on to Chapter 2.

When to step back to unplugged. If a screen session turns into tapping with no guesses, put the tablet away and be the robot for a few days. Correct the wrong reason kindly ("The cat isn't broken; it did what the blocks said"), without crushing the try.

Tools, including AI

No tools are needed this week beyond paper, a crayon, a jar, and a clear floor.

If you want a first look at a real program on Thursday, ScratchJr is a free tablet app for ages 5–7, and Scratch is free for 8–10.^{4,28} If you prefer a structured course, CodeAI (formerly Code.org) offers free elementary courses, including a version for children who don't yet read.^{33,21} CS Unplugged, from the University of Canterbury in New Zealand, offers free activities with cards, string, and crayons.³⁴ Chapter 5 covers screen tools properly, and Chapter 10 lists resources by fit.

AI is a topic later in the year (Chapter 8), not a helper for your child. For this week, if you use an AI tool at all, use it on your own account to plan, for example to invent three more robot-path

challenges for your living room. The full rules are in the AI box in **The Computing Hour**.

What “done enough” looks like

Before moving on, check:

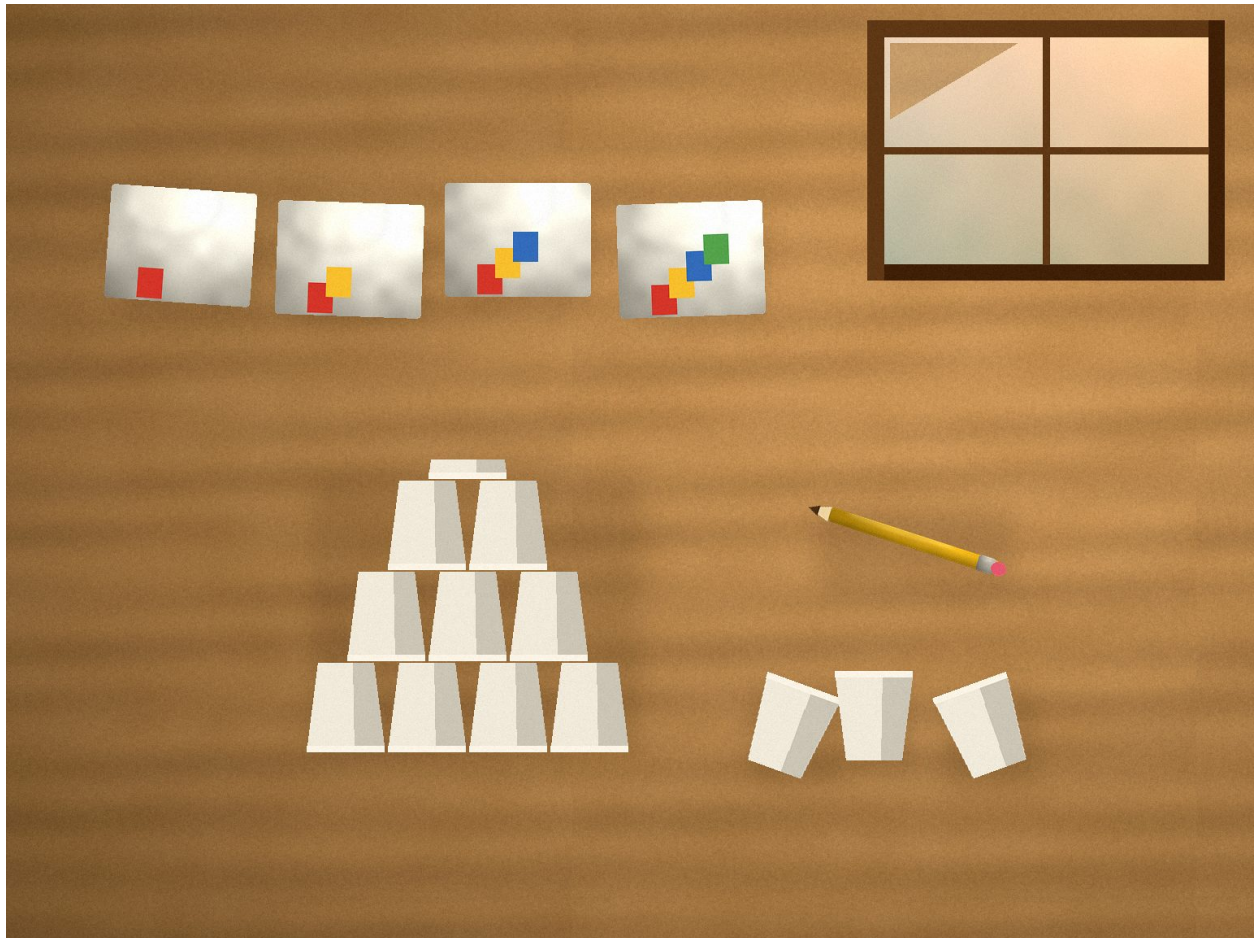
- ☐ Your child made a prediction before running something, at least three times this week (out loud, drawn, or on the Expected/Actual card).
- ☐ Your child found one wrong step in a card program or robot path and changed just that step (Debug on the Spot, AS-03).
- ☐ You asked “What did you expect?” and waited, at least once, without fixing it yourself.
- ☐ The Bug Jar has at least one token, and someone named the bug.
- ☐ Your child gave an exit crumb at least once: “I predicted... It did... Next I’d change...” (AS-01).
- ☐ One photo or card went into the folder, chosen by your child with a reason (Portfolio Pick, AS-05).
- ☐ One errand or outing included a prediction about a machine.

Place by skill, not birthday. A nine-year-old who needed three tries at the guessing habit isn’t behind. A six-year-old who loved every bug isn’t skipping ahead. Both are ready for Chapter 2 when the boxes above are checked.

A program’s course name, like “Course C,” describes that program’s scope. It isn’t a grade and it isn’t a line on a transcript. If you keep records, this week’s photos and cards live under **Computer Science**.

Keep the habit going: next week, every Hour still starts with “What do you think will happen?”

Chapter 2



Steps and Algorithms

Why this matters

Here's a picture. It's an illustration, not a real family.

On a Saturday morning, a parent draws the family's routine on six index cards: wake up, get dressed, brush teeth, eat pancakes, put on shoes, go to the park. She shuffles them and hands them to her

five-year-old. “Can you put these in order?”

He puts “eat pancakes” first. Pancakes are the best part. Then “go to the park,” because that’s the second-best part. His mother doesn’t correct him. She just reads it back, one card at a time, acting it out: eating pancakes in pajamas, walking to the park in pajamas, putting on shoes after she gets home. He is laughing before she reaches the end. “No! Dressed first!”

Later that week, a strip of masking tape on the kitchen floor becomes a grid of squares. A toy mouse sits in one corner. A paper “cheese” sits in another. A row of arrow cards tells the mouse how to get there. When the mouse ends up under the table instead, there’s one card to find.

That’s this chapter. An **algorithm** is exact steps to reach a goal. A recipe is an algorithm. So are the directions to Grandma’s house, and so is every program on every computer your child will ever use. Computers need every step, in the right order, said clearly. They can’t fill in what you left out, and they can’t guess what “over there” means.

Learning to write good steps unlocks the whole rest of the book. Loops in Chapter 3 are steps that repeat. Bugs in Chapter 4 are steps that went wrong. Block programs in Chapter 5 are steps written in blocks. If your child can write steps that someone else can follow, they can program.

It’s also worth the struggle for a quieter reason. Writing exact steps makes you notice what you actually do. Most of us have never thought about how many separate moves it takes to get dressed. A child who has to tell a robot each move starts to see that big jobs are made of small ones, and small ones can be done one at a time. That’s a gift for tidying a room as much as for programming.

You don’t need to be a programmer to teach this. You need a stack of cards, a strip of tape, and the patience to follow instructions exactly as given, even when they’re wrong.

For the parent: understand it yourself

You already know more about algorithms than you think. Every time you’ve written a note for a babysitter, followed a flat-pack furniture leaflet, or given directions over the phone, you’ve worked with one. This week you’ll learn to look at steps the way a computer does, so you can hear when one is missing.

An everyday picture

Think of a recipe. It starts somewhere (a list of ingredients), gives steps in order (mix, pour, bake), and ends somewhere (a cake). If step four says “bake” and step three was skipped, you get a hot bowl of flour. If a step says “add some sugar,” two cooks will add different amounts and get different cakes. A good recipe has every step, in the right order, and each step means only one thing.

That’s an algorithm. Every good one has the same three parts: a starting point, steps in order, and a goal you can check.

A precise picture

An **algorithm** is a finite list of exact steps that takes you from a starting point to a goal. Four things make it work:

- **Order.** Step two happens after step one, every time. Swap them and you get a different result.
- **Precision.** Each step means one thing. “Forward one square” is precise. “Go a bit that way” is not.
- **Completeness.** Nothing is left for the follower to guess. The step you think is obvious is the one most often missing.
- **Same start, same steps, same result.** Run it again from the same place and it does the same thing. That’s what makes prediction possible, and it’s why a program that went wrong once will go wrong the same way again until someone changes a step.

A **program** is an algorithm written in a language a machine can follow. When your child lays arrow cards on the floor, they are writing a program in a tiny language with three words: forward, turn left, turn right. Turning steps into symbols like that is called encoding. Chapter 5 uses blocks instead of cards, but it’s the same idea.

Decomposition is a long word for something simple: breaking a big job into small jobs. “Tidy the bookshelf” is too big to do in one step. “Take the books off, wipe the shelf, sort the books by size, put them back” is four small jobs anyone can follow.

Finally, there’s almost always more than one algorithm for the same goal. Two paths can both reach the cheese. One might be shorter. One might be easier to read. At 8–10, comparing two algorithms that both work is a real step up in thinking.

What the standards expect

The 2026 CSTA standards set a modest ladder here. For pre-K and kindergarten, children “carry out algorithms in daily activities” and “create a sequence of commands to solve a problem or express an idea.” They also “identify everyday gestures and symbols that represent information,” which is exactly what an arrow card is.²

By second grade, children “create an algorithm that includes sequence, events, and iteration,” which means steps in order, steps that start when something happens, and steps that repeat.²⁵

By fourth grade, they “create a written representation of an algorithm” that also includes selection (if-then choices), and they “compare different programming solutions to the same problem based on correctness and clarity.”²⁶ That last one is the Two Ways to the Door activity in this chapter, in grown-up words.

You may also find it reassuring that the CDC’s milestone list says most five-year-olds can tell a story with at least two events and can follow the rules of a game.⁹ Telling events in order is sequencing. Following rules is what a robot does. Your five-year-old already has the raw materials.

Wrong answers you should be able to hear

1. **Steps in favorite order, not time order.** Pancakes first, because pancakes are best. This is very common at five, and it's not carelessness. The child is sorting by a different rule. Reply: "What has to happen first?" Then act it out exactly as ordered, and let the silliness do the teaching.
2. **Giant steps.** "Go to the kitchen." That one step is really a whole algorithm hiding inside. Reply: "Show me with your feet." Count the steps it takes, and write those instead.
3. **Left and right flipped.** The child faces the toy, and the toy faces the child, so their right is its left. This is normal at 5–7, and for many older children too. Reply: "Stand behind the toy and face the same way it does." A sticker on the right hand helps.
4. **The "obvious" step missing.** "Put the berries in the yogurt" with no "take the lid off." Reply: "Let's run it exactly as written." The robot holding a closed yogurt cup makes the point better than any explanation.
5. **"It knows what I mean."** Vague words like "a bit," "over there," or "the usual way." Usually means the child is talking to a person, not a machine. Reply: "The robot can't guess. How many squares? Which way?"

Your five-minute warm-up

Alone, before tomorrow's Hour, lay out tomorrow's arrow-card path on the tape grid and walk the toy through it card by card. Plant one wrong card and walk it again, so you know what the mistake looks like. Then practice the opening question aloud once: "What is the very first step — and what comes right after it?" Count to five in your head after you say it.

How to teach it this week

Every Hour runs on the five steps from **The Computing Hour**. This week they look like this:

1. **Unplug First** (3–8 minutes). A quick body warm-up: Robot Grown-Up from Chapter 1, or ordering three morning-routine cards.
2. **Predict** (2–5 minutes). Before the toy moves, your child says where it will end up, or fills in the Expected/Actual card (BUG-02).
3. **Build and Run** (8–15 minutes). Your child lays the cards and moves the toy, one card at a time.
4. **Bug Hunt** (3–8 minutes). Where did the toy first leave the path? Change that card. Run from the start. A bug found earns a Bug Jar token (BUG-08).
5. **Save and Tell** (2–5 minutes). Photograph the card strip. Your child tells someone the first step and the last step.

Exact words you can say

- Opening: "What is the very first step — and what comes right after it?"
- Setting the goal: "The mouse starts here. The cheese is there. Your cards tell the mouse every move."

- Before the run: “Where do you think the mouse will end up?”
- When a step is too big: “Show me that with your feet. How many squares?”
- When order is muddled: “What has to happen first?”
- At the bug: “Run it card by card. Tell me when the mouse first leaves the path.”
- For a check: “Could someone who’s never seen this follow your cards?”
- At the end: “Is there a shorter way? You don’t have to change it. Just wonder.”

In-home — STEP-03 Arrow Card Path

This week’s first try-it for every child.

- **Time:** about 15 minutes.
- **Materials:** masking tape for a grid of about four by four squares, each roughly the size of a sheet of paper; a dozen arrow cards (forward, turn left, turn right) drawn on index cards; a small toy; a paper “cheese.”
- **Safety and privacy:** tape on a clear floor away from doorways. Lift the tape gently afterward so it doesn’t take the floor finish with it.
- **The fun:** rescuing a hungry toy with your own code.
- **The skill:** an algorithm as a path, and steps as symbols.
- **What to save:** a dated photo of the card line, cropped to the cards.

How it goes. Set the toy in one square facing a clear direction, and put the cheese two or three squares away. Your child lays arrow cards in a row, left to right, then predicts where the toy will end up. Then they move the toy one card at a time. “Forward” means one square ahead. “Turn left” and “turn right” mean the toy turns on the spot without moving. That turning rule is the trickiest part, so say it plainly before the first run.

If the toy reaches the cheese, celebrate the prediction and move the cheese. If it doesn’t, ask where it first left the path. Your child changes one card. Back to the start square, and run the whole strip again.

Stuck pattern. Left and right confusion. Put a small sticker on your child’s right hand and have them stand behind the toy, facing the same way. If turning on the spot is confusing, let the child be the toy for one run, walking the grid themselves.

How to fade help

- **Day 1.** Build the first path together, card by card, with you asking “and then?” after each.
- **Day 2.** Your child builds the whole path before running it. You ask only for the prediction and, after a bug, where it first went wrong.
- **Day 3.** Your child sets the cheese, builds the path, runs it, and finds their own bug. You watch and ask one question.
- **Any day.** Your hands stay off the cards. To show something, lay a separate strip of your own, then put it away.

When to stop talking

After the prediction, stop. During the run, let the toy and the cards do the talking. If your child is staring at the strip, they're tracing it in their head. Let them. When it works, be specific: "You put the turn right before the last forward, and that's what got it there."

Age-band moves: 5–7 and 8–10

5–7. Picture cards, arrow cards, and a grown-up robot. Paths of three to six cards. Predictions spoken or drawn. Introduce one new card at a time: forward only on Monday, then add one turn. Use the Morning Steps Strip (STEP-01) for sequencing without directions. Keep turning simple by having your child walk the path first. Ten good minutes is a full Build and Run.

8–10. Written steps. Your child writes a short algorithm on paper and someone else follows it exactly: Draw It From Words (STEP-05), Two Ways to the Door (STEP-07), Block Build Instructions (STEP-08). Paths get longer and include obstacles. Late in the week, start a paper game plan (DECOMP-04), which Chapter 5 turns into a real Scratch game. If written steps are a struggle, return to arrow cards for a few days. That's placement by skill, not a step back.

This week at a glance

Day	Focus	Try-it
Mon	Order matters	STEP-01 Morning Steps Strip; Robot Grown-Up warm-up
Tue	An algorithm as a path	STEP-03 Arrow Card Path
Wed	Steps as symbols	STEP-04 Cup Tower Code
Thu	Big jobs, small jobs	DECOMP-01 Big Job, Small Jobs; 8–10 add STEP-07 Two Ways to the Door
Outing	Steps in the world	OUT-04 Playground Program
Fri	Short check	AS-02 Show Me Two Ways, AS-03 Debug on the Spot, and the "done enough" list

Practice that actually builds learning

Each try-it lists time, materials, safety and privacy, the fun, the skill, and what to save. Pick one or two per Hour. The Expected/Actual card (BUG-02) and the Bug Jar (BUG-08) from Chapter 1 come along to every one.

In-home — STEP-01 Morning Steps Strip

- **Time:** about 10 minutes.

- **Materials:** five or six cards with simple drawings of your real morning routine.
- **Safety and privacy:** draw, rather than photograph, so no faces are needed.
- **The fun:** watching a grown-up act out a scrambled morning.
- **The skill:** sequence, and noticing a missing step.
- **What to save:** a photo of the strip in order, and the “first..., then...” sentence your child says.

How it goes. Your child puts the cards in order. Read the strip aloud exactly as laid out, acting it if you can. Once it’s right, secretly remove one card (“get dressed”) and read it again. Who notices the missing step, and what happens without it?

In-home — STEP-04 Cup Tower Code

- **Time:** about 15 minutes.
- **Materials:** six to ten paper cups; symbol cards for “pick up a cup,” “put it down,” “step right,” and “step left.”
- **Safety and privacy:** paper cups only; nothing that breaks.
- **The fun:** building a tower you never touched, using only a code.
- **The skill:** encoding steps as symbols someone else can read.
- **What to save:** the code strip beside a photo of the tower it built.

How it goes. Show a small target tower drawn on paper, such as three cups in a pyramid. Your child writes the code with symbol cards. Then you (or a sibling) build the tower using only the code, with no peeking at the drawing. Does the built tower match? If not, find the first symbol where the builder’s tower went different. At 5–7, keep it to three cups. At 8–10, let your child design the target and swap roles.

In-home — STEP-06 No-Knife Snack Algorithm

- **Time:** about 15 minutes.
- **Materials:** a yogurt cup, a spoon, and a few soft berries your child already eats safely; a bowl.
- **Safety and privacy:** check allergies first. No knives, no stove, nothing hot. Wash hands before you start.
- **The fun:** an algorithm you get to eat.
- **The skill:** order matters, and so does the “obvious” step.
- **What to save:** the drawn or written steps, and the answer to “What happens if step 3 comes first?”

How it goes. Your child says or draws the steps to make the snack. You follow them exactly. “Put the berries in” with the lid still on gets a puzzled robot and a closed cup. Fix the step, start again, and eat the result. Then ask the “what if” question: what if “stir” came before “add the berries”?

In-home — STEP-05 Draw It From Words (8–10)

- **Time:** about 10 minutes.

- **Materials:** two sheets of paper, two pencils, and a simple line drawing (a house, a boat) that only the describer sees.
- **Safety and privacy:** none needed.
- **The fun:** the hilarious gap between what was meant and what was drawn.
- **The skill:** precise instructions, and how vague words cause bugs.
- **What to save:** both drawings side by side, and the one word that caused trouble.

How it goes. Sit back to back. The describer gives instructions without showing the picture; the drawer can't peek and can't ask questions. Compare. Which instruction first sent the drawing off course? Swap roles and try again with that lesson in mind.

In-home — STEP-07 Two Ways to the Door (8–10)

- **Time:** about 10 minutes.
- **Materials:** a simple paper map of a room with a start mark and the door marked; pencils.
- **Safety and privacy:** if you walk the routes, clear the floor first.
- **The fun:** a contest between two plans that both win.
- **The skill:** comparing two algorithms for the same goal.
- **What to save:** both written routes, and one sentence on which is clearer and why.

How it goes. Your child writes two different algorithms to reach the door. Walk both. Did both work? Which has fewer steps? Which would a younger sibling find easier to follow? There can be two good answers. The point is the comparing, which is the grade 4 standard about correctness and clarity in action.²⁶

In-home — STEP-08 Block Build Instructions

- **Time:** about 15 minutes.
- **Materials:** building bricks, two matching sets if you have them; paper.
- **Safety and privacy:** small bricks are a choking risk for babies and toddlers nearby, so build at a table.
- **The fun:** making your own instruction booklet, like the ones in the box.
- **The skill:** writing steps that work for someone else.
- **What to save:** the instructions and a photo of both models.

How it goes. Your child builds a small model of five or six bricks, then draws or writes the instructions. A sibling or parent builds from the instructions alone. Where did the follower first get stuck? That's the step to rewrite.

In-home — DECOMP-01 Big Job, Small Jobs

- **Time:** about 10 minutes.
- **Materials:** a real chore, such as tidying a bookshelf or a toy bin; paper.
- **Safety and privacy:** nothing heavy to lift; you handle anything high.
- **The fun:** a big chore that suddenly looks small.
- **The skill:** decomposition, breaking a big job into small jobs.

- **What to save:** the list, and the number of small jobs.

How it goes. Name the big job. Ask, “What are the small jobs inside it?” Write or draw them. Put them in order. Do them, checking each off. Many children are surprised to find a job they’ve been avoiding is just four small ones.

In-home — DECOMP-02 Draw a Robot in Parts (5–7)

- **Time:** about 10 minutes.
- **Materials:** paper and crayons.
- **Safety and privacy:** none needed.
- **The fun:** a robot that appears one part at a time.
- **The skill:** decomposition for the youngest.
- **What to save:** the drawing, with your child pointing to each part as they name it.

How it goes. Draw a robot together, one part per step: head, body, arms, legs, buttons. Say each step before you draw it. Then let your child give you the steps for a robot of their own design.

In-home — DECOMP-03 Story in Three Boxes

- **Time:** about 15 minutes.
- **Materials:** a sheet of paper folded into three boxes.
- **Safety and privacy:** use made-up characters or first names only.
- **The fun:** a tiny comic.
- **The skill:** splitting a story into beginning, middle, and end.
- **What to save:** the three-box strip.

How it goes. Your child draws a beginning, a middle, and an end. Ask what happens first, and what comes right after it. Keep the strip: in Chapter 5, it becomes a three-page ScratchJr story (JR-04).

In-home — DECOMP-04 Game on Paper (8–10)

- **Time:** about 20 minutes.
- **Materials:** paper and pencil.
- **Safety and privacy:** none needed.
- **The fun:** inventing your own game before anyone touches a screen.
- **The skill:** breaking a game into characters, a goal, and rules.
- **What to save:** the plan, with at least one rule written as “when... then...”

How it goes. Your child lists the characters, the goal, and the rules: “When the cat touches the cheese, then the score goes up.” Keep it simple. In Chapter 5, this plan becomes a real Scratch maze game (SCR-05).

In-home — BOT-01 Kidbot

- **Time:** about 15 minutes.

- **Materials:** the tape grid; arrow cards; a person to be the robot.
- **Safety and privacy:** clear floor; slow steps; the robot stops before anything hard.
- **The fun:** a sibling who has to obey your cards.
- **The skill:** running an algorithm exactly, and writing one for a real “machine.”
- **What to save:** a photo of the program cards.

How it goes. This is the arrow-card path with a person as the toy, an idea CS Unplugged calls Kidbots and marks for ages 5–10.³⁴ One child programs; the other walks the grid exactly as the cards say, one square per forward. Swap. A Kidbot is free, and it does nearly everything a floor robot does.

In-home — BOT-03 Floor-Robot Treasure Map (optional robot)

- **Time:** about 15 minutes.
- **Materials:** a button-programmed floor robot, if you have or can borrow one; a paper treasure map with squares sized to the robot’s step.³⁵
- **Safety and privacy:** check the age label; small parts are a choking risk at 5–7. An adult changes batteries.
- **The fun:** a real machine following your code.
- **The skill:** planning on paper, entering the program, then running and debugging.
- **What to save:** a photo of the paper plan, and whether the robot reached the treasure.

How it goes. Plan the route on paper first with arrow cards. Your child enters it into the robot, predicts, and presses go. When it misses, compare the paper plan with what the robot did. Before the Hour, learn how your robot starts a fresh program, so an old one doesn’t sneak into the new run.

In-home — BOT-04 Robot Obstacle Course (robot or Kidbot)

- **Time:** about 20 minutes.
- **Materials:** a floor robot or a Kidbot; a few books standing as walls.
- **Safety and privacy:** books on a clear floor, nothing that can fall.
- **The fun:** a maze that fights back.
- **The skill:** longer algorithms and repeated debugging.
- **What to save:** the number of debugging tries before it worked.

How it goes. Build a short maze. Your child writes and runs the route, fixing one step at a time. Count the bugs found. A higher count is a better story, not a worse score.

A word on buying a robot: one is plenty if you like them, and none is fine. Structured, hands-on activities suit the youngest children well, and a person on a tape grid is as hands-on as it gets.²² Chapter 5 covers robots properly. Check current prices yourself.

Out-of-home — OUT-04 Playground Program

- **Time:** about 15 minutes at a playground.
- **Materials:** the playground; paper and a pencil, or just your voices.

- **Safety and privacy:** usual playground rules; adult nearby; take turns; your child follows their own steps carefully, holding rails on ladders. Write first names only if you write anything.
- **The fun:** a program you run with your whole body.
- **The skill:** writing a real-world sequence, and spotting what repeats.
- **What to save:** the drawn or written steps for the slide, and one repeat your child noticed.

How it goes. Before your child goes down the slide, ask: “What is the very first step — and what comes right after it?” Together, say or draw the algorithm: walk to the ladder, hold both rails, climb one step, climb one step, climb one step, sit at the top, let go, slide. Then your child runs it, exactly as written. Did you forget anything? (Waiting your turn is a common missing step.) Notice that “climb one step” repeated. Then watch the swings: push, swing, push, swing. Chapter 3 calls that a loop.

Talk box (this chapter)

Opening: “What is the very first step — and what comes right after it?”

Follow-ups: 1. Prediction: “Where do you think the toy will end up?” 2. Evidence: “What did you see the toy do that makes you say that?” 3. Location: “Point to the card where it first went wrong.” 4. Change: “What did we forget? What’s one card we could change?” 5. Revision: “Could someone who’s never done it follow this? You can change it. Let’s run it again.”

For 8–10, add: “Is there a shorter way?”

How to wait. Count to five in your head after you ask. When your child is studying the card strip, that silence is tracing, so give it longer. Look at the cards, not at your child.

What a stuck silence usually means. The program was too long to hold in mind, so cover all but the first three cards. The child doesn’t know where to look, so run it again, one card at a time, stopping after each. They may be afraid of choosing wrong, so offer a choice of two: “Is it this card or this card?” Or they may be waiting for you to fix it. Put your hands in your lap and say, “I’m waiting for the programmer.”

Blocked and mixed practice

Blocked practice for the new move. For the first few days, every Hour is about order: routine cards, arrow paths, cup codes. Same idea, different materials.

Mixed practice. Later in the week, mix in decomposition and comparison. Give your child a goal and let them choose the tool: arrow cards, a written list, or a cup code. Bring back Chapter 1’s move, too: plant a bug in a routine strip and let them find it. Ask for a quick retrieval at the start of an Hour: “Show me last week’s turn card.”

One incorrect example to diagnose

Here’s an illustration, not a real family. A seven-year-old’s arrow strip reads: forward, forward, forward, turn right, forward. The toy is supposed to reach the cheese, which is two squares ahead

and one square to the right. The toy ends up one square beyond the cheese.

Diagnosis. There's one forward too many before the turn. The toy went three squares when it needed two. The child, asked where it went wrong, points to the last card, because that's where the toy ended up.

Rescue. Run it again, card by card. After each card, ask, "Is the toy still where it should be?" After the third forward, it isn't. That's the first wrong step. The fix is to take away one forward card. The last card was fine all along. Finding the *first* wrong step, not the last, is the habit Chapter 4 builds.

For the student

An **algorithm** is a set of exact steps to reach a goal. A recipe is an algorithm a robot could follow.

But robots can't guess. If you skip a step, they skip it too. If you say "go over there," they don't know where "there" is.

A tiny example. Your toy mouse needs to reach the cheese. The cheese is two squares ahead and then one square to the right. Here's your program:

Forward. Forward. Turn right. Forward.

Move the mouse one card at a time. Forward, forward: two squares. Turn right: the mouse turns but doesn't move. Forward: one square. Cheese!

What if you'd written forward, turn right, forward, forward? Try it in your head. The mouse would end up somewhere else. Same cards, different order, different result. Order matters.

Try this today. Ask a grown-up to make a grid of squares with tape on the floor. Put a toy in one square and a "cheese" in another. Lay arrow cards to get the toy there. Before you move it, say where you think it will end up. Then run your program, one card at a time.

If the toy goes the wrong way, find the *first* card that sent it wrong. Change just that card.

Explain it back. Tell someone: what is an algorithm? Why did the order of your cards matter?

Challenge. Write the steps for making your bed, or getting a snack, on paper. Give it to a grown-up and have them follow it *exactly*. No helping! Where did they get stuck? Fix that step.

Programmers check their steps, and then they check them again.

If it isn't clicking

Left and right keep getting mixed up. That's very common, and it's about which way you're facing, not about being clever. Next move: a sticker on your child's right hand, and have them stand behind the toy, facing the same way it faces. Or let them be the toy and walk the path. If it's still hard, use just "forward" and one kind of turn for a week.

Steps are too big. “Go to the kitchen.” “Make the snack.” The child sees the whole job as one step. Next move: “Show me with your feet.” Count the actual steps. Then try Big Job, Small Jobs (DECOMP-01) with a chore. Seeing one job turn into four small ones makes the idea stick.

Your child is bored. Usually the robot is too sensible. Next move: make the robot sillier, and more literal. Pour imaginary berries onto the closed lid. Walk into the wall in slow motion. Or hand the robot role to your child and let them be literal with you. A bored child often comes alive when they’re the one who gets to say “Robot does not understand.”

When to slow down. Your child can’t yet order three routine cards by time. Stay with STEP-01 and Robot Grown-Up for another week.

When to go ahead. Your child builds a four- or five-card path, predicts it, and fixes their own bug. Move on to Chapter 3.

When to step back to unplugged. If a floor robot or a tablet turns into button-mashing, put it away and return to the tape grid and a Kidbot for a few days. A wrong step is a clue; tell your child so, and keep their program in their hands.

Tools, including AI

Paper, index cards, masking tape, paper cups, a small toy, and a few berries are all you need. A Kidbot, which is a person on a tape grid, is free.

An optional button-programmed floor robot is fun if you have one or can borrow one from a library or co-op. One is plenty, and none is fine. Check the age label for small parts at 5–7, and check the current price yourself. Chapter 5 covers robots in more detail. CS Unplugged offers free Kidbot activities.³⁴

No AI is needed for this chapter. If you want help planning, use a tool on your own account to invent extra grid puzzles or routine strips, with the answers kept for you. The full rules are in the AI box in **The Computing Hour**.

What “done enough” looks like

Before moving on, check:

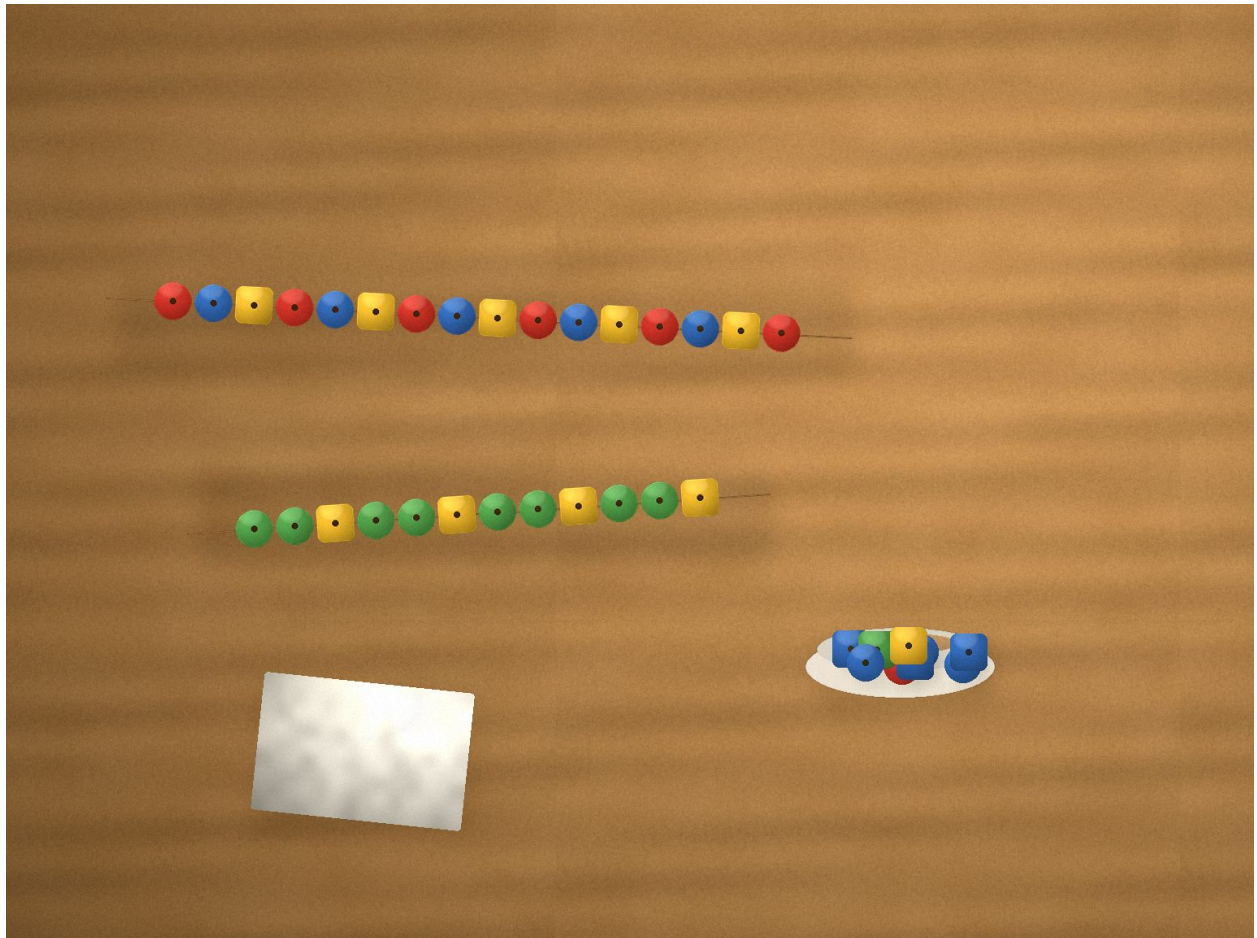
- ☐ Your child put a five-step routine in time order and noticed a missing step.
- ☐ Your child built and ran a card path to a goal, predicting first.
- ☐ Your child found and fixed at least one missing or wrong step, starting from the first wrong one (Debug on the Spot, AS-03).
- ☐ Your child showed two different paths to the same goal (Show Me Two Ways, AS-02).
- ☐ At 8–10: your child wrote instructions that someone else followed successfully, or found where the follower got stuck.
- ☐ Your child taught today’s idea to a sibling or a teddy in a sentence or two (Teach It Back, AS-04).

- ☐ One card strip, cup code, or routine went into the folder, chosen by your child with a reason (Portfolio Pick, AS-05).
- ☐ One outing included a real-world sequence, such as the playground slide.

Place by skill, not birthday. A seven-year-old who still flips left and right is right on track. A ten-year-old who writes clear steps but rushes the prediction still needs the guess habit. Both can move on when most boxes are checked, and both will keep practicing steps in every chapter that follows.

A program's level name describes that program, not your child's grade. If you keep records, this week's work lives under **Computer Science**.

Chapter 3



Patterns, Loops, and If-Then

Why this matters

Clap, stomp, stomp. Clap, stomp, stomp. Clap...

You stop. Your six-year-old finishes it without being asked: “Stomp, stomp.” Nobody taught a lesson. They heard something repeat, guessed what came next, and were right. That small, happy

guess is where this chapter begins.

A **pattern** is something that repeats in a way you can predict. Once a child can see a pattern, they can do three things a programmer does every day. They can predict what comes next. They can say the repeating part once and add “do it again,” which is what a **loop** is. And they can notice when something *changes* the pattern, which is the door to **if-then**: “If it’s raining, then boots. If not, sandals.”

Add one more idea and you have the whole chapter. An **event** is “when this happens, do that.” The doorbell rings, so you answer it. The green flag is tapped, so the program starts. Events are how programs wait for the world.

Why spend a few weeks here? In Chapter 2, your child spelled out every step. That works for short paths and gets tiring fast. A robot that walks forward eight times needs eight cards; a loop says it in one: “repeat 8 times: forward.” And an if-then lets one program handle more than one situation. A sock machine that knows “if it’s striped, put it here; if not, put it there” is starting to think in rules.

Patterns also feed the first half of the book’s focus line: predict before you run. When your child says “the cat will jump three times because the repeat says three,” they’re making the kind of guess that turns a program into an experiment.

Pattern games are not a secret math program, and nobody needs them to be. They practice noticing, describing, and testing a rule. Those are real computing habits, and that’s reason enough.

This week, specifically. You can learn to hear the difference between “a loop makes it faster” and “a loop makes it shorter,” and you can learn to ask “what happens when it’s *not* true?” Your child can try a clap-stomp pattern before breakfast and shrink a long arrow strip with one repeat card after lunch.

For the parent: understand it yourself

You don’t need to know any programming language for this chapter. You do need four plain ideas, one everyday picture, and one precise picture. Then you’ll be able to hear the common wrong turns.

The four ideas, in ordinary words.

- **Pattern and attribute.** A pattern repeats in a way you can predict: red, blue, red, blue. An **attribute** is a feature you can sort or notice by: color, size, stripes, number of legs. Children see patterns by noticing attributes. The kindergarten target in the national computer science teachers’ standards says it neatly: children should “recognize attributes of objects to notice patterns and make decisions.”²
- **Loop.** A loop means “do it again.” There are three kinds your child will meet. *Repeat a number of times* (“repeat 3 times: hop”). *Repeat until something happens* (“stir until there are no lumps”). And *repeat forever* (“keep the fish swimming”). A forever loop needs a way to stop, which is usually a stop button or a grown-up saying “freeze.”
- **If-then-else.** Check something; then choose. “If it’s striped, put it in this basket. Else, put it in that one.” The “else” part is what happens when the check is *not* true. Children often forget it. So do adults.

- **Event.** “When this happens, do that.” An event is a doorbell. Nothing happens until it rings.

The ladder these ideas climb. The same standards place sequence, events, and repetition together at about second grade (“create an algorithm that includes sequence, events, and iteration,” where *iteration* is the formal word for looping), and add if-then choices, called *selection*, at about fourth grade.^{25,26} That ordering is useful to you. Loops and events come first and come easily. If-then takes longer. In one kindergarten robotics study, children picked up the basic programming ideas, but the more complex ones, such as conditionals, needed more teaching time.³⁶ So if your five-year-old is fluent with repeat cards and wobbly with “if,” that’s normal. Keep if-then concrete and physical at 5–7, and keep it there as long as it needs.

One everyday picture. Your morning routine is full of these. “Brush until the timer beeps” is repeat-until. “If it’s cold, then a coat” is if-then-else, and “else, no coat” is the part that gets skipped in November. “When the bus comes, go” is an event. You’ve been running small programs at the front door for years.

One precise picture. Lay out a strip of arrow cards from Chapter 2, the kind that walk a toy across a masking-tape grid:

forward, forward, forward, forward, turn right, forward, forward

Now replace the first four cards with one card that says **repeat 4: forward**. The strip is shorter. The toy does exactly the same thing. That’s the whole point of a loop: it changes how the program is *written*, not what it *does*. The toy still takes every step.

Now add a rule card: **if the square is blue, then hop; else, step**. Color two squares of the grid blue. The toy’s path is the same, but what it does on each square depends on the square. That’s if-then-else.

Wrong answers you should be able to hear. These are the common ones, with what each usually means and what to say.

1. **“The loop makes it go faster.”** This usually means the child sees fewer cards and assumes fewer steps. Try: “Let’s count the steps the toy takes, not the cards. Did it skip any?” They’ll find it didn’t. A loop makes the program shorter. The computer still does every step.
2. **The child copies a pattern perfectly but can’t extend it.** This usually means they haven’t found the repeating unit, the small chunk that repeats. Try: “Show me the part that repeats. Put your hands around it.” Go shorter if needed: an AB pattern (clap, stomp) before ABB (clap, stomp, stomp).
3. **“If” is treated as “always.”** The sorting machine puts *every* sock in the striped basket, or the child claps after every move in Simon Says If. This usually means they’ve heard the action and not the check. Try: “What happens when it’s *not* true?” Then play a round where the answer is “nothing.”
4. **“If” and “when” get swapped.** The child says “if the doorbell rings” and “when it’s striped” as if they mean the same thing. This is very common and only matters a little at 5–7. A good picture for later: *if* is a question you ask; *when* is a doorbell you wait for.
5. **The wrong part goes inside the loop.** The child wraps the whole program in “repeat 3,” so the toy walks the entire path three times and ends up in the hallway. This usually means they

haven't yet asked "which part repeats?" Try: "Put your finger on the first card that repeats and the last one. Only those go inside."

None of these are silly. Each is a reasonable idea, slightly off. Hear it as information, then ask one question.

Your 5-minute parent warm-up. Before tomorrow's Hour, do this alone. Lay out a seven-card arrow strip with four forwards in a row. Run it with a toy and count the steps. Now replace the four forwards with a repeat card and run it again. Count again. Then say tomorrow's opening question out loud, once, so it feels natural in your mouth: "What keeps repeating — and what would make it change?" Practice waiting after you say it. Count to five in your head. It feels long. It isn't.

How to teach it this week

The shape of a pattern-and-loop Hour. Use the Computing Hour's five steps exactly as you met them up front. Here's how they look this week, with the minutes for each step.

- **Unplug First (3–8 min).** A body pattern. PAT-01 Clap-Stomp Pattern for 5–7; a quick PAT-03 What's My Rule? for 8–10. No screen.
- **Predict (2–5 min).** Before anything runs, ask for a guess: "If we use a repeat card, how many cards will we save?" or "What will the toy do on the blue square?" Draw or write the guess on the Expected/Actual card (BUG-02).
- **Build and Run (8–15 min).** The child builds the card program with their own hands and runs it with a toy or with you as the robot. Your hands stay in your lap.
- **Bug Hunt (3–8 min).** Compare the guess to what happened. If the toy walked into the couch, find the first wrong step. Change one thing. Run it again. A bug found earns a Bug Jar token (BUG-08).
- **Save and Tell (2–5 min).** Photograph the "before" strip and the "after" strip side by side, cropped to the cards. Then the child tells one sentence to someone else: "I made it shorter with a repeat."

Exact wording you can use. These are sentences you can say almost word for word.

- Opening: "What keeps repeating — and what would make it change?"
- For a pattern: "Show me the part that repeats. Put your hands around it."
- For a loop: "Is there a way to say all those forwards with one card?"
- For counting: "Count the steps the toy takes. Now count the cards. Same or different?"
- For if-then: "What's the rule? And what happens when it's *not* true?"
- For an event: "What starts it? What are we waiting for?"
- When they're stuck: "Is it this card or this card?" (a choice of two).
- At the end: "Save it, name it, screen off — then tell me one thing."

The first try-it for the student. Start the week with LOOP-02 Shorten the Code, below. It builds directly on the arrow strips from Chapter 2, so the child begins on familiar ground and meets one new move: the repeat card. If you haven't done arrow strips yet, start with PAT-01 for two days and come back.

How to fade help across the week.

- **Day 1.** You lay out the long strip; the child swaps in a repeat card; you ask the counting question.
- **Day 2.** The child builds a long strip of their own, then shortens it. You only ask, “What keeps repeating?”
- **Day 3.** The child uses a repeat card from the start and predicts the step count. You watch the toy.
- **Days 4–5.** If-then joins in (IF-03), then a mixed day.

When to stop talking. After the opening question, while the cards go down, and while the toy moves. You’ll want to say “careful, that’s five, not four.” Let the toy say it instead. A toy that ends one square short is a better teacher than your warning, and the discovery stays with the child.

Age-band moves. Place by skill, not birthday. A nine-year-old who is new to all this can spend a week on the 5–7 moves without losing anything.

- **5–7.** Keep it physical and short. Clap patterns, bead strings, dance moves drawn as pictures, a repeat card with a big number on it. If-then is played with bodies first (IF-01 Simon Says If, IF-02 Red Light, Green Light Rules) and with socks or coins second. Predictions are spoken or pointed. You may write the child’s words on the Expected/Actual card. One loop per program is plenty. Repeat-until can be played (LOOP-03) without being written down.
- **8–10.** Write it. The child writes a short algorithm on paper with a loop and an if-then in it, then someone else runs it. Try loops with stop conditions (“repeat until you reach the door”). Try a two-part rule (“if it’s striped *and* blue”). Try IF-05 Choose-Your-Path Story. Move to the screen with LOOP-05 and, for confident readers, Scratch’s “repeat until” and “if-then-else” blocks. Variables, like a score that changes, can wait until Chapter 5 and late in the year (SCR-04); there’s no rush.

A week at a glance (a suggestion, not a script).

Day	Unplug First	Build and Run	Card codes
Mon	Clap-stomp	Shorten the Code	PAT-01, LOOP-02
Tue	Extend the beads	Dance Loop or Necklace Loop	PAT-02, LOOP-01 or LOOP-04
Wed	Simon Says If	Sorting Machine	IF-01, IF-03
Thu	Repeat Until the Music Stops	Weather Wardrobe; 8–10 Choose-Your-Path Story	LOOP-03, IF-04, IF-05
Fri	What’s My Rule?	Loop It on Screen (if ready) + short check	PAT-03, LOOP-05, AS-03
Out	—	Traffic Light Algorithm Walk	OUT-01

Three to five Hours a week is a full week. A ten-minute Hour on a rough day counts.

Siblings and one table. Run Unplug First with everyone together; clap patterns scale beautifully. For Build and Run, use driver and navigator: one child lays the cards, the other holds the Expected/Actual card and reads the plan aloud, and you swap on a timer. Neither child is the other's teacher. Both are programmers.

Practice that actually builds learning

Practice this week has two jobs. First, *blocked* practice: several short goes at one new move until it feels easy, such as a whole day of loops. Second, *mixed* practice: a set of puzzles where the child has to decide which move fits. The second job is the one that sticks, so don't skip it once the new move feels steady.

In-home — LOOP-02 Shorten the Code

- **Time:** about 10 minutes inside Build and Run.
- **Band:** both (5–7 with picture arrows; 8–10 with written steps).
- **Materials:** a masking-tape grid on the floor or table, arrow cards from Chapter 2, one or two blank cards for “repeat,” a small toy, a marker.
- **Safety and privacy:** clear the floor if a person is the robot; slow steps only.
- **The fun:** a long, clumsy program shrinks to a neat one, and the toy still gets to the cheese.
- **The skill:** a loop shortens a program without changing what it does.
- **What to save:** a photo of the before and after strips side by side, plus the card count (“9 cards → 5 cards”).

How it goes. Lay out a long strip with a run of identical cards: five forwards, a turn, three forwards. Run it once so everyone agrees it works. Ask the opening question. Hand over a blank card and a marker and let the child make a repeat card: “repeat 5” with a forward arrow drawn underneath, or the forward card tucked under the repeat card like a sandwich. Ask for a prediction: “Will the toy end up in the same place?” Run it. Count steps and cards. If the toy ended up somewhere else, you've got a bug, and that's a good Hour, not a bad one.

At 8–10, ask for two loops in one strip, or a loop inside a loop if they're keen (“repeat 2: forward, forward, turn right” walks a little square). At 5–7, one loop is a triumph.

In-home — IF-03 Sorting Machine

- **Time:** about 10 minutes.
- **Band:** both.
- **Materials:** a basket of clean, mixed socks (or coins, buttons, or toy cars); two bowls or baskets; an index card for the rule.
- **Safety and privacy:** small buttons and coins are a choking risk for younger siblings, so use socks if a toddler is near.
- **The fun:** the child is the machine and gets to be very strict about it.
- **The skill:** if-then-else; following and writing a rule.
- **What to save:** the rule card itself, in the child's words or yours.

How it goes. Write a rule on the card: “If it’s striped, put it in the left basket. Else, put it in the right basket.” The child becomes the machine. Feed in socks one at a time. The machine may only follow the card, not its own opinion about which socks are nicest. Then swap: the child writes a rule, and you are the machine. Be a kindly literal machine. If the rule says “if it’s striped” and doesn’t say what to do with the others, hold the plain sock in the air and wait. That pause teaches “else” better than any explanation.

At 8–10, try two-part rules: “If it’s striped *and* has blue in it.” Try a rule with three outcomes (“if striped, left; else if spotted, middle; else, right”). Then ask the question that turns this into real computing: “Is there any sock your rule doesn’t know what to do with?”

In-home — PAT-03 What’s My Rule? (and the smaller pattern cards)

- **Time:** about 10 minutes.
- **Band:** both.
- **Materials:** buttons, socks, toys, or fruit pictures.
- **Safety and privacy:** watch small objects near toddlers.
- **The fun:** a guessing game where the grown-up has a secret.
- **The skill:** noticing attributes and saying a rule in words.
- **What to save:** the rule, written in the child’s words.

How it goes. Sort objects into two piles by a secret rule (wheels / no wheels). The child places the next object, then guesses the rule. Swap roles.

Quick pattern warm-ups for any day: **PAT-01 Clap-Stomp Pattern** (5–7: AB, ABB, ABC, then one of their own); **PAT-02 Extend the Beads** (both: continue your bead pattern, then hide a bead for you to find); **PAT-04 Pattern Hunt Indoors** (5–7: three repeating patterns in the house, one drawn); **PAT-05 Odd One Out** (both: which doesn’t belong, and why? Two good answers are allowed).

In-home — LOOP-03 Repeat Until the Music Stops (and the other loop cards)

- **Time:** about 5 minutes.
- **Band:** both.
- **Materials:** music you can pause.
- **Safety and privacy:** clear a space for moving.
- **The fun:** a freeze dance with a programmer’s name.
- **The skill:** a loop with a stop condition (“until”).
- **What to save:** one “until” sentence the child says: “I hop until the music stops.”

How it goes. Choose a move. Do it again and again *until* the music stops. Then ask, “What made the loop end?” Switch the stop condition: “until I clap,” “until you’ve done ten,” “until you touch the door.” At 8–10, compare: “Which of those knew how many times it would repeat before it started?”

Two more loop cards fit this week: **LOOP-01 Dance Loop** (5–7: four moves drawn as pictures plus a “repeat 3 times” card) and **LOOP-04 Necklace Loop** (5–7: one bead unit, repeated all the

way round a string).

In-home — IF-05 Choose-Your-Path Story

- **Time:** about 20 minutes, possibly over two Hours.
- **Band:** 8–10.
- **Materials:** paper, pencil, sticky notes or index cards.
- **Safety and privacy:** characters can be invented; use first names or nicknames only for family members.
- **The fun:** a story where the reader decides, and every choice matters.
- **The skill:** branching if-then-else, planned on paper.
- **What to save:** a drawn branch map (boxes and arrows).

How it goes. Page one ends with a choice: “If you open the door, go to page 2. Else, go to page 3.” Each page offers two choices. Keep it to six or seven pages. Then someone else “runs” the story by following the choices. Ask: “Is there any path that ends nowhere?” A dead end is a bug, and finding it is the point.

Quick if-then warm-ups: **IF-01 Simon Says If** (5–7: “If I touch my nose, you clap”); **IF-02 Red Light, Green Light Rules** (5–7: “When you say green, I walk”); **IF-04 Weather Wardrobe** (both: “If rain, then boots; else, sandals”).

In-home — LOOP-05 Loop It on Screen

- **Time:** about 10 minutes of screen inside Build and Run (10–15 minutes at 5–7 is the upper end).
- **Band:** both, once the child has done LOOP-02 on cards.
- **Materials:** a tablet with ScratchJr, a free app for ages 5–7,⁴ or Scratch for 8–10 on the family’s shared device. Chapter 5 covers setting both up.
- **Safety and privacy:** the device lives in a shared room; no account is needed for ScratchJr; the parent handles any login for Scratch.
- **The fun:** the same loop they made with cards makes a cat dance.
- **The skill:** the same idea in a new window; a loop on cards and a loop on screen are the same thing.
- **What to save:** a photo of the script (crop to the blocks) and the project saved with a date-first name.

How it goes. Rebuild the card loop with blocks. In ScratchJr, the **Repeat** block wraps around other blocks and “runs the blocks inside a specified number of times”; **Repeat Forever** runs the script over and over.³⁷ In Scratch, the matching blocks are **repeat**, **forever**, and **repeat until**, and Scratch also has **if-then** and **if-then-else** blocks.³⁸ ScratchJr has no “if” block. Its closest cousins are its trigger blocks, such as **Start on Bump**, which starts a script when another character touches this one.³⁷ That’s an event, not an if-then, and it’s fine for 5–7.

Before the green flag, ask, “How many times will the cat jump?” A forever loop that doesn’t stop is a lovely moment to ask: “What would make it change?”

Out-of-home — OUT-01 Traffic Light Algorithm Walk

- **Time:** about 20 minutes, as part of a walk or errand.
- **Band:** both.
- **Materials:** a pencil and a folded index card; a crossing with a traffic light and, ideally, a walk signal.
- **Safety and privacy:** hold hands; stand well back from the curb; follow road rules; the grown-up decides when to cross. The light is something to watch, not a game to play in the road.
- **The fun:** spying on a machine that runs a program all day.
- **The skill:** spotting a loop (the colors repeat) and an event or condition (the walk light).
- **What to save:** the light's sequence drawn on the card, and one "when... then..." sentence.

How it goes. Stand somewhere safe where you can see the light for a few minutes. Ask the child to say the colors as they change. Then: "What comes next? How do you know?" They're predicting from a pattern. Watch a full cycle. Ask the opening question: "What keeps repeating — and what would make it change?"

If there's a walk button, watch what happens after someone presses it. Right away, or after a while? Let the child guess and check over a few cycles. At 8–10, they write the light's program as a loop with a "when" inside it. At 5–7, three colored circles with an arrow going round is a fine crumb.

Weather or no-crossing alternative. Make a traffic light from a shoebox and three paper circles. One child is the light and runs the loop. Another child is a walker who presses an imaginary button. The light-child has to decide what the button changes. You've built the same idea indoors.

Talk box (this chapter)

Opening: "What keeps repeating — and what would make it change?"

Follow-ups (pick three to five):

1. **Prediction:** "How many times will it repeat? What will it do next?"
2. **Evidence:** "What did you see that makes you say that?"
3. **Location:** "Point to the part that repeats. Where does the loop start and stop?"
4. **Change:** "What's the rule? What happens when it's *not* true?" or "What's one thing we could change?"
5. **Revise:** "You can change your guess. Shall we run it again?"

How to wait. After you ask, count to five in your head. That pause is a teaching habit that comes from classroom studies of "wait time," used here as plain good manners, not a magic number.⁵ After the child answers, wait again; the second answer is often better. If they're counting cards or watching the toy, they're working. Don't cut in.

What a stuck silence usually means. Often the question was too big. "What's the pattern?" is harder than "What comes after the stomp?" Sometimes the child can't see the repeating unit because the pattern is too long. Sometimes they're afraid a wrong guess means they're bad at this. And sometimes they skipped the prediction, so there's nothing to compare.

The move. Make it smaller and more physical. Offer a choice of two ("Does a clap or a stomp

come next?”). Ask them to point instead of speak. Put your hands around the first unit to show what “the part that repeats” looks like, on your own cards, then hand theirs back.

Blocked and mixed practice

Blocked (early in the week). Several short goes at one new move. Three different long strips to shorten with a repeat card. Three different sorting rules. Repetition is fine when the move is new; the child is building a feel for it.

Mixed (later in the week). Now the child has to decide. Set out five quick puzzles on index cards and let the child sort them into “needs a loop,” “needs an if-then,” and “needs both”:

- a toy that walks forward six squares;
- a rule for which bin the recycling goes in;
- a dance that does the same three moves four times;
- a robot that should hop on blue squares and step on the others, all the way across the grid;
- a story where the reader chooses a door.

There’s room for argument on some of these, and that’s good. Ask, “What makes you say that?” The answer matters more than the bin.

Quick retrieval. At the start of the next week’s Unplug First, ask: “Show me last week’s repeat card. What did it do?” A one-minute recall keeps the idea alive while new ideas arrive.

One incorrect example to diagnose

Show the child this program, on cards or written out, and say it was made by “a robot-builder who needs help.” (It’s better if the mistake isn’t theirs.)

Rule card: **If the sock is striped, put it in the left basket.** The machine puts a striped sock in the left basket. Then it puts a spotty sock in the left basket. Then a plain sock in the left basket.

Ask: “What did the builder expect? What did the machine do instead?” Then: “Where did it first go wrong?” The first wrong move is the spotty sock. The rule never said what to do when the sock is *not* striped, so the machine did the only thing it knew. This is the “if treated as always” bug. The fix is one change: add “else, put it in the right basket.” Run it again.

At 8–10, add a second bug for a challenge: a repeat card wrapped around the whole program instead of just the forwards, so the toy walks the entire path twice.

A worked Hour (illustration)

Picture a seven-year-old on a Tuesday. Unplug First: she and her dad clap a pattern, and she invents one with a pat on the head. Predict: he lays out six forwards and asks how many cards a repeat would save. She guesses “three.” Build and Run: she makes a “repeat 6” card and runs the toy. It stops one square short. Bug Hunt: she counts squares with her finger and finds the path needed seven; the long version had been wrong all along, and the short one made it easier to see. She

changes one number. Save and Tell: a photo of both strips, and a phone call to her grandmother about the bug she found in “the old one.” The screen never came on. That’s a full Hour.

Specific encouragement for the week

- **What you can learn this week:** to say “a loop makes it shorter, not faster,” and to ask “what happens when it’s *not* true?” without explaining the answer.
- **What your child can try today:** extend a clap-stomp pattern, then invent one for you to extend. It takes three minutes and it’s real computing.

For the student

Some things happen again and again. Clap, stomp, stomp. Clap, stomp, stomp. That’s a **pattern**: something that repeats so you can guess what comes next.

Programmers love patterns, because they get to be lazy in a good way. Instead of writing the same step five times, they write it once and say “do it again 5 times.” That’s a **loop**. A loop means “do it again.”

Here’s an example. A toy needs to go forward, forward, forward, forward, then turn. That’s five cards. With a loop, it’s two: **repeat 4: forward**, then **turn**. The toy still takes every step. The program is just shorter.

Sometimes a program has to decide. “**If** the sock is striped, put it here. **If not**, put it there.” That’s an **if-then**. The “if not” part is easy to forget. A good programmer always asks, “What happens when it’s *not* true?”

And some things only start when something else happens. **When** the doorbell rings, you answer it. That’s called an **event**.

Try this today. Make up a clap-and-stomp pattern. Do it for a grown-up and see if they can keep it going. Then make a long arrow path for a toy and shrink it with a repeat card.

Explain it back. Tell someone what a loop does, using your arrow cards. Does a loop make the toy go faster, or does it make the program shorter?

Challenge. Write a rule for a sock machine with three baskets instead of two. Then give a sock your rule doesn’t know what to do with. Can you fix the rule so it does?

If it isn’t clicking

Diagnostic 1: The child can copy a pattern but can’t continue it. They can lay red, blue, red, blue beside yours, but on their own they stall or go random. This usually means they’re matching item by item and haven’t found the unit that repeats.

Next move: go shorter. AB patterns only (clap, stomp; red, blue) for a day or two. Put your hands around one unit on your own beads and say, “This part repeats.” Then ask them to put their hands

around a unit on theirs. When AB is easy, try ABB.

Diagnostic 2: Loops seem to “break” the program. Every time the child adds a repeat card, the toy goes too far, not far enough, or wanders off. This usually means one of two things: the count is off (they counted cards, not steps), or the wrong cards went inside the loop.

Next move: step back to bodies. You be the robot: “repeat 3: hop,” and you hop exactly three times; “repeat 3: hop, turn,” and you hop-turn three times, looking a bit silly. For the count, they walk the path with a finger first and count squares aloud.

Diagnostic 3: If-then just won’t land. The machine sorts everything into one basket, or the child mixes “if” and “when” so much that the rules stop making sense. This usually means if-then is still too abstract, which is normal at 5–7.

Next move: make the “else” physical. Two baskets, two colored sheets of paper on the floor, and the child stands on one or the other for each sock. Play Simon Says If (IF-01) for a week and leave written if-then cards for later. For the if/when mix-up, a simple picture helps: *if* is a question you ask; *when* is a doorbell you wait for.

When to slow down, go ahead, or step back. If every Hour ends in tears or a shrug, spend a week on patterns and bodies alone. If the child shortens strips easily and writes a rule with an “else,” go ahead to LOOP-05 and Chapter 4. If a screen loop confuses things, go back to cards for two Hours; the idea is the same, and cards are easier to see.

The rescue order. Ask a question, wait, give a small hint, then, if needed, show a short demo on *your own* cards. Then hand the child’s cards back. Their program stays theirs. If you’ve already moved their cards, say so kindly: “I moved that one. Let’s put it back and you try.”

Tools, including AI

What you need. Beads or pasta and string; a basket of socks; masking tape; index cards and a marker; a small toy; music you can pause. That’s the whole kit for most of this chapter.

Optional screen. ScratchJr’s Repeat and Repeat Forever blocks for LOOP-05 at 5–7; Scratch’s repeat, repeat until, and if-then blocks at 8–10. Both are free. Chapter 5 explains setting them up, with the parent handling any account.

AI. The child doesn’t use AI for any of this. You may, on your own account, use an AI tool to invent extra pattern puzzles or sorting rules for yourself, with the answers kept away from the child until after they’ve tried. The full guidance is in the AI box in **The Computing Hour**.

What “done enough” looks like

Before you move on, look for these. You don’t need every box on the same day.

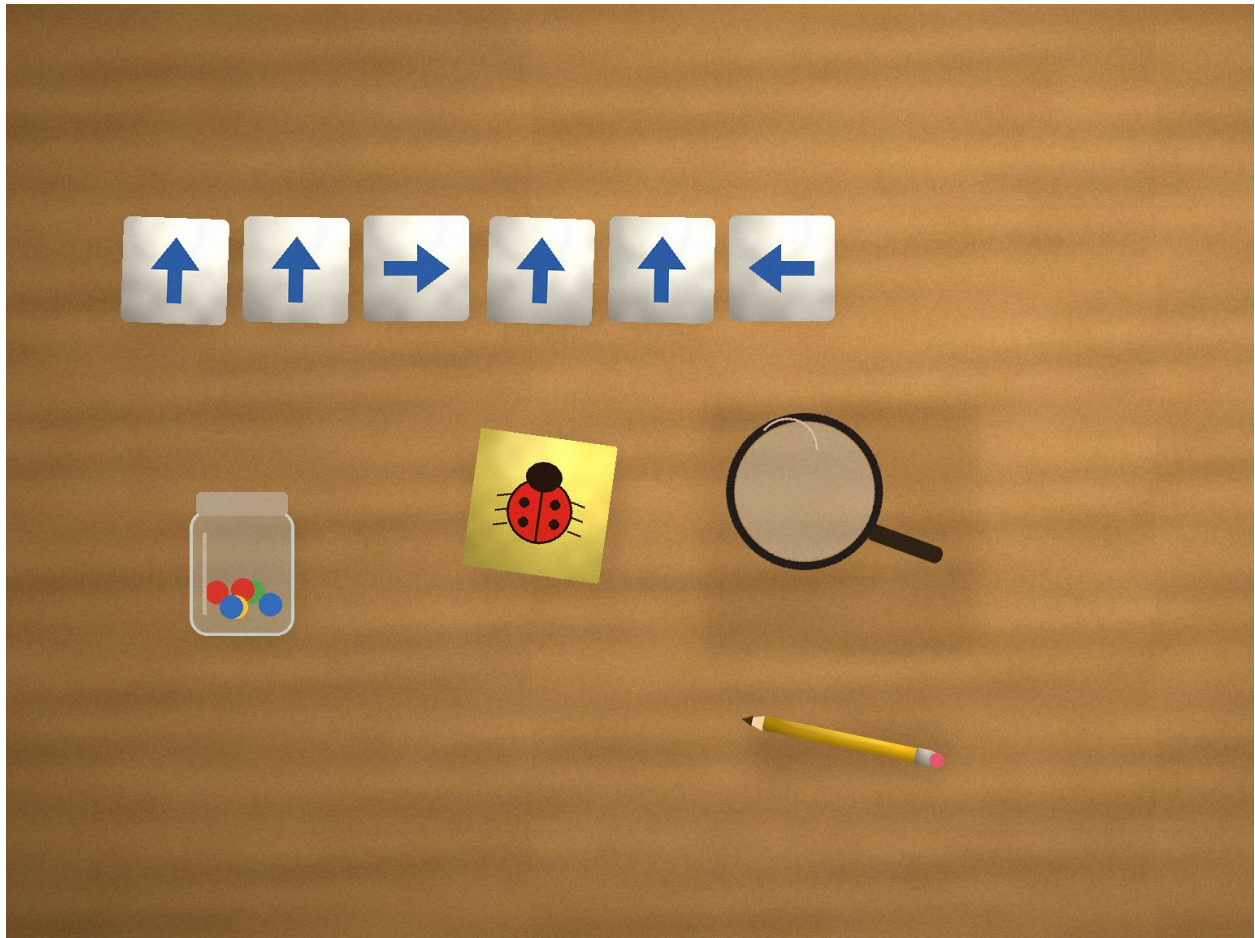
- ☐ The child extends a pattern they didn’t make, and invents one of their own.
- ☐ The child replaces a run of repeated steps with a loop, and can say that the toy still takes every step.

- ☐ The child follows an if-then rule, including the “else” part, and writes or says a rule of their own.
- ☐ The child names an event in daily life (“when the timer beeps, the oven turns off”).
- ☐ **AS-01 Exit Crumb Trio**, said (5–7) or written (8–10) at least twice: “I predicted... It did... Next I’d change...”
- ☐ **AS-03 Debug on the Spot**: you show a four-to-six-card program with one loop or if-then bug, and the child finds it, or you note it for next week.
- ☐ **AS-02 Show Me Two Ways (8–10)**: the same path, once with every step written out and once with a loop.
- ☐ **AS-05 Portfolio Pick**: the child chooses one before-and-after photo or rule card for the folder and says why.

Placement by skill, not birthday. A six-year-old who shortens strips confidently and plays Simon Says If is done enough, even if written if-then cards are still months away. A ten-year-old who is new to all this may need the 5–7 moves first. That’s not a step backwards. It’s the right step.

The habit going forward. Loops and if-thens will turn up in every chapter from here. When a later program gets long, ask, “What keeps repeating?” When a rule misbehaves, ask, “What happens when it’s not true?” Those two questions travel. The dated photos and rule cards go in the folder under **Computer Science**. Chapter 9 shows how.

Chapter 4



Bugs Are Clues: Debugging as a Habit

Why this matters

Picture a six-year-old at the kitchen table. She has laid seven arrow cards to walk a toy mouse to a paper cheese. She presses an imaginary go button and moves the mouse, card by card. On the fifth card the mouse turns the wrong way and bumps into the sugar bowl. She looks at it for one second.

Then she sweeps every card off the table with her arm.

Her mother doesn't say "don't do that," and she doesn't pick up all seven cards. She picks up one. "This was the card where it went wrong, I think. Let's look at just this one."

That small move is the whole chapter. A **bug** is a mistake in the instructions. **Debugging** is finding it and fixing it. Every person who writes programs meets bugs, every day, including people who've done it for decades. What separates a calm programmer from a frustrated one isn't fewer bugs. It's a habit: look for the *first* thing that went wrong, change one thing, and try again.

That habit is the second half of the book's focus line: **debug before you start over**. It's half of what this book is for.

There are good reasons to give it a whole chapter. Computer science teachers' standards expect a kindergartner to "identify a step in a sequence of commands that does not work as expected."² They expect a second grader to "debug a program," and a fourth grader to "debug a program incrementally and repeatedly throughout the development process."^{25,26} In plain words: find the broken step at five, fix whole programs at seven, and make checking a normal part of building by nine or ten. Researchers who study how children learn to program list testing and debugging as one of the core practices, alongside things like building in small pieces and reusing good ideas.²⁴

There's also a quieter reason. Bugs bring feelings. A child who expected the cat to jump and watched it spin instead may feel silly, upset, or sure they're "bad at this." The way you respond in that moment teaches more than any card. This chapter gives you words for it.

Here is the honest limit. Debugging practices staying with a problem when it's annoying. That's a good thing to practice. It isn't a proven way to give children "grit" in everything else, and nobody needs it to be. Habits tend to grow where they're used; decades of research on early programming found that skills don't spread to other areas all by themselves.³² So we practice debugging because it's how programs get made. If calm persistence turns up at the Lego table too, that's a welcome bonus.

This week, specifically. You can learn one question and one silence: "What did you expect it to do — and what did it do instead?" and then five quiet seconds. Your child can try finding a bug *you* planted, which is a much easier first bug than their own.

For the parent: understand it yourself

The plain version. The computer, robot, or toy did exactly what the instructions said. It didn't misbehave. It didn't decide to be difficult. Somewhere, the instructions said something different from what the child meant. Debugging is finding where.

The five-step shape. Every bug hunt in this book follows the same five steps, whether it's cards on a table or blocks on a tablet:

1. **What did you expect?** (This is why we predict before we run. Without a prediction, there's nothing to compare.)
2. **What did it do instead?**

3. **Where did it first go wrong?**
4. **Change one thing.**
5. **Run it again.**

Two extra moves help with stubborn bugs: **read before you run** (trace the program with a finger before pressing go) and **explain it to someone** (a parent, a sibling, a teddy bear). And one house rule turns bugs from bad news into good news: **a bug found is a win**. It earns a token in the Bug Jar.

One everyday picture. You follow a recipe and the pancakes come out flat. You don't throw away the cookbook, and you don't change the flour, the milk, the pan, and the heat all at once. You think back: "Did I forget the baking powder?" You find the first step that went differently from the plan, and next time you change that one thing. If the pancakes come out right, you know what the problem was. If you'd changed five things, you'd have good pancakes and no idea why.

One precise picture. Here's a card program for a toy on a grid, with the plan beside it:

Card	What the child meant	What the toy did
1 forward	move to square 2	moved to square 2
2 forward	move to square 3	moved to square 3
3 turn left	face the cheese	faced the wall
4 forward	move toward the cheese	bumped the wall
5 forward	reach the cheese	stayed stuck

Cards 4 and 5 *look* like the problem, because that's where the toy got stuck. But the first wrong step is card 3. The child meant "turn right." Cards 4 and 5 were fine; they just started from the wrong place. Fix card 3, and 4 and 5 work without changing. That's why we hunt for the *first* wrong step and not the place where it all looks bad. It's also why "start over" is usually a mistake: four of the five cards were right.

Wrong answers you should be able to hear. These come up in almost every family. Here's what each usually means and what to say.

1. **"It's broken."** The child thinks the toy or the app is faulty. This usually means they haven't yet separated "the machine" from "my instructions." Try: "It's doing exactly what we told it. Let's find the step."
2. **The child deletes everything at the first bug.** This usually means the bug felt like a judgment, and starting fresh feels safer than looking. Try: "Wait — most of this worked. Bugs are clues. Where did it *first* go wrong?" If something was deleted on a screen, use undo.
3. **The child changes five things at once.** This usually means they want it to work *now* and don't yet see that one change at a time tells you what fixed it. Try: "Change one thing, then run it. We're detectives, not magicians."
4. **The child finds a wrong step, but not the first one.** They fix card 5, which looked wrong, when card 3 was the cause. This usually means they looked where the trouble showed up instead of tracing from the start. Try: "Let's go back to the very beginning and walk it, one card at a time."

5. **“I’m bad at this.”** This usually means the child believes good programmers don’t make bugs. Try: “Everyone who writes programs finds bugs. You just found one. That’s the job.”

One more you may hear from yourself: **“Let me just fix it.”** It comes from love and from wanting the frustration to stop. It also takes the clue away from the child. Your hands in your lap are part of the lesson.

Your 5-minute parent warm-up. Alone, lay out a six-card arrow path with one wrong card in the middle. Run it with a toy and notice where the toy *looks* wrong versus where the first wrong card actually is. Then practice saying the chapter’s opening out loud: “What did you expect it to do — and what did it do instead?” Say it twice, slowly. Then practice the five-second wait with your eyes on the cards, not on the imaginary child. That’s the skill you’ll use tomorrow.

How to teach it this week

The shape of a bug-hunting Hour. The Computing Hour’s five steps already have debugging built in. This week, Bug Hunt gets the spotlight.

- **Unplug First (3–8 min).** BUG-01 Spot the Wrong Arrow: a short card strip with one wrong card. The child runs it with a toy and points to the wrong one.
- **Predict (2–5 min).** Fill in the Expected half of the Expected/Actual card (BUG-02). At 5–7, a drawing or a pointed finger is enough.
- **Build and Run (8–15 min).** The child runs a program: your planted-bug program on day one (BUG-04), their own program after that.
- **Bug Hunt (3–8 min).** The five steps. Ask, wait, point back at the cards. A bug found earns a Bug Jar token (BUG-08).
- **Save and Tell (2–5 min).** Photograph the fixed program, fill in the Actual half of the card, and the child tells someone one sentence: “The bug was the turn card.”

Exact wording you can use.

- Opening: “What did you expect it to do — and what did it do instead?”
- When they spot trouble: “Oh, a clue.”
- To find the first wrong step: “Where did it *first* go wrong? Let’s walk it from the start.”
- To slow down changes: “What’s one thing we could change?”
- After a change: “Shall we run it again?”
- After a fix: “You found a bug. That’s a token.”
- When they’re upset: “Bugs are clues, not grades. Let’s look at just this one card.”
- When you don’t know either: “I’m not sure. Let’s predict and run it to find out.” (That’s the method, and it’s honest.)

The first try-it. Start with BUG-04 The Parent’s Planted Bug, below. It’s your program and your mistake, so there’s nothing for the child to feel bad about. They get to be the detective who catches a grown-up out, which most children enjoy a great deal.

How to fade help across the week.

- **Day 1.** Your planted bug. You walk the program with them, card by card, and ask “here?” at each step.
- **Day 2.** Another planted bug. You ask the opening question and the “first wrong step” question, and that’s all.
- **Day 3.** The child’s own program. When a bug appears, you ask the opening and wait. You don’t point.
- **Days 4–5.** The child runs the five steps mostly alone. At 8–10, they write a Bug Log line (BUG-06) without being asked.

When to stop talking. The moment you see the bug, you’ll want to say where it is. That’s the moment to go quiet. Look at the cards, not at your child. Count to five in your head. If they’re tracing with a finger, they’re working, so let them. A good rule: if you’ve spoken more than one sentence since the bug appeared, you’ve probably said too much.

Age-band moves.

- **5–7.** Short programs (four to six cards) and visible bugs. The child points to the wrong card. You model saying “oops, a clue” about your own mistakes, out loud, often. The Bug Jar does a lot of work at this age; tokens are visible proof that bugs are good news. Predictions are spoken or drawn.
- **8–10.** Longer programs, on cards or on a screen. Add **BUG-03 One Change at a Time** as a strict rule, **BUG-06 Bug Log** as a written habit, and **BUG-07 Read Before You Run** before every green flag. Written predictions on the Expected/Actual card. At this age, the hardest bugs are often in programs the child is proud of, so plant your bugs in *your* programs for a while yet.

Place by skill, not birthday. A nine-year-old who melts down at every bug can stay with planted bugs and the Bug Jar for as long as that takes.

A week at a glance (a suggestion).

Day	Unplug First	Build and Run / Bug Hunt	Card codes
Mon	Spot the Wrong Arrow	The Parent’s Planted Bug; start the Bug Jar	BUG-01, BUG-04, BUG-08
Tue	Expected/Actual warm-up	A second planted bug; Explain It to Teddy	BUG-02, BUG-04, BUG-05
Wed	Spot the Wrong Arrow, harder	The child’s own card program	BUG-01, BUG-02
Thu	One-change race (8–10)	One Change at a Time; Bug Log; Read Before You Run	BUG-03, BUG-06, BUG-07
Fri	Retrieval: last week’s loop	Debug on the Spot check; count the Bug Jar	AS-03, BUG-08

Out	—	A bug-in-the-world walk	OUT-01 or OUT-05
-----	---	-------------------------	------------------

Siblings. Debugging pairs well. The driver runs the program; the navigator holds the Expected/Actual card and says “stop” at the first step that doesn’t match. Swap on a timer. The rule for both: point and say, but don’t touch the other person’s cards. And each bug found goes in the jar as a shared win, not a score.

Practice that actually builds learning

Blocked practice this week means lots of short bug hunts with one kind of bug: a wrong arrow, then a missing step, then an extra step. Mixed practice means the child doesn’t know which kind of bug is coming, so they have to look. Both matter. Start blocked; end the week mixed.

In-home — BUG-04 The Parent’s Planted Bug

- **Time:** about 10 minutes.
- **Band:** both.
- **Materials:** arrow cards and a tape grid with a toy (unplugged), or a short program you’ve built in ScratchJr or Scratch on your own device or in a project labeled as yours.
- **Safety and privacy:** if you’re using a screen, it lives in the shared room; the project is yours, not the child’s.
- **The fun:** catching a grown-up’s mistake, and being thanked for it.
- **The skill:** finding the first wrong step in someone else’s program, without the sting of it being your own.
- **What to save:** the child names the bug in one sentence, and you write it on the back of the card or in the Bug Log.

How it goes. Before the Hour, build a short program that should reach a goal: five or six cards, or five or six blocks. Put in one deliberate mistake: a left that should be a right, a missing forward, or an extra hop. Tell the child what the program is *supposed* to do. Ask for a prediction. Run it. Then ask the opening question and wait.

When they find the bug, let them fix it, with their hands on your cards. Run it again. Thank them properly: “I’d never have found that. You’re a good bug hunter.” Then plant another tomorrow. At 8–10, plant two bugs in one program and see whether they find the first one first.

In-home — BUG-05 Explain It to Teddy

- **Time:** about 5 minutes.
- **Band:** both.
- **Materials:** a stuffed animal, a toy, or a willing sibling; the buggy program.
- **Safety and privacy:** nothing to watch for.
- **The fun:** teaching a teddy bear to program.

- **The skill:** explaining each step out loud, which often reveals the bug mid-sentence.
- **What to save:** a note of the step where the child stopped talking. That pause is usually the bug.

How it goes. When the child is stuck, hand them the teddy. “Teddy doesn’t know what these cards do. Can you tell him, one card at a time?” Then stay quiet. Very often, somewhere around card three, the child will say “and then it turns left — oh. It should turn right.”

This isn’t only a children’s game. Professional programmers do the same thing. A well-known programming book calls it “rubber ducking,” after a colleague who kept a small rubber duck on his computer and talked his code through to it. The book’s explanation is plain: “the simple act of explaining, step by step, what the code is supposed to do often causes the problem to leap off the screen and announce itself.”³⁹ The same section adds that if you don’t have a person, “a rubber duck, or teddy bear, or potted plant will do.” Your child is in good company.

In-home — BUG-01 Spot the Wrong Arrow

- **Time:** about 5 minutes.
- **Band:** both.
- **Materials:** a strip of four to eight arrow cards with one wrong card; a toy; the tape grid.
- **Safety and privacy:** nothing to watch for.
- **The fun:** a quick puzzle with a clear “got it.”
- **The skill:** finding the first wrong step by running the program and watching.
- **What to save:** the child points to the card; a photo is enough.

How it goes. This is the warm-up version of the planted bug: short, fast, and unplugged. Run the strip with the toy. “Point to the first card that went wrong.” At 5–7, keep the strip short and the wrong card obvious (an arrow that points straight off the grid). At 8–10, use longer strips, and sometimes make the bug a *missing* card instead of a wrong one, which is harder to see.

In-home — BUG-02 Expected / Actual Card

- **Time:** about 5 minutes, split around Build and Run.
- **Band:** both. This card is used in every chapter.
- **Materials:** an index card divided into two boxes labeled “Expected” and “Actual.”
- **Safety and privacy:** first names only if you write a name on it.
- **The fun:** checking whether your guess came true.
- **The skill:** the prediction that makes debugging possible.
- **What to save:** the card itself. It’s one of the best records you’ll have.

How it goes. Before running, the child draws or writes what will happen in the Expected box: “The cat goes to the tree.” After running, they fill in Actual: “The cat went off the edge.” The gap between the two boxes is the bug. At 5–7, a drawing or your writing of their words is fine. At 8–10, they write it. If the two boxes match, celebrate that too. A program that does what you predicted is the other half of the game.

In-home — BUG-08 Bug Jar

- **Time:** ongoing, a few seconds per bug.
- **Band:** both.
- **Materials:** a clear jar and tokens (buttons, dried beans, or paper slips). Use large tokens if a toddler is near.
- **Safety and privacy:** small tokens can be a choking risk for little ones.
- **The fun:** watching the jar fill up.
- **The skill:** treating a bug found as a win, so children look for bugs instead of hiding from them.
- **What to save:** today's bug, named in a word or two on a slip of paper if you like, dropped into the jar.

How it goes. Every time someone finds a bug, a token goes in. That includes you. Celebrate the jar, not the bug-free run. When it's full, do something small together that has nothing to do with screens. The jar quietly turns the emotional meaning of a bug upside down: from "I got it wrong" to "I found one."

In-home — BUG-03 One Change at a Time (with BUG-06 Bug Log and BUG-07 Read Before You Run)

- **Time:** about 10–15 minutes.
- **Band:** 8–10 (a simple version works at 5–7 with cards).
- **Materials:** any small program on cards or a screen; a notebook page for the Bug Log.
- **Safety and privacy:** screen in the shared room; nothing personal in the program.
- **The fun:** solving it like a detective with a notebook.
- **The skill:** changing one thing, running, and noticing what happened; keeping a record.
- **What to save:** one Bug Log line: date, what went wrong, how it was found, what fixed it.

How it goes. Three habits work together here.

Read before you run (BUG-07). Before pressing go, the child traces the program with a finger and says what each part will do. On screen, this means reading the blocks before touching the green flag. It's the program-reading version of a prediction. Many bugs get caught before anything runs.

One change at a time (BUG-03). When the program misbehaves, the child changes exactly one thing, runs it, and says what happened: "I changed the 3 to a 4 and it went one step further." If they change several things, ask them to undo and try again with one. The point isn't obedience to a rule; it's knowing *which* change worked.

The Bug Log (BUG-06). A notebook page with four short columns:

Date	What went wrong	How I found it	What fixed it
2026-10-14	Cat walked off the stage	Read the blocks; move was 10 not 1	Changed 10 to 1

One line per bug is plenty. Over a few weeks, the log becomes a record of real thinking, and a nice thing to look back on when a child is sure they “never get anything right.”

Out-of-home — OUT-01 and OUT-05: A Bug-in-the-World Walk

- **Time:** about 15–20 minutes, as part of a walk or errand.
- **Band:** both.
- **Materials:** a folded index card and pencil.
- **Safety and privacy:** hold hands near roads and follow road rules; at automatic doors and elevators, stand to the side and let others pass; don’t test machines by blocking them.
- **The fun:** noticing that grown-up machines have bugs too.
- **The skill:** expected versus actual, in the wider world.
- **What to save:** one “I expected... / it did...” sentence about something you saw.

How it goes. This walk borrows the traffic-light walk from Chapter 3 (OUT-01) and the automatic-door detective from Chapter 1 (OUT-05), with a new question: “Where does a machine do something different from what people expected?” Keep it gentle and kind. You’re not hunting for things to complain about, and you’re not poking at anything.

Possible finds: an automatic door that opens when someone just walks past, or one that’s slow to notice a small child; a hand dryer that doesn’t start until you wave twice; a crossing where the walk light takes a long time; a self-checkout that says “unexpected item.” For each, ask the opening question: “What did people expect it to do — and what did it do instead?” At 8–10, add: “What do you think the rule inside it is? What might be the bug?” The door that misses a small child, for example, might have a sensor that’s set to notice grown-up heights. That’s a guess, not a fact, and that’s fine. Guessing at the rule is the skill.

Rainy-day alternative. Hunt for household bugs: a timer that beeps too early, a toy that plays the wrong sound, instructions on a box that skip a step. Same question, same card.

Talk box (this chapter)

Opening: “What did you expect it to do — and what did it do instead?”

Follow-ups (pick three to five):

1. **Prediction:** “What will it do if we run it again just as it is?”
2. **Evidence:** “What did you see that tells you that?”
3. **Location:** “Where did it *first* go wrong? Point to it.”
4. **Change:** “What’s one thing we could change?”
5. **Revise:** “Shall we run it again? You can change your mind about where the bug is.”

How to wait. Count to five in your head after you ask. When the child is tracing a program, give them longer. That silence is thinking time, and debugging needs more of it than most questions.⁵ Look at the cards or the screen, not at the child, if the silence feels heavy.

What a stuck silence usually means. Often it’s fear: the child believes a bug means they’re bad at this, and saying where it is feels like confessing. Sometimes there was no prediction, so

there's nothing to compare. Sometimes they're waiting for you to fix it, because you have before. Sometimes the program is too long to see all at once.

The move. Make it smaller. “Is it this card or this card?” Or: “Point to where the toy was when it first looked wrong.” Or hand them the teddy. If the program is too long, cover the second half with a sheet of paper and run just the first half.

A sample bug hunt (illustration)

- Parent: “What did you expect it to do — and what did it do instead?”
- Child: “It was supposed to go to the cheese but it went into the wall.”
- Parent (waits, counting to five in their head): “Where did it first go wrong?”
- Child: “Here — no, here. It turned the wrong way.”
- Parent: “What’s one thing we could change?”
- Child: “This arrow.”
- Parent: “Shall we run it again?”

The parent never names the bug and never touches the cards. That’s the whole technique.

When bugs bring big feelings

This chapter carries the emotional side of mistakes, because debugging is where those feelings show up first. Three patterns are common at this age. None of them is a problem with your child.

Frustration. Tears, an angry sweep of cards, a slammed tablet cover. This usually means the bug arrived when the child was tired or the program was too long. Stop. Have a snack. Come back in ten minutes or tomorrow. Before you stop, if you can, praise the bug found rather than the fix you didn’t get to: “You saw it went wrong at the turn. That’s the hard part.” Five-year-olds give an activity about five to ten minutes of focused attention, so short goes are normal, not a failing.⁹

Perfectionism. The child won’t run a program until it’s “right,” or won’t predict in case they’re wrong. Planted bugs help here, because the mistake is yours. So does saying “oops, a clue” about your own errors in front of them, often and cheerfully. A child who sees you find your own bugs calmly learns that bugs are normal.

Giving up. “I can’t do it. You do it.” This usually means the child has learned that if they wait, a grown-up will step in. Kindly decline: “I’ll sit right here. You find the first wrong step; I’ll ask questions.” Then offer a smaller version of the problem. If they still can’t, show a short demo on *your own* cards, put them away, and hand theirs back.

What to say, in any of these moments: “Bugs are clues, not grades.” “Everyone who writes programs finds bugs.” “You found one. That’s the job.” What to avoid: “It’s easy, look,” and any version of “you’re just not a computer kid.” Nobody is born one.

One incorrect example to diagnose

Show the child this story, and say it happened to “a robot-builder who needs help”:

The builder's robot was supposed to walk to the door. It walked into the couch. The builder was upset, so they changed the first card, the third card, and the last card, and added two new cards. Now the robot walked to the door. "Fixed it!" said the builder. The next day, they tried to make the robot walk to the window, and it went into the couch again.

Ask: "What did the builder do when they found the bug?" (They changed lots of things at once.) "Do they know which change fixed it?" (No.) "What would you tell them to do next time?" (Change one thing, then run it.) At 8–10, add: "What should they write in their Bug Log?"

Blocked and mixed practice

Blocked. Three planted bugs of the same kind in a row: three wrong arrows, then on another day three missing steps. The child gets a feel for what each kind of bug looks like.

Mixed. A small stack of buggy card strips, each with a different kind of bug: a wrong arrow, a missing step, an extra step, a loop that repeats one time too many, an "if" with no "else." The child runs each, finds the bug, and names what kind it was. Now they're deciding, not just spotting.

Quick retrieval. Next week, start the Hour with one planted bug from this week's kinds. One minute. It keeps the habit alive.

Specific encouragement for the week

- **What you can learn this week:** the opening question, the five-second wait, and the habit of saying "oops, a clue" about your own mistakes.
- **What your child can try today:** find the bug you planted in a five-card path. It takes five minutes, and they'll probably enjoy catching you out.

For the student

A **bug** is a mistake in a program's instructions. Not a mistake in *you*. A mistake in the instructions.

Here's a secret: everyone who writes programs finds bugs. Every day. Finding them and fixing them is a big part of the job. When you find a bug, you haven't failed. You've found a clue.

Good bug hunters ask five questions:

1. What did I expect it to do?
2. What did it do instead?
3. Where did it *first* go wrong?
4. What's one thing I could change?
5. What happens when I run it again?

Here's an example. Your toy was supposed to go forward, forward, turn right, forward, to reach the cheese. Instead it bumped the wall. You walk through the cards from the start. Forward: fine.

Forward: fine. Turn... left? That's the first wrong step. You change that one card to "turn right." You run it again. Cheese.

Notice you didn't throw away the whole path. Three of the cards were right all along.

Try this today. Ask a grown-up to hide a bug in a card path. Find it. Change one card. Run it again. Put a token in the Bug Jar.

Explain it back. Tell a teddy bear (or a person) why we look for the *first* wrong step, not the last one.

Challenge. Make a card path with a bug hidden in it, and give it to a grown-up to fix. Watch them. Did they find the first wrong step?

If it isn't clicking

Diagnostic 1: Meltdowns at every bug. Tears, sweeping cards away, "I hate this." This usually means bugs still feel like a judgment, or the programs are too long for today's energy.

Next move: stop, snack, and return later or tomorrow. When you come back, use only planted bugs in *your* programs for a few days. Keep programs to four cards. Fill the Bug Jar fast. Praise the bug found, not the fix.

Diagnostic 2: The child changes everything. Five changes, then either it works by luck or it's worse than before. This usually means they want the problem gone and haven't yet seen why one change at a time matters.

Next move: undo back to the buggy version. Make it a game: "We get one change. Choose carefully." At 5–7, physically hand them one card at a time. At 8–10, the Bug Log helps, because there's only room for one fix per line.

Diagnostic 3: You keep fixing it. This one is about the grown-up, and it's very common. You see the bug, the silence feels long, and your hand reaches out. This usually means you care and the frustration is hard to watch.

Next move: hands in your lap, literally. Ask "Where did it first go wrong?" and look at the cards. If you need to show something, do it on your own cards, then put them away. If you've already moved a card, say so: "I moved that one. Let's put it back so you can find it."

When to slow down, go ahead, or step back. If bugs still bring tears after a week, slow down: planted bugs and the Bug Jar only, very short programs, lots of "oops, a clue" from you. If the child finds the first wrong step on their own most of the time, go ahead to Chapter 5, where the bugs move onto a screen. If screen bugs are causing distress, step back to cards for a few Hours. The habit is the same; cards are easier to see.

The rescue order. Ask, wait, hint, then a short demo on *your own* cards or device, then hand it back. The child's program stays the child's.

Tools, including AI

What you need. Arrow cards, tape, a toy, an index card for Expected/Actual, a jar and some tokens, a teddy bear. For 8–10, a notebook for the Bug Log. On a screen, the undo button is a debugging tool too, because it lets a child back out of a five-change tangle.

AI. The child doesn't use AI to find bugs. As the AI box in **The Computing Hour** explains, you may, on your own account, look at a screenshot of a saved project's code (the code, never the child) to understand a bug yourself before you ask your questions. The child still finds and fixes it.

What “done enough” looks like

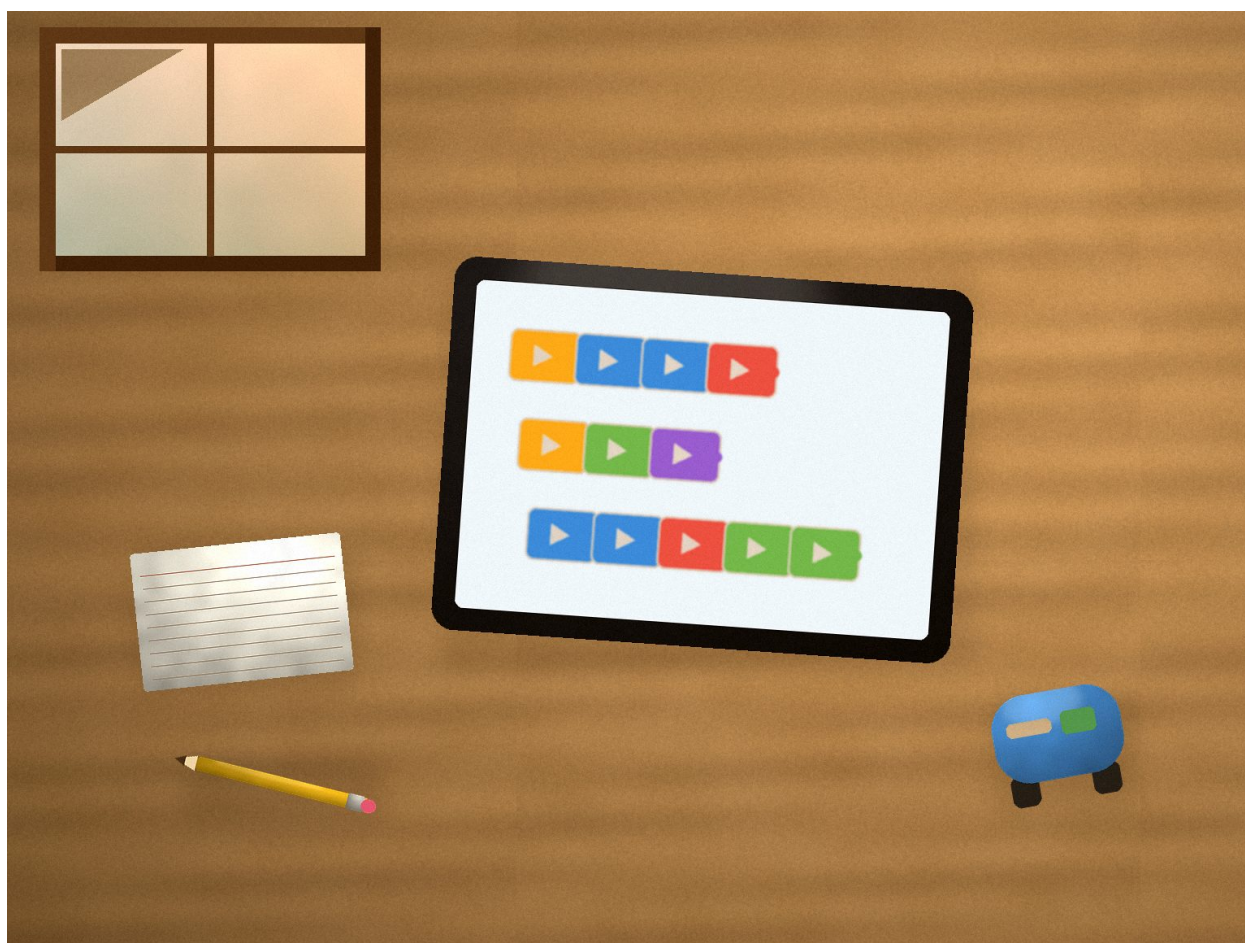
Before you move on, look for these across the week:

- ☐ The child finds the first wrong step without help at least once.
- ☐ The child makes one change before rerunning, at least once, without being reminded.
- ☐ The Expected/Actual card has been used before running, and the gap between the boxes was talked about.
- ☐ At least a few tokens are in the Bug Jar, including one for a bug you made.
- ☐ (8–10) At least one Bug Log line, written by the child.
- ☐ **AS-03 Debug on the Spot:** you show a four-to-six-step program with one bug; the child finds it, or you note it for next week.
- ☐ **AS-04 Teach It Back:** the child explains the five bug-hunting questions to a sibling or a teddy, and you write down one sentence they say.
- ☐ **AS-01 Exit Crumb Trio** at least twice: “I predicted... It did... Next I'd change...”

Placement by skill, not birthday. A five-year-old who points to the wrong card and puts a token in the jar is done enough. A ten-year-old who still sweeps the cards away needs more planted bugs first, and that's fine.

The habit going forward. This chapter never really ends. Bug Hunt is in every Computing Hour, and the Bug Jar stays on the shelf. When a later chapter goes wrong, come back to the five questions. The Expected/Actual cards and Bug Log pages go into the folder under **Computer Science**; Chapter 9 shows how to keep them.

Chapter 5



First Block Coding — and Robots If You Like

Why this matters

Picture a five-year-old with a tablet on the kitchen table. She drags a green flag block into the coding area, then three blue “move right” blocks behind it. She taps the flag. The cat walks across the screen. She roars with delight, and then, without being asked, taps it again to watch it a second

time.

Now picture a nine-year-old at the family computer. Before he touches anything, he plays a small starter game three times and reads its instructions. Then he opens the code, finds the block that makes the character jump, and changes one number. He predicts the jump will be higher. It is. He grins and writes “jump 10 → 15” in his notebook.

Both children are programming: writing real instructions that a real computer follows. That’s what **block coding** offers. Instead of typing words that must be spelled exactly right, children snap together blocks that each mean one instruction. No typos. The ideas are the ones they’ve met on cards in Chapters 2 to 4: steps in order, loops, events, and bugs. A child who has shortened an arrow strip with a repeat card already knows what the Repeat block will do. The screen isn’t a new subject. It’s a new window onto the same ideas.

What block coding does and doesn’t do. Young children who are taught block coding well do get better at programming. In a large school trial of a ScratchJr curriculum, with more than a thousand children from kindergarten to second grade, children who had the lessons improved their programming skills compared with children who didn’t. The same trial didn’t find a difference on a test of general thinking skills, and the study was run by the curriculum’s own developers.^{29,30} This is a useful study, not a promise that every home will see the same result. So we teach block coding because programming is worth knowing, and because it’s a joyful way to make things. Anything else that comes along is a welcome extra.

Robots, if you like. Some families want a small robot that crawls across the floor following the child’s program. That’s lovely, and it’s optional. One is plenty. None is fine. A person following arrow cards (a Kidbot) does the same teaching job for free, and this chapter shows you both.

This week, specifically. You can learn to open one free app, build a three-block program by yourself, and then hand the device over and keep your hands in your lap. Your child can try making a cat walk across a screen, with a prediction first.

For the parent: understand it yourself

The plain version. A block program is a stack or row of blocks. Each block is one instruction. The program usually starts with a **trigger block**, such as “start when the green flag is tapped,” which is the event that sets it going. After that, the computer runs the blocks in order. Some blocks wrap around others, like a Repeat block that runs whatever is inside it a set number of times. That’s the whole idea. Everything else is detail.

One everyday picture. Think of the picture instructions that come with a building-brick set. Each panel shows one step. You follow them in order. If panel 4 is missing, the model goes wrong at panel 4, and everything after it is built on a mistake. A block program is the same, except the computer follows the panels instead of your child, and it never skips one or guesses.

One precise picture. Here’s a ScratchJr program, the kind a five-year-old might build. In ScratchJr the blocks snap together in a row, read left to right:

Start on Green Flag → Move Right 3 → Hop → Say “Hi!” → End

Tap the green flag and the character moves three grid squares right, hops, and shows “Hi!” in a speech bubble.³⁷ Change the 3 to 5 and it moves five squares. Swap Hop and Move Right and it hops first. Every change is visible right away, which is why this is such a good place to practice predicting.

Here’s a Scratch program, the kind a nine-year-old might build. In Scratch the blocks stack top to bottom:

when green flag clicked repeat 10 move 10 steps next costume

The sprite walks forward with its legs moving. The “repeat 10” is the same loop your child made with a repeat card in Chapter 3.

Which tool, at which age. Here are the three main options, each free.

- **ScratchJr, for about 5–7.** A free tablet app made for young children ages 5–7.⁴ Its blocks use pictures, not words, so pre-readers can use it.²⁸ No account is needed. Projects, including any photos or sound recordings, are stored only on the tablet; the app’s privacy policy says they are not sent to the makers.⁴⁰ That’s good for privacy. It also means that deleting the app deletes the projects, and that everyone on one tablet shares the same set of projects, so name them clearly.⁴¹ ScratchJr has an official free curriculum called *Coding as Another Language* if you’d like structure.⁴
- **Scratch, for about 8–10.** A free program from the Scratch Foundation, first developed at MIT in 2007, with blocks that use words.⁴² The Foundation lists its age range as 8 to 13 and up.²⁸ There’s a free Scratch app that lets you create and save projects without an internet connection; it runs on many computers and on Android tablets, but not on phones.⁴³ This book’s default for 8–10 is that offline app, or an online account that you create and confirm, with nothing shared publicly. Since November 2024, anyone under 16 who signs up online needs a parent or guardian to confirm by email before they can share or comment; an unconfirmed account can still make and save projects.⁴⁴ Choose a username that doesn’t reveal your child’s real name.⁴⁵
- **CodeAI (formerly Code.org) CS Fundamentals, for a structured path at 5–10.** The organization renamed itself CodeAI on 2 June 2026.²¹ Its CS Fundamentals curriculum offers six courses, A to F, roughly one per elementary grade, plus two self-paced “Express” courses, one of them for pre-readers. Each course has 13 to 17 lessons designed for 45-minute class periods, and the curriculum and platform are offered at no cost.³³ At home, split a 45-minute lesson across two Computing Hours. You create the account and hold the login. CodeAI’s built-in AI Tutor is currently offered in a few courses for students 13 and up, not in CS Fundamentals.⁴⁶ If an AI helper ever appears in a course your child uses, it stays off or you run it yourself.

These aren’t ranked. ScratchJr is the default for 5–7 and Scratch for 8–10, with CodeAI as the choice for families who’d like a ready-made sequence. Combining them is fine. Place by skill, not birthday: a confident seven-year-old reader may be ready for Scratch; a nine-year-old who finds reading hard can do excellent work in ScratchJr or CodeAI’s pre-reader course. ScratchJr is not “babyish.” It’s a real programming tool sized for young hands.

Why touch for the youngest. In a study of young children using a mouse, dragging and dropping

was faster and less error-prone than clicking, moving, and clicking again.⁴⁷ Blocks are drag-and-drop, and a tablet removes the mouse altogether. Slowness with a mouse at five is normal.

Why “tour, then remix, then make.” For screen work at 8–10, this book uses a three-step path. First, **tour** a working project before touching it: play it, read it, and look at its code. Second, **remix**: change one thing on purpose and predict what will happen. Third, **make** a small project of your own from a plan on paper. The idea comes from teachers and researchers who found that children do well when they *use* something that works, then *modify* it, then *create* their own.⁴⁸ For the youngest, research on early coding points the same way: the clearest benefits for preschoolers and first graders came from structured activities and from tangible ones like robots, with more open-ended Scratch-style coding showing effects for older children.²² So at 5–7, keep it short and guided; at 8–10, open it up.

Wrong answers you should be able to hear.

1. **The child taps the green flag without predicting.** This usually means the screen is exciting and the habit isn’t set yet. Try: “Hands on the table. Tell me what it’ll do first.” Let them hold a finger over the flag but not tap until they’ve guessed.
2. **“The cat is broken.”** This usually means the child sees the character, not the blocks. Try: “The cat does exactly what the blocks say. Point to the block that made it do that.”
3. **A giant script and no idea where the bug is.** Thirty blocks went in before anything ran. Try: “Let’s run just the first three blocks.” Build a little, run a little.
4. **“Can we put it online?”** This usually means they’re proud, which is lovely. Try: “Let’s show Grandma on the tablet. Projects stay in the family.” Public sharing isn’t part of this book at 5–10.

Your 5-minute parent warm-up. The night before, alone, open the app you’ll use. Build the ScratchJr program above (or, for Scratch, the “repeat 10” walk). Run it. Change one number and predict what will happen before you tap the flag. Then delete it, so your child starts with a blank screen. Practice tomorrow’s opening out loud: “Point to the block that makes that happen — how do you know?” That’s all the preparation you need.

How to teach it this week

The shape of a first screen Hour. Use the Computing Hour’s five steps. The screen comes on only for Build and Run.

- **Unplug First (3–8 min).** Be the character. Lay out a tape grid and have the child walk your instructions: “move right 3, hop.” Or, for 8–10, draw a three-box plan of what the program should do.
- **Predict (2–5 min).** Expected/Actual card (BUG-02): “What will the cat do when we tap the flag?” At 8–10, Read Before You Run (BUG-07): trace the blocks with a finger first.
- **Build and Run (8–15 min).** The child’s hands on the screen. At 5–7, about 10–15 minutes of screen at most, and many Hours have none. At 8–10, a project day can run longer, up to about 25 minutes, with you nearby.
- **Bug Hunt (3–8 min).** The five questions from Chapter 4. Point at blocks, not at the character.

- **Save and Tell (2–5 min).** Save the project with a date-first name, such as “2026-10-14 Mia cat walk.” Screen off. Then one sentence to someone: “I made the cat hop with a repeat.”

Exact wording you can use.

- Opening: “Point to the block that makes that happen — how do you know?”
- Before running: “Finger off the flag. What will it do?”
- During: “Your hands, your program. I’m right here.”
- When they’re stuck: “Which block starts everything?”
- For a bug: “What did you expect — and what did it do instead?”
- For a change: “What would happen if we moved that block?”
- At the end: “Save it, name it, screen off — then tell me one thing.”

The first try-it. For 5–7, start with JR-01 Move the Cat. For 8–10, start with SCR-01 Tour Before You Touch. Both are below.

How to fade help across the week.

- **Day 1.** A one-minute tour on a project labeled as yours: where blocks live, how to drag one, where the flag is. Then the child’s blank screen.
- **Day 2.** No demo. Just the opening and the predict question.
- **Day 3.** The child plans on paper, then builds from the plan.
- **Days 4–5.** A small project of their own (JR-04; SCR-02 then SCR-05). You only ask questions.

When to stop talking. Once the screen is on, talk less than you think you should; your voice is competing with a moving cat. Ask, then wait. Point with words (“look near the start of your blocks”), never with a finger on the screen. If your hand keeps drifting toward the tablet, sit on it. It works.

Age-band moves.

- **5–7.** ScratchJr, about ten-minute screen blocks inside an Hour that’s mostly unplugged. One character, three to six blocks, one trigger. Spoken predictions. First names only in any text or recording. If the child records their voice in a project, that’s fine: it stays on the tablet, and the project stays in the family. Stop while it’s still fun.
- **8–10.** Scratch (offline app or parent-confirmed account, nothing shared publicly) or CodeAI. Tour, then remix, then a paper-planned project. Written predictions and a Bug Log (BUG-06). A project can stretch across two Hours; save at the end of each.

Siblings and one device. Use driver and navigator with a timer. The driver’s hands are on the screen. The navigator holds the paper plan and the Expected/Actual card and reads out what should happen next. Swap every five minutes or so. One family shared device in a common room is all you need.

A week at a glance (a suggestion).

Day	Unplug First	Build and Run (5–7 / 8–10)	Card codes
Mon	Be the cat on a tape grid	Move the Cat / Tour Before You Touch	JR-01 / SCR-01
Tue	Loop warm-up from Ch. 3	Forever Fish / Remix One Thing	JR-05 / SCR-02
Wed	Story in three boxes / Game on paper	Three-Page Story begins / Maze game begins	JR-04 / SCR-05
Thu	Read the blocks aloud	Bump and Message / maze game continues	JR-06 / SCR-05, BUG-07
Fri	Planted bug on screen	Show the project; short check	BUG-04, AS-05
Out	—	Library visit	OUT-03

Practice that actually builds learning

Blocked practice this week means repeating one new move until it's easy: three short goes at “trigger plus move blocks.” Mixed practice comes later in the week, when the child has to choose whether a project needs a loop, an event, or just more steps.

In-home — JR-01 Move the Cat

- **Time:** about 10 minutes of screen inside Build and Run.
- **Band:** 5–7 (and older beginners).
- **Materials:** a tablet with ScratchJr; the Expected/Actual card.
- **Safety and privacy:** tablet in the shared room; no account; first names only.
- **The fun:** making a character move with your own instructions.
- **The skill:** a trigger block plus motion blocks, in order; predicting a distance.
- **What to save:** the project, saved with a clear name, and the answer to “how many steps did it move?”

How it goes. Open a new project. Show where the yellow trigger blocks and blue motion blocks are, once. Ask the child to put a Start on Green Flag block down, then one Move Right block. Before tapping the flag, ask for a prediction: “How far will it go?” Tap. Then add more blocks, one at a time, predicting each time. Change a number. Swap two blocks. Each small change is a tiny experiment.

If the cat goes off the edge of the screen, celebrate: that's a bug, and it earns a token. ScratchJr has a Go Home block that sends a character back to its starting spot, which is handy when a cat wanders off.³⁷

In-home — SCR-01 Tour Before You Touch

- **Time:** about 10 minutes.
- **Band:** 8–10.
- **Materials:** Scratch (offline app or parent-confirmed account) with a small starter project you’ve previewed or built yourself; a notebook.
- **Safety and privacy:** device in the shared room; the parent handles any login; nothing is shared publicly.
- **The fun:** being a detective in someone else’s game before changing it.
- **The skill:** reading a program before running or changing it.
- **What to save:** one event named in the notebook (“when space key pressed → jump”).

How it goes. Four steps, in order. Read the project’s **title** and any **instructions**. Guess its **purpose**: what is this game for? Then **play** it, two or three times. Only after that, open the code and look at the characters (called *sprites* in Scratch) and the **events** that start each script. Ask the opening question about one thing that happened in the game: “Point to the block that makes that happen — how do you know?” The tour is based on a teaching routine developed for upper-elementary Scratch classrooms.⁴⁹

In-home — SCR-02 Remix One Thing

- **Time:** about 15 minutes.
- **Band:** 8–10.
- **Materials:** the starter project from the tour; the Expected/Actual card.
- **Safety and privacy:** as above; the remix stays in the family.
- **The fun:** changing a game and seeing your change come to life.
- **The skill:** changing one thing on purpose, with a prediction.
- **What to save:** one line: “I changed ____ and it ____.”

How it goes. Choose one thing to change: a speed, a costume, a sound, a number. Write the prediction first. Run it. Did it match? Then one more change, with a prediction. If the child wants to change five things at once, remind them of Chapter 4: one change, then run it.

Remixing is a normal part of learning to program, and a good moment to talk about whose work it is. The starter game was someone else’s; the change is the child’s. Saying so in the file name (“starter game by ____; jump change by Sam”) is honest record-keeping.

In-home — JR-04 Three-Page Story

- **Time:** about 20 minutes, possibly across two Hours.
- **Band:** both (ScratchJr).
- **Materials:** the folded-paper Story in Three Boxes from Chapter 2 (DECOMP-03); the tablet.
- **Safety and privacy:** characters can be animals or invented people; first names only; any voice recording stays on the tablet.
- **The fun:** a story that moves.
- **The skill:** turning a paper plan into a program, one page per box; the Go to Page block.

- **What to save:** the child retells the story while it plays; save the project.

How it goes. Each box of the child’s three-box comic becomes a ScratchJr page. At the end of page one’s script, a Go to Page block moves the story on.³⁷ One page per Hour is fine. The paper plan stays on the table so the child can check: “Does page two match box two?”

In-home — SCR-05 Paper-Planned Maze Game (with BUG-07 Read Before You Run)

- **Time:** two or three Hours.
- **Band:** 8–10.
- **Materials:** the Game on Paper plan from Chapter 2 (DECOMP-04); Scratch; the Bug Log.
- **Safety and privacy:** as above; nothing shared publicly.
- **The fun:** a real game you designed and can play.
- **The skill:** building from a plan, testing in small pieces, and debugging along the way.
- **What to save:** the project, saved with a date-first name at the end of each Hour, and at least one Bug Log line.

How it goes. The paper plan names the characters, the goal, and the rules, written as “when... then...” sentences (“when the cat touches the wall, then it goes back to the start”). Build one rule at a time. After each rule, use **Read Before You Run:** trace the blocks with a finger and predict, then press the green flag. Test, then add the next rule. When something goes wrong, the Bug Log gets a line.

Arrow-key movement (SCR-03 Arrow-Key Sprite) usually comes first: “when right arrow key pressed, change x by 10,” and so on for four directions. Scratch’s event blocks include “when [key] pressed” and “when this sprite clicked.”³⁸ A score (SCR-04 Score Keeper) can come later. A score is a **variable**, a named box that holds a number that changes. Fourth-grade standards include tracing how variable values change in a program,²⁶ and CodeAI introduces variables in its final course;³³ there’s no need to hurry them.

Smaller ScratchJr cards for the week

These fit any day once JR-01 is comfortable:

- **JR-02 Say Hello** (5–7): a Say block shows a speech bubble. The child may record their own voice with the record-sound tool; it stays on the tablet.⁴⁰ Use first names or no names.
- **JR-03 Two Characters Dance** (5–7): two characters, two scripts, both starting on the green flag. “Who moved first?” is a lovely question, because the honest answer is “both at once.”
- **JR-05 Forever Fish** (5–7): a fish swims with a Repeat Forever block. The best question: “How do you stop it?” (One answer: ScratchJr’s Stop block.)³⁷
- **JR-06 Bump and Message** (both): Start on Bump runs a script when another character touches this one; Send Message and Start on Message let one character tell another to start.³⁷ “What event started it?”

Robots, if you like

Robots are optional. If you have one, or can borrow one, here's how to use it well. If you don't, BOT-01 does the same job with a person.

One is plenty, and borrow first. Ask your library, co-op, or homeschool group. Prices change, so this book leaves them out; check the current price and age label yourself. Small parts are a choking risk at 5–7. A grown-up handles batteries and charging, and nobody opens a battery or power supply.

In-home — BOT-01 Kidbot

- **Time:** about 15 minutes.
- **Band:** both.
- **Materials:** a masking-tape grid on the floor, arrow cards, and a person who agrees to be the robot.
- **Safety and privacy:** clear the floor; slow steps; no running robots.
- **The fun:** programming a sibling or a grown-up who must obey exactly.
- **The skill:** a robot program with no robot; the same idea as a floor robot, for free.
- **What to save:** a photo of the card program, cropped to the cards.

How it goes. The child lays cards; the human robot follows them exactly, one square per forward card. This is a free CS Unplugged activity, marked for ages 5 to 10,³⁴ and the best warm-up for a floor robot.

In-home — BOT-03 Floor-Robot Treasure Map (and BOT-04 Robot Obstacle Course)

- **Time:** about 15–20 minutes.
- **Band:** 5–7 for the treasure map; both for the obstacle course.
- **Materials:** a button-programmed floor robot of the kind sold for preschool classrooms³⁵ (or a Kidbot for the obstacle course); a paper map drawn in squares the size of the robot's step; books for walls.
- **Safety and privacy:** clear floor; check the age label; adult handles batteries.
- **The fun:** a robot that drives to treasure on your instructions.
- **The skill:** planning on paper, entering the program, running, and debugging.
- **What to save:** did it reach the treasure? How many tries? (A photo of the paper plan.)

How it goes. Plan on paper with arrows, then press the robot's buttons to match. Predict where it will stop. Run it. If it misses, use Chapter 4's questions. If your robot remembers its last program, learn how to clear it before you start; a robot running yesterday's program is a classic first bug. For the obstacle course (BOT-04), add books as walls and count the debugging tries. More tries just means more bugs found.

In-home — BOT-05 Light-Up Name Badge (and BOT-06 Shake-a-Die)

- **Time:** about 20 minutes.
- **Band:** 8–10.
- **Materials:** a micro:bit (a small programmable circuit board with lights and buttons, generally suited to about age 8 and up)⁵⁰ and a computer with Microsoft MakeCode, a free block editor in a web browser.⁵¹
- **Safety and privacy:** the parent sets up the board and the computer; first name or nickname only on the badge; adult handles the battery pack.
- **The fun:** your name scrolling across tiny lights.
- **The skill:** input starts a program (a button press); output shows the result (the lights).
- **What to save:** a photo of the blocks, and an answer to “what input starts it?”

How it goes. MakeCode has short built-in tutorials, including a name tag and a dice project.⁵² Its editor also shows a picture of the board, a simulator, that runs the program on screen before you send it to the real board.⁵¹ That’s a perfect place to practice predicting: “What will the simulator do when you press button A?” Then send it to the board and check. Shake-a-Die (BOT-06) uses the board’s motion sensor: shake it (input) and a random number appears (output). Chapter 6 uses it again to talk about input and output.

One thing to know: MakeCode’s editor can show an AI button, “Explain with AI,” when a program hits an error.⁵² Those are for you, if anyone. The child debugs with Chapter 4’s five questions.

BOT-02 Cardboard Robot Design is an unplugged cousin: the child designs a robot from a box and labels what it senses (inputs) and what it does (outputs). It fits here or in Chapter 6.

Out-of-home — OUT-03 Library Computing Visit

- **Time:** about 30 minutes.
- **Band:** both.
- **Materials:** a library card in the parent’s name; a short list of things to look for.
- **Safety and privacy:** stay together; the parent logs in to any library computer; no new accounts for the child; follow the library’s posted rules.
- **The fun:** finding a picture book about coding or robots, and seeing computers used by the whole community.
- **The skill:** noticing computing in the wider world; choosing a book that connects to the Hour.
- **What to save:** the title of the book chosen, written on the Expected/Actual card or in the folder.

How it goes. Ask a librarian for picture books about coding, robots, or how computers work. Look at the library’s computers together: who uses them, and what for? Ask whether the library lends robots or runs coding clubs. For a family without a device at home, a short session on a library computer is a fine Build and Run. That evening, read the book together and ask: “Was there a bug in this story?”

Talk box (this chapter)

Opening: “Point to the block that makes that happen — how do you know?”

Follow-ups (pick three to five):

1. **Prediction:** “What will happen when you tap the flag?”
2. **Evidence:** “What did you see that tells you it’s that block?”
3. **Location:** “Which block starts everything?”
4. **Change:** “What would happen if we moved it? What’s one thing you’d change?”
5. **Revise:** “You can change your guess. Shall we run it again?”

How to wait. Count to five in your head after you ask, as in the Computing Hour. On a screen, waiting is harder, because something is usually moving. If the character is still dancing, press stop first, then ask. Look at the blocks, not at the child.

What a stuck silence usually means. The screen is more interesting than the question (turn the tablet face-down for a moment, or press stop). There are too many blocks to see at once. The child skipped the prediction, so there’s nothing to compare. Or they’re waiting for you to fix it, because you have before.

The move. Press stop. Ask them to put a finger on the first block, then the next, saying what each does. Or offer a choice of two: “Is it this block or this one?”

Blocked and mixed practice

Blocked. Early in the week, several short builds with the same shape: one trigger, a few motion blocks. Three different cats walking three different distances. The move becomes automatic.

Mixed. Later, give a small challenge card and let the child decide what it needs: “Make the fish swim forever” (a loop), “Make the cat say hi when the dog bumps it” (an event), “Make the cat walk to the tree” (just steps), “Make the cat jump three times when tapped” (an event *and* a loop). Ask, “What does this one need?” before they build.

Quick retrieval. Start next week’s Hour with one minute on paper: “Draw the block that starts a program.” Then go.

One incorrect example to diagnose

Show the child this situation and ask for help:

A cousin opened a ready-made game, changed nothing, and saved it as “My Game by Leo.” Then they saved another project as “Untitled.”

Ask: “Whose work is the first game?” (Someone else’s, unless Leo changed something; then the change is Leo’s.) “How could Leo name it honestly?” (“Starter game by ___, remixed by Leo,” and only if he changed something.) “What’s wrong with ‘Untitled’?” (Nobody will find it again.) At 8–10, add: “If a computer program had made the whole game for Leo, whose work would it be?” The answer this book uses: made by a tool. Chapter 8 returns to this idea.

Specific encouragement for the week

- **What you can learn this week:** to open one free app, build three blocks yourself, and then keep your hands off the child's screen while you ask "Point to the block that makes that happen."
- **What your child can try today:** predict how far the cat will walk, then tap the flag and find out.

For the student

A **program** is a set of instructions a computer follows. In block coding, each instruction is a block. You snap blocks together in order, and the computer does them one by one. You don't have to spell anything. You just have to think clearly.

Most programs start with a **trigger block**. In ScratchJr, it's often "Start on Green Flag." Nothing happens until someone taps the flag. Then the blocks run, in order.

Here's an example: **Start on Green Flag** → **Move Right 3** → **Hop**. Tap the flag. The cat walks three squares and hops. Change the 3 to a 5, and it walks five. You're the boss, and the cat is a very patient helper who does exactly what you say. Exactly.

Before you tap the flag, make a guess. What will the cat do? Then tap it and check. If the cat does something you didn't expect, you've found a bug, and you know what to do from Chapter 4.

Try this today. Make a character walk to one side of the screen. Guess how many blocks it will need before you build it. Then run it. Were you right?

Explain it back. Point to the block that starts your program. Tell someone how you know it's that one.

Challenge. Make two characters move at the same time when you tap the flag. Then make one of them say something when the other one bumps into it.

If it isn't clicking

Diagnostic 1: The child only plays with costumes, colors, and the paint editor. The Hour turns into drawing. This usually means the art tools are new and exciting, which is fine for one Hour.

Next move: let one Hour be a drawing Hour. Next time, start with a paper plan (one character, one thing it should do), and the paint editor opens only after the character moves.

Diagnostic 2: Reading gets in the way. A child who is eight or nine finds Scratch's word blocks hard work, and the frustration is about reading, not programming. This usually means the tool doesn't fit yet.

Next move: switch to ScratchJr or CodeAI's pre-reader course without fuss. Those are real programming tools. Place by skill, not birthday. You can also read the blocks aloud for a child while they make every decision; record it honestly ("blocks read aloud by parent").

Diagnostic 3: Screen fights. Arguments about turns, about stopping, or about “just one more minute.” This usually means the end of the screen time isn’t predictable yet.

Next move: a visible timer, set at the start. Save and Tell ends every screen block, and the screen goes off before the “tell.” Driver and navigator for siblings, swapping on the timer. If the tablet keeps winning, do two unplugged Hours, then try a shorter screen block.

When to slow down, go ahead, or step back. Slow down to three-block programs if bugs bring tears. Go ahead when the child builds from a plan, predicts, and fixes a bug. Step back to cards if the screen keeps winning; the same ideas live on the tape grid.

The rescue order. Ask, wait, hint, then a short demo on *your own* device or a project labeled as yours, then hand it back. The child’s program stays the child’s.

Tools, including AI

Free tools. ScratchJr (tablet, ages 5–7); Scratch (offline app or parent-confirmed account, ages 8–10); CodeAI CS Fundamentals (structured courses A–F and pre-reader Express). All need only one shared family device in a common room.

Optional hardware. One button floor robot (5–7) or a micro:bit with MakeCode (8–10). Borrow first. No prices here; check current prices and age labels.

AI. AI helpers inside children’s coding tools stay off, or you run them yourself. That includes AI tutors in courses and AI buttons in editors like MakeCode. The child builds and debugs. A game, story, or code made by an AI tool is “made by a tool,” never the child’s work. The full guidance is in the AI box in **The Computing Hour**.

What “done enough” looks like

Look for these before moving on.

5–7:

- ☐ The child builds a script with a start trigger and three or more blocks.
- ☐ The child predicts what it will do before tapping the flag.
- ☐ The child finds and fixes one bug.
- ☐ A project is saved with a name you can find again.

8–10:

- ☐ The child tours a starter project before changing it, and names one event.
- ☐ The child remixes one thing with a written prediction.
- ☐ The child builds a small project from a paper plan, with at least one Bug Log line.
- ☐ The project is saved with a date-first name and found again the next day.

Checks for both bands:

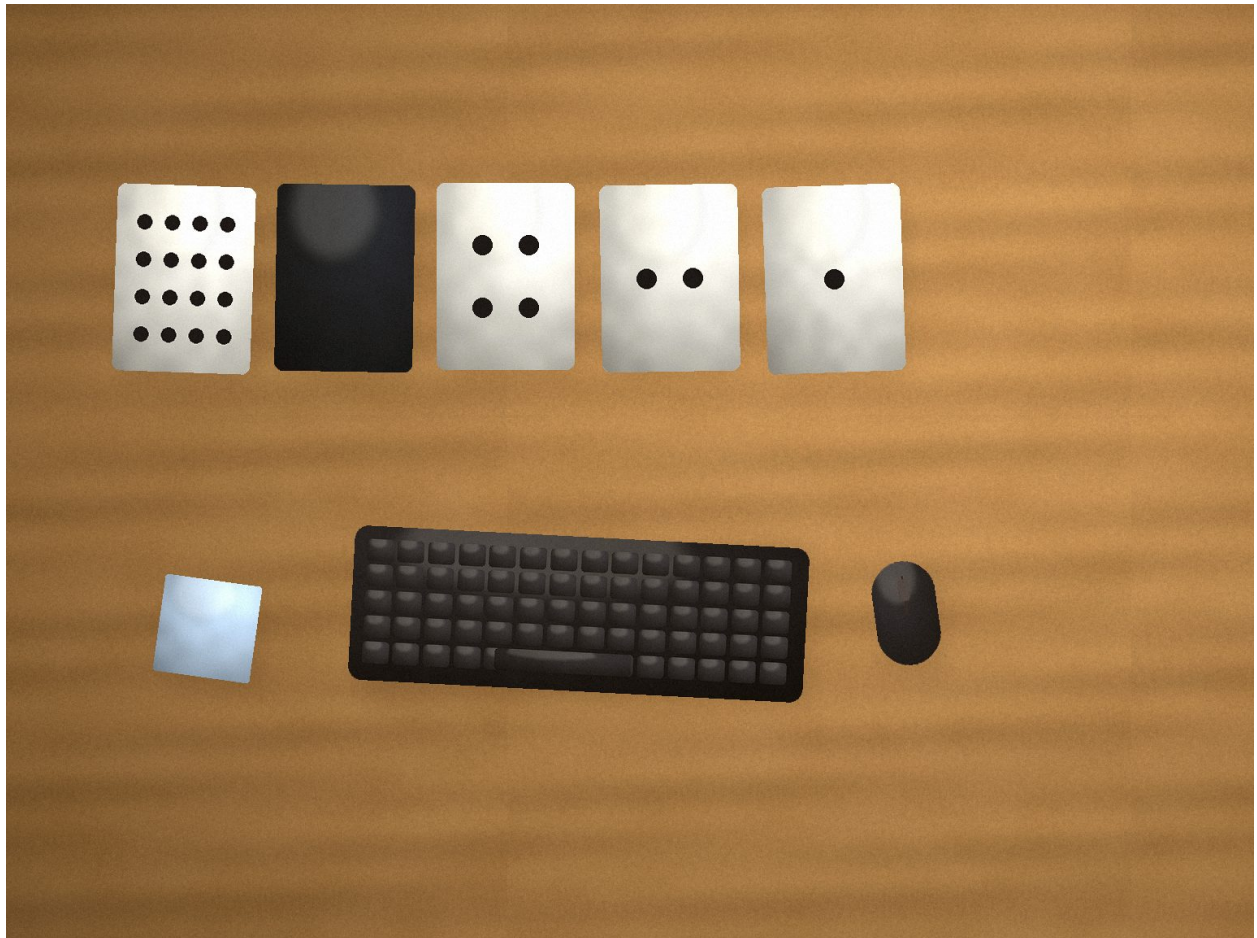
- ☐ **AS-01 Exit Crumb Trio** at least twice.

- ☐ **AS-02 Show Me Two Ways:** the same result built two ways (for example, five Move blocks, or one Repeat block).
- ☐ **AS-05 Portfolio Pick:** the child chooses one project or photo of a script for the folder and says what it shows they can do.

Placement by skill, not birthday. “Course C” on a website is that program’s scope, not your child’s grade and not a transcript line. A six-year-old who builds a three-block ScratchJr script and fixes one bug is done enough. A ten-year-old new to screens might start with ScratchJr for a week or two, and that’s the right start.

The habit going forward. Most weeks from here, once the child is ready, the Computing Hour can include a real program: a short ScratchJr script, a Scratch remix, or a CodeAI lesson split across two days. Save, name, screen off, tell. Saved projects and photos of scripts go in the folder under **Computer Science**; Chapter 9 shows how.

Chapter 6



Computers Up Close: Parts, Hands, Keys, and Files

Why this matters

*Picture a family on a Thursday afternoon. The eight-year-old is **input**: she reads instructions off a card and passes them along. The five-year-old is **memory**, holding a sticky-note pad and writing down anything he's told to keep. Dad is the **processor**: he does the working-out. The ten-year-old*

is **output**, the screen, and he holds up a sign with the answer. Someone says “add three and four.” The note travels from input to processor, the processor works it out, memory writes “7” on a sticky note just in case, and output holds up a big 7. The processor gets muddled and has to ask memory twice. Everyone laughs. That’s a fair picture of a real computer, minus the laughing.

This chapter opens the box. Children use computers all the time, and “it just knows” is the usual theory of what happens inside. A better theory lets a child reason about why a computer does what it does, including when it does something odd.

Here is the whole idea in one line: **a computer takes something in, works on it, gives something back, and remembers.** Input, process, output, storage. Everything else, from a tablet to a supermarket scanner to an automatic door, is those four, very fast.

The chapter also covers hands and habits: the mouse and touch, the first steps of typing, and the unglamorous habit that saves more tears than any other, saving a file with a name you can find again.

One plain distinction will help you keep records honest later. Understanding input, output, and binary numbers is computer science. Typing and file-saving are useful skills that sit beside it. Both are worth teaching. Chapter 9 shows how to record them separately if you choose.

This week, specifically. You can learn to name the four parts of any device in the house, and to make the number 13 with five dot cards. Your child can try playing “Be the Computer” at the kitchen table today, and finding the bumps on the F and J keys.

For the parent: understand it yourself

The four parts, in plain words.

- **Input** is what goes in: a tap, a key press, a voice, a camera picture, a button, a barcode under a scanner.
- **Process** is the working-out. A part inside called the *processor* follows the program’s instructions, very fast.
- **Output** is what comes out: a picture on the screen, a sound, a light, a printed page, a door sliding open.
- **Storage** is remembering: keeping information for later, even after the machine is switched off. That’s where saved files live.

Hardware and software. **Hardware** is the parts you can touch: the screen, the keys, the board inside. **Software** is the instructions that tell the hardware what to do: the apps and programs. A simple test for children: “Can you drop it on your foot?” If yes, hardware. If no, software. (A tablet is hardware. ScratchJr is software. Please don’t test this on anyone’s foot.)

Data and binary. **Data** is information a computer can store: numbers, words, pictures, sounds. Inside, computers store all of it using just two states, like a switch that’s on or off. Each on-or-off is called a *bit*, and patterns of bits can stand for a file, a message, a web page, or even a program.⁵³ Writing numbers with only two states is called **binary**.

Here's the trick that makes binary make sense to a child. Take five cards. Put 1 dot on the first, 2 on the next, then 4, 8, and 16. Lay them in a row, biggest on the left: 16, 8, 4, 2, 1. Each card is either face up (counts) or face down (doesn't). To make 13, turn up the 8, the 4, and the 1: $8 + 4 + 1 = 13$. Written in binary, with 1 for up and 0 for down, that's 01101. Every number from 0 to 31 can be made with those five cards, each in exactly one way. This is a CS Unplugged activity, marked for ages 5 to 10;³⁴ at 5–7, children count dots and turn cards; at 8–10, they write the 1s and 0s.

Pixels. A picture on a screen is made of tiny squares of color called **pixels**. The computer stores a picture as a list of numbers, one for each square. Children can see this with graph paper: “2 white, 3 black, 2 white” on a row becomes a little drawing. That's CS Unplugged's image representation idea, also marked 5 to 10.³⁴

Catching mistakes (8–10). Computers have clever ways to notice when stored or sent data has been changed by accident. One simple version is a card trick: lay out a 5-by-5 grid of two-colored cards, then secretly add one extra row and column so that every row and column has an even number of black cards. When someone flips one card, the row and column with an odd count cross at the flipped card. The extra cards are called *parity* cards, and computers use similar (more sophisticated) methods for almost all stored and sent data.⁵³ CS Unplugged's error-detection topic has lessons for ages 5 to 7 as well as 8 to 10, and this card trick is its 8-to-10 lesson;⁵³ in this book it's an 8–10 card (HW-06), because the counting is easier then.

One everyday picture. A microwave. You press buttons (input). A small computer inside works out how long to run (process). It hums, heats, and beeps (output). It remembers the time of day even when it isn't cooking (storage). Ask your child the chapter's question about the microwave at lunchtime, and you've had a computing lesson.

One precise picture. Your child taps the green flag in ScratchJr.

- Input: a finger touches the screen at the flag.
- Process: the processor runs the blocks, one by one.
- Output: the cat moves on the screen.
- Storage: when the child saves, the project is kept on the tablet; ScratchJr stores projects in a database on the device itself.^{41,40}

What the standards expect. Second graders are expected to explain how basic hardware parts “work together to perform input and output.” Fourth graders are expected to “apply a basic troubleshooting process to identify and fix common hardware and software issues.”^{25,26} At home, that looks like restarting an app or noticing the volume is off. Those are the targets this chapter climbs toward.

Hands: touch, mouse, and keys. Young children can learn a mouse, and dragging is easier than clicking, moving, and clicking again.⁴⁷ So at 5–7, touch and drag-and-drop come first, with big targets and patience; a few minutes of mouse practice is fine but not urgent. Typing, in the most widely used U.S. school standards, starts at grade 3 “with guidance and support from adults”; by grade 4 the target is typing “a minimum of one page in a single sitting,” and by grade 5, two pages.⁵⁴ That's a public-school map, not homeschool law, and this book treats it as a gentle ceiling, not a target. Here's the plan:

- **5–7:** find letters, the space bar, Enter, and Backspace; type your first name. No home-row drills.
- **8–10:** home-row typing in five-minute bursts, accuracy first, a few times a week. Handwriting stays.

Files: names, saving, finding. A **file** is one saved piece of work: a project, a picture, a story. A **folder** holds files, like an envelope holds papers. The habits that matter are small: save before you stop; give the file a name you'll recognize; know where it went; and keep important work in two places. This book uses date-first names, like **2026-10-14 Mia cat walk**, because they sort themselves into order and a stranger can read them.

Wrong answers you should be able to hear.

1. **“The computer is the screen.”** This usually means the child sees the output and nothing else. Try: “The screen shows us what the computer did. Where does the working-out happen?”
2. **“The computer just knows.”** This usually means the inside is a mystery. Try: “It knows what it’s been told and what it’s stored. What went in? What did it do? What came out?”
3. **“Software is the stuff inside.”** A very reasonable mix-up, since the processor is inside too. Try: “Can you drop it on your foot?”
4. **“I saved it!” but it’s gone.** It was saved as “Untitled,” or in an unknown place, or not saved at all. This usually means saving hasn’t become a habit with a name and a place attached. Try: “Let’s save, close, and find it again, right now, together.”
5. **One-finger typing, eyes on the keys, at six.** Not a wrong answer. Normal. Home-row typing waits until 8–10.

Your 5-minute parent warm-up. Pick one device in your house and name its input, process, output, and storage out loud. Then make five dot cards (1, 2, 4, 8, 16) and make 13, then 6, then 31. Finally, save a file on the family device with a date-first name, close it, and find it again. If you can do those three things, you’re ready. Practice tomorrow’s opening once: “What went in, what did the computer do, and what came out?”

How to teach it this week

This chapter covers three kinds of Hour: a *machine* Hour (how computers work), a *hands* Hour (mouse, touch, keys), and a *files* Hour. Mix them across the week. Each uses the Computing Hour’s five steps.

A machine Hour.

- **Unplug First (3–8 min).** HW-01 Input–Output Hunt: walk around the house and name what goes in and out of the doorbell, the microwave, the lamp.
- **Predict (2–5 min).** “If I say ‘add five and two,’ what will Output hold up?” On the Expected/Actual card.
- **Build and Run (8–15 min).** HW-02 Be the Computer, or HW-03 Binary Cards.
- **Bug Hunt (3–8 min).** If the answer comes out wrong, find the first part that went wrong: did Input pass the right note? Did Memory write it down?

- **Save and Tell (2–5 min).** A photo of the sticky notes or dot cards. One sentence: “I was the memory.”

A hands Hour. Paper keyboard first; predict which finger presses J; a few minutes of real typing or dragging; a typo is a bug, and Backspace fixes it; save, screen off.

A files Hour. The Folder Box first; predict “Where will we find it tomorrow?”; save a real project with a date-first name; close the app and hunt for it; say the file’s name out loud.

Exact wording you can use.

- Opening: “What went in, what did the computer do, and what came out?”
- For storage: “Where did it keep it?”
- For events: “What made it start?”
- For parts: “Is that hardware or software? Could you drop it on your foot?”
- For binary: “Which cards are up? Which are down? Count the dots.”
- For typing: “Slow and right beats fast and wrong.”
- For files: “Save it, name it, close it — now find it.”

The first try-it. HW-02 Be the Computer, below. It’s physical, funny, and it gives everyone in the family a part to play.

How to fade help across the week.

- **Day 1.** You run Be the Computer and choose who plays what.
- **Day 2.** The child explains the four parts using a household device, with you asking the opening question.
- **Day 3.** Binary cards: you make 5; the child makes 6, then a number you call out.
- **Days 4–5.** The child saves, closes, and finds a file on their own, and (8–10) makes 13 in binary without help.

When to stop talking. Binary cards need long, quiet counting. Typing needs one finger reminder per burst, not ten. A lost file deserves a minute of searching before you help; it’s a small bug hunt.

Age-band moves.

- **5–7.** Input-output hunts, Be the Computer with simple jobs, dot counting (turn up the 1 and the 2: how many dots?), drag-and-drop on a tablet, finding the letters of your name, the Folder Box. Oral answers; you write. Typing is letter-finding only.
- **8–10.** Binary with all five cards and written 1s and 0s, pixel pictures from a code, the parity trick, home-row typing in five-minute bursts, a couple of keyboard shortcuts, and the full save-close-find routine. A Bug Log line when something on the device goes wrong.

A week at a glance (a suggestion).

Day	Unplug First	Build and Run	Card codes
Mon	Input–Output Hunt	Be the Computer	HW-01, HW-02
Tue	Dot counting	Binary Cards	HW-03

Day	Unplug First	Build and Run	Card codes
Wed	Folder Box	Name It So Grandma Can Find It; Save, Close, Find	FILE-01, FILE-02, FILE-03
Thu	Paper keyboard	Find Your Name Keys (5–7) / Five-Minute Typing Burst (8–10)	KEYS-02 / KEYS-03, KEYS-04
Fri	Pixel Picture Code or Parity Card Trick	Short check; portfolio pick	HW-04 / HW-06, AS-04, AS-05
Out	—	Barcode Hunt	OUT-02

Siblings. Be the Computer is built for a crowd. For typing, each child gets their own short burst while the other is the “accuracy checker” for one line, then they swap. For files, the older child can show the younger where the Folder Box is, but each child saves and names their own work.

Practice that actually builds learning

Blocked practice here means several goes at one new move: five binary numbers in a row, five saves in a row. Mixed practice means a set of questions where the child decides which idea applies: is this an input or an output? Is this hardware or software? Does this file name tell you what’s inside?

In-home — HW-02 Be the Computer

- **Time:** about 15 minutes.
- **Band:** both.
- **Materials:** four signs on paper (INPUT, PROCESSOR, MEMORY, OUTPUT), sticky notes, a pencil, a few simple instruction cards (“add 3 and 4,” “remember the word *cat*,” “say what memory is holding”).
- **Safety and privacy:** nothing to watch for; use first names on any signs.
- **The fun:** the whole family becomes one machine, and the machine is a bit silly.
- **The skill:** input, process, output, and storage working together.
- **What to save:** a photo of the sticky notes, and the answer to “Who was memory?”

How it goes. Each person takes a sign. Input reads an instruction card and passes it to the processor. The processor does the work. If the instruction says to remember something, the processor tells memory, who writes it on a sticky note. Output says or shows the result. Try a few cards. Then add a trick card: “say what memory is holding.” The processor has to ask memory, and output has to wait. That small delay is real. In a real computer, fetching from memory takes time too.

When something goes wrong (it will), ask Chapter 4’s question: “Where did it first go wrong?” That’s debugging a computer, with people as the parts.

At 5–7, keep instructions to one step and give the youngest the memory job; writing or drawing on sticky notes is satisfying. At 8–10, add a card with two steps, or a loop: “repeat 3 times: add 1 to the number in memory.”

In-home — HW-03 Binary Cards

- **Time:** about 15 minutes.
- **Band:** 8–10 for binary; 5–7 for dot counting only.
- **Materials:** five cards with 1, 2, 4, 8, and 16 dots, blank on the back.
- **Safety and privacy:** nothing to watch for.
- **The fun:** making any number from 0 to 31 with just five cards, like a magic trick.
- **The skill:** representing numbers with two states (up and down, 1 and 0).
- **What to save:** a photo of the cards making 13, and (8–10) the binary written underneath: 01101.

How it goes. Lay the cards in a row with the biggest on the left: 16, 8, 4, 2, 1. Ask what the child notices about the dots. (Each card has twice as many as the one to its right.) Turn them all face down. Turn up the 1: one dot. Turn up the 2 as well: three dots. Ask for a prediction before each flip: “How many will we have if we turn up the 4?”

Then the challenge: “Make 13.” Let them try. Most children start by turning up the 16, find it’s too many, and turn it back down. That’s debugging. When they get $8 + 4 + 1$, write it in 1s and 0s: up, up, down, up becomes 1101, and with the 16 face down in front, 01101.

At 5–7, stop at counting: turn up two or three cards and count all the dots together. That’s plenty. At 8–10, try your child’s age, then the date, then the biggest number possible (31, all up), then ask, “What would the next card need?” (32 dots.)

In-home — HW-04 Pixel Picture Code

- **Time:** about 15 minutes.
- **Band:** both.
- **Materials:** graph paper (or a hand-drawn grid of 8×8 squares), a pencil, a dark crayon.
- **Safety and privacy:** nothing to watch for.
- **The fun:** a secret code that turns into a picture.
- **The skill:** a picture stored as numbers, one row at a time.
- **What to save:** the decoded picture with its code written beside it.

How it goes. Write a short code for each row, starting with white: “3, 2, 3” means 3 white squares, 2 black, 3 white. Give the child eight rows of code and let them color the squares. A simple shape appears: a heart, a house, a smiling face. Then swap: the child draws a picture on a blank grid and writes its code, and you decode it. At 8–10, ask: “What happens if one number in the code is wrong?” (The picture gets a bug.) At 5–7, use a 5×5 grid and say each row’s code out loud.

In-home — HW-06 Parity Card Trick

- **Time:** about 15 minutes.

- **Band:** 8–10.
- **Materials:** 36 cards that are black on one side and white on the other, or 36 coins (heads and tails).⁵³
- **Safety and privacy:** coins are a choking risk near toddlers.
- **The fun:** a magic trick that turns out to be a real computer technique.
- **The skill:** adding extra data to catch a mistake.
- **What to save:** the child’s explanation of how the trick works, in one or two sentences.

How it goes. Ask the child to lay out a 5-by-5 grid of cards, any colors, as random as they like. Then say you’ll make it “harder” by adding a row and a column. Secretly, choose each extra card so that every row and every column has an even number of black cards. Turn your back. The child flips one card. Turn around and look for the row and the column that now have an odd number of black cards; the flipped card is where they cross. “This one.”

Do it twice, then crack the trick together by counting black cards in each row. Let them be the magician. Then tell them the true part: computers use ideas like this to notice when data gets changed by accident.⁵³

In-home — HW-05 Old Device Look (and HW-01 Input–Output Hunt)

- **Time:** about 10 minutes.
- **Band:** both.
- **Materials:** an old keyboard or mouse that’s unplugged and no longer needed.
- **Safety and privacy:** an adult handles the device and any screws. Nobody opens batteries, power supplies, or anything with a screen. Wash hands afterward. If in doubt, look without opening.
- **The fun:** seeing the inside of something you use every day.
- **The skill:** naming hardware parts; seeing that a key is just a switch.
- **What to save:** three parts named, drawn, or photographed.

How it goes. Look at the underside first. If it’s a simple keyboard with screws, an adult opens it. Press a key and watch what moves underneath. “When you press this, the computer gets a signal: input.” Name three parts. Enough.

HW-01 Input–Output Hunt is the quick everyday version: walk around the house and name what goes in and what comes out of the doorbell (button in, chime out), the lamp (switch in, light out), the tablet (tap in, picture out). Two pairs make a good crumb.

In-home — KEYS-02 Find Your Name Keys (5–7) and KEYS-01 Drag-and-Drop Garden

- **Time:** about 5–10 minutes.
- **Band:** 5–7.
- **Materials:** a paper keyboard (printed or drawn), the real family device, and any simple drag-and-drop activity (ScratchJr works).

- **Safety and privacy:** first name only; the child types into a blank document or app you've opened, never into a website box.
- **The fun:** seeing your own name appear.
- **The skill:** finding letters, the space bar, and Backspace; dragging with control.
- **What to save:** the typed name (a photo of the screen is fine), and "what was hard?"

How it goes. Point to each letter of your name on the paper keyboard, then type it on the real one, fixing mistakes with Backspace (a bug fix; celebrate it). For drag-and-drop, any big-target activity will do. Five minutes. No speed goal.

In-home — KEYS-03 Paper Home Row and KEYS-04 Five-Minute Typing Burst (8–10)

- **Time:** five minutes each.
- **Band:** 8–10.
- **Materials:** a printed keyboard; the family device with a free typing activity such as BBC Bitesize's Dance Mat Typing.⁵⁵
- **Safety and privacy:** a free activity that needs no account; the device lives in the shared room.
- **The fun:** a game, a dancing character, and a level finished.
- **The skill:** home-row touch typing, accuracy first.
- **What to save:** one level completed and a note of accuracy ("only two mistakes in stage 2").

How it goes. First, on paper: fingers rest on A, S, D, F and J, K, L, and the semicolon. Find the small bumps on F and J; they let fingers find home without looking. Tap and say each letter. Then, on the device, five minutes of a typing activity. Dance Mat Typing has four levels, each in three stages, and level 1 starts with the home row keys. Its own advice is the right advice: "use the correct fingers," and "there's no need to rush!"⁵⁵

Stop at five minutes, even if it's going well. Three or four short bursts a week build more than one long session. Accuracy comes first; speed arrives on its own later. If your child is keen, you might keep a monthly note of accuracy for the record. Typing is a useful skill, not computer science; Chapter 9 shows how to record it as **Keyboarding** if it became a real, separate strand.

KEYS-05 Shortcut Treasure (8–10) adds a few keyboard shortcuts: save, undo, copy, and paste (on many computers, Ctrl or Command with S, Z, C, and V). Undo is the one that saves the day; it's a debugging tool.

In-home — FILE-01 Folder Box (with FILE-02, FILE-03, and FILE-04)

- **Time:** about 10 minutes for the box; five minutes for each screen card.
- **Band:** both (FILE-03 and FILE-04 at 8–10).
- **Materials:** a shoebox or crate, a few envelopes or paper folders, labels, and some drawings; then the family device.
- **Safety and privacy:** first names in file names; nothing personal like an address.
- **The fun:** a filing system you can hold, then the same idea on the screen.

- **The skill:** naming, saving, and finding.
- **What to save:** a file name the child chose, written down.

How it goes. *FILE-01 Folder Box (both).* Make a few folders and label them: “Drawings,” “Robot plans,” “Stories.” Put this week’s papers into the right folders. Close the box. Tomorrow, ask the child to find one particular paper by name. A folder on a computer works just like this.

FILE-02 Name It So Grandma Can Find It (both). When saving a project, use a date-first name: the date, then the child’s first name, then what it is: **2026-10-14 Sam maze game**. Ask: “Would Grandma know what this is, just from the name?” At 5–7, you type it while the child tells you what to call it.

FILE-03 Save, Close, Find (8–10). Save the project. Close the app completely. Open it again and find the project. If it’s not where you expected, that’s a bug hunt: where did it go?

FILE-04 Two Places Rule (8–10, parent-run). Important work lives in two places. You copy the project to a second place, such as a family drive or a USB stick kept in a drawer. Choose places you control, not a shared computer that AI helpers use (see the AI box in **The Computing Hour**). This matters more than it sounds: ScratchJr keeps its projects only on the tablet, so if the app is deleted, the projects go with it.^{41,40} Ask your child: “Where are our two places?”

Robot cross-links: BOT-02 and BOT-06

BOT-02 Cardboard Robot Design (both, about 20 minutes): the child designs a robot from a box and labels its inputs (eyes that see, ears that hear, a button) and outputs (lights, wheels, a voice). It’s an unplugged way to practice this chapter’s four words. **BOT-06 Shake-a-Die** (8–10, needs a micro:bit) does the same thing for real: shake the board (input), it picks a number (process), and the number lights up (output). MakeCode has a dice tutorial;⁵² Chapter 5 covers setting it up.

Out-of-home — OUT-02 Barcode Hunt

- **Time:** part of a grocery trip.
- **Band:** both.
- **Materials:** a folded index card and pencil; the shopping list.
- **Safety and privacy:** stay together in the store; watch the scanner, but let the cashier work; no photos of people.
- **The fun:** spotting the stripes everywhere and seeing the machine read them.
- **The skill:** input and output in the wider world.
- **What to save:** one input-and-output pair: “The scanner read the stripes (input); the screen showed the price (output).”

How it goes. Before you go, find a barcode on something in your kitchen and look at it closely. At the store, hunt for barcodes on everything in the cart. At the checkout, watch the scanner. Ask the opening question: “What went in, what did the computer do, and what came out?” The stripes go in; the computer looks up what the item is; the name and price come out on the screen, often with a beep.

At 8–10, add one more question: “What do you think happens if the barcode is scratched or dirty?” CS Unplugged points out that scanning a barcode is a kind of sending information, and that dirt or scratches can change it, which is why codes include extra checking information.⁵³ A barcode that won’t scan is a bug in the world. If the cashier has to type the number in instead, your child has just seen a different input do the same job.

No-store alternative. Five packages from the pantry: one child reads out the numbers under the stripes (scanner), another looks them up on a “price list” you wrote (screen).

Talk box (this chapter)

Opening: “What went in, what did the computer do, and what came out?”

Follow-ups (pick three to five):

1. **Prediction:** “What will come out if we put this in?”
2. **Evidence:** “What did you see or hear that tells you that?”
3. **Location:** “Where did it keep it? Point to the part that remembers.”
4. **Change:** “What made it start? What would happen if we changed the input?”
5. **Revise:** “Is that hardware or software? You can change your mind.”

How to wait. Count to five in your head after you ask. With binary cards, give much longer; counting dots takes time, and lips moving silently means it’s working.

What a stuck silence usually means. The words are new, so the child may know the idea but not trust the label. Or the device is too complicated to start with (a tablet does a hundred things at once). Or they’re afraid of getting “input” and “output” backwards.

The move. Go simpler. Start with a doorbell or a lamp: one input, one output. Offer a choice of two: “Is the light coming *in* or coming *out*?” Or let them point instead of naming.

Blocked and mixed practice

Blocked. Five binary numbers in a row. Five saves with date-first names. Five minutes of the same typing stage.

Mixed. A pile of picture cards (keyboard, screen, speaker, microphone, a game app, a camera, a printer, a drawing app, a tablet) for the child to sort two ways: input or output (some are both), and hardware or software. Ask “What makes you say that?” for the tricky ones. A touchscreen is both input and output, which children love discovering.

Quick retrieval. Next week, start with one binary number and one question: “Where does a saved file live?”

One incorrect example to diagnose

Show the child this list of files on a pretend family tablet:

Untitled Untitled 2 project cat 2026-10-14 Mia cat walk

Ask: “Which one would you find tomorrow? Which ones would be lost?” Then: “What’s the bug in the first four names?” (They don’t say whose it is, when it was made, or what it is.) “How would you rename ‘cat’?” At 8–10, add: “If this tablet broke tonight, how many copies of Mia’s cat walk would be left?” (One, unless the Two Places Rule was used.)

Specific encouragement for the week

- **What you can learn this week:** to name the input, process, output, and storage of a microwave or a tablet, and to make 13 with five dot cards.
- **What your child can try today:** be one part of the family computer, then find the bumps on F and J.

For the student

A computer only does four things:

1. It takes something in. That’s **input**: a tap, a key, a voice, a button.
2. It works on it. That’s **processing**: following its instructions, very fast.
3. It gives something back. That’s **output**: a picture, a sound, a light.
4. It remembers. That’s **storage**: keeping things for later.

Everything a computer does is those four things, over and over, very fast.

You tap the green flag (input). The tablet follows your blocks (processing). The cat walks (output). You save it for tomorrow (storage).

Computers store everything, even pictures and music, as patterns of just two things, like a light switch that’s on or off. You can try this with five dot cards: 1, 2, 4, 8, and 16. Turn some up and some down, and you can make any number up to 31. To make 13, turn up the 8, the 4, and the 1.

Try this today. Pick something in your house, like a doorbell, a lamp, or a microwave. What goes in? What comes out? Does it remember anything?

Explain it back. Tell someone the four things a computer does, using a device in your house as the example.

Challenge. Make your age with the dot cards. Then make the biggest number you can. Then save a file with a name that tells Grandma exactly what it is.

If it isn’t clicking

Diagnostic 1: Binary feels like nonsense. The child can’t see why $8 + 4 + 1$ has anything to do with 01101. This usually means the written 1s and 0s arrived before the cards made sense.

Next move: stay with dots and “card up, card down” for longer, even at nine or ten. Make numbers by counting dots only. Write 1s and 0s only when the child asks how to write it down. At 5–7, binary doesn’t need to happen at all; dot counting is the whole goal.

Diagnostic 2: Mouse frustration. Clicks miss, drags drop things in the wrong place, and tempers rise. This usually means the targets are too small or the mouse arrived too early.

Next move: go back to touch and drag-and-drop on a tablet. Big targets. Short sessions. A few minutes of mouse practice can wait for another month, and that's fine.

Diagnostic 3: Files keep getting lost. “Where’s my game?” every Hour. This usually means there’s no single naming habit and no single place.

Next move: one rule, said every time: date first, first name, what it is. One place. Save, close, find, together, at the end of every screen Hour for two weeks. Then the Two Places Rule.

When to slow down, go ahead, or step back. Slow down if typing brings tears: shorter bursts, or wait a few months. Go ahead when the child can name the four parts in any device and save-close-find alone; Chapter 7 is next. Step back to unplugged (Be the Computer, the Folder Box) whenever a screen idea feels slippery; the cards and boxes are easier to see.

The rescue order. Ask, wait, hint, then a short demo on *your own* cards or device, then hand it back. The child’s work stays the child’s.

Tools, including AI

What you need. Paper and sticky notes for Be the Computer; five dot cards; graph paper; 36 two-colored cards or coins for the parity trick; an old keyboard (adult-handled); a paper keyboard; a shoebox and envelopes; the family’s one shared device. A free typing activity such as Dance Mat Typing for 8–10.

AI. The child doesn’t use AI for any of this. Keep your child’s files and records in places you control, not on a shared computer that AI helpers use. The AI box in **The Computing Hour** has the details.

What “done enough” looks like

Look for these before moving on.

- ☐ The child names the input, output, and storage of one household device.
- ☐ The child can say whether a thing is hardware or software, with a reason.
- ☐ (5–7) The child counts the dots on two or three turned-up cards. (8–10) The child makes a number, such as 13, in binary and writes it.
- ☐ The child saves a file with a date-first name, closes it, and finds it again (5–7 with you typing the name).
- ☐ (5–7) The child types their first name. (8–10) The child does a five-minute accuracy-first typing burst.
- ☐ **AS-04 Teach It Back:** the child explains input, process, output, and storage to a sibling or a teddy; you write down one sentence.
- ☐ **AS-01 Exit Crumb Trio** at least twice: “I predicted... It did... Next I’d change...”

- **AS-05 Portfolio Pick:** the child chooses one thing (the binary photo, a pixel picture, a typed line) for the folder and says what it shows.

Placement by skill, not birthday. A six-year-old who plays memory in Be the Computer, counts dots, and finds their name on the keyboard is done enough. A ten-year-old new to all this may need a week of dot counting before binary. That's the right pace.

The habit going forward. From now on, every screen Hour ends the same way: save, name it date-first, close it, find it, screen off. Photos of dot cards and pixel pictures go in the folder under **Computer Science**; typing samples go under **Keyboarding** only if typing was its own strand. Chapter 9 shows how.

Chapter 7



The Internet: How It Works, and How to Be Safe and Kind

Why this matters

Picture a seven-year-old on a video call with Grandma. The picture freezes, then jumps ahead. “The internet is broken,” she says, and taps the screen harder. Grandma’s voice comes back a second later, a little out of step with her mouth. (This is an illustration, not a real family.)

Nothing is broken. Grandma's face and voice were cut into thousands of tiny pieces, sent through many machines, and put back together on your tablet. When a few pieces arrive late, the picture freezes. Once a child knows that, the freeze stops being magic and becomes a clue.

This chapter has two halves, and they belong together.

The first half is **how the internet works**, in kid terms. A message travels in pieces. Every device has an address. A box with blinking lights passes the pieces along. People made every page your child will ever see, and a child who knows that can ask the next question: who, and why?

The second half is **how to be safe and kind** there. That means a few calm routines your child can say out loud and use without you standing over them:

- Some things are private. Some are okay to share.
- Passwords are like toothbrushes. You don't share them, and a grown-up keeps the list.
- When something weird, scary, or pushy pops up: **Stop, tell, close.**
- Photos of people need permission.
- Words online land on a real person.

The standards writers agree this is age-reasonable. CSTA (the Computer Science Teachers Association) expects children in pre-kindergarten and kindergarten to "differentiate between public and private information," and second graders to "explain how online actions have real-world consequences."^{2,25} That's the whole spirit of this chapter in two lines.

The tone here is calm. No horror stories, no scary videos. Safety at this age is taught the way you teach crossing the street: a short routine, practiced often, until it is ordinary. These routines won't make any child perfectly safe. Nothing does. They give your child a plan and keep you in the loop.

One timing note. If your child already uses a tablet, a phone, or a family computer, please teach two cards from this chapter in your very first week: SAFE-02 Passwords Are Like Toothbrushes and SAFE-03 Stop, Tell, Close. They take five minutes each. Come back for the rest later.

This week you can learn how a message travels in pieces, well enough to explain it with paper strips. **Today your child can try** saying three words, Stop, tell, close, and acting them out with a paper "screen."

For the parent: understand it yourself

You don't need to be a network engineer. You do need a clear picture of what happens when you press send, and a short list of what your child should do when something online feels wrong.

The everyday picture

Imagine you want to mail a long letter to a cousin, but the post office only takes postcards. So you cut the letter into pieces. On each postcard you write your cousin's address, your return address, and a number: 1 of 12, 2 of 12, and so on. You drop them all in the mailbox.

The postcards travel by different sorting offices, and postcard 7 might arrive before postcard 3. Your cousin lays them out in number order. If postcard 9 never arrives, she writes back, “Please send 9 again.”

That is the internet, almost exactly.

The precise picture

The **internet** is a huge network of networks: millions of computers joined by cables and radio signals so they can pass information to each other.

When you send something (a photo, a message, a video call), your device cuts it into small pieces called **packets**. Each packet carries the address it’s going to, the address it came from, and its place in the line.

Every device on the internet has an address, a number that works like a house address (you’ll see it called an **IP address**). That’s how pieces find their way.

A **router** is a machine that passes packets along toward their address. Your home router is the box with the blinking lights, the first stop for everything your family sends. From there, packets hop from router to router until they reach the address on the label.

Packets may take different paths. The receiving computer puts them back in order. If some are missing, it asks for them again. That’s why a video freezes for a moment and then catches up.

Wi-Fi is the radio signal between your device and the router; past the router, most of the journey runs through cables. A grade 4 CSTA standard asks children to compare wired and wireless connections.²⁶ “The tablet talks to the box by radio; the box talks to the world by cable” is plenty.

Web pages are made by people

Every web page, app, and video was made by someone: a library, a company that earns money when you click, a person who wants you to believe something. An **ad** is a message someone paid to put in front of you. Some look like ads; some look like a game or a “you won!” box. At 8–10, your child can start asking two questions about any page: *Who made this?* and *Is this part trying to sell me something?* They’ll need your help to answer. The habit of asking is the skill.

Private and okay to share

Private information is anything that could help a stranger find, contact, or pretend to be your child: full name, home address, school name, phone number, passwords, photos of their face, their location, their birthday.

Okay to share (with a grown-up’s say-so) are things that don’t point to a real person: a favorite color, a made-up nickname in a game, a drawing of a dragon.

Children 5–7 can sort picture cards into those two piles. Children 8–10 can explain why a photo of your front door is private even without a name on it.

Passwords, and the difference between “who” and “may”

At this age, **the grown-up holds every password and every account**. The child never types a password into a new site. The Computing Hour sets this out once, along with the rest of Family safety basics; this chapter is where you teach it to the child.

The toothbrush picture works because children already know it: everyone has their own, and you don’t share it. For 8–10, there’s a lovely extra idea. A grade 4 CSTA standard asks children to “distinguish between authentication and authorization.”²⁶ In ordinary words: a password proves **who you are**. Permission is **what you’re allowed to do** once you’re in. Your house key proves you live there. It doesn’t mean you’re allowed to use the stove.

Pop-ups and tricks

A **pop-up** is a box that jumps onto the screen, often uninvited. Many are harmless. Some are tricks:

- “You won a prize! Click here.”
- “Your tablet has a virus! Tap now.”
- “Type your name to keep playing.”
- “Chat with new friends!”

The rule for children is the same for all of them, and it doesn’t require the child to judge which is which: **Stop** (hands off the screen), **tell** (get a grown-up), **close** (the grown-up closes it, or the child closes the lid if no one is near). No shame, ever, for a pop-up. The child who tells you is doing exactly the right thing.

Photos

Photos of people need the person’s permission, and photos of children need a parent’s decision. In this family, faces don’t go into apps. Record photos are cropped to the work: the card strip, the drawing, the screen of blocks.

COPPA in two sentences

There’s a United States law, called COPPA, that protects the online information of children under 13. Services aimed at children under 13 must get a parent’s consent before collecting a child’s personal information, and that information includes the child’s name, address, location, and **photos, videos, and voice recordings**.⁶

COPPA is a rule for companies; it doesn’t make any site safe or replace you. The updated rule took effect in June 2025, and companies have had to comply with most of it since April 2026.⁷

So the book’s rule is simple: no accounts for children under 13 unless you set them up and hold them, and most of this book needs none. ScratchJr needs no account. New Scratch accounts for anyone under 16 need a parent’s email confirmation before the child can share or comment.^{44,45} Chapter 5 covers that choice.

Kind words

A message typed in a hurry lands on a real person who can't see your face or hear your tone. "Would you say it to their face, in that voice?" is a test even a six-year-old can run.

Wrong answers you should be able to hear

"The internet is inside the tablet." / "The cloud is up in the sky." *What it usually means:* the child thinks the internet is a place or a thing in the device, not other computers far away. *What you say:* "The tablet is talking to other computers, a long way off. Let's be the pieces and carry a message."

"My password is..." (said to a friend, a sibling, or you, out loud in a store) *What it usually means:* the child thinks passwords are like a secret club word, fun to share. *What you say:* "That's a toothbrush. Only I keep the list. Let's change that one later, together."

The child typed their name into a pop-up, or tapped "You won!" *What it usually means:* the box was designed to be tapped. Adults tap them too. *What you say:* "Thank you for telling me. That's exactly right. Stop, tell, close. Let's close it together." Then close it, and check nothing was installed.

"It's on the website, so it's true." *What it usually means:* the child hasn't yet noticed that a person made the page. *What you say:* "Somebody made this page. Who do you think? Why did they make it?"

"It was just a joke. They can't see me anyway." *What it usually means:* the child hasn't connected the words on the screen to a real person's feelings. *What you say:* "Would you say it to their face, in that voice? Let's write it the kind way."

Five-minute parent warm-up

Before your first Hour this week:

1. Write a short sentence on a strip of paper, cut it into five pieces, and number them. Shuffle them, then put them back in order. That's tomorrow's NET-01.
2. Find your router and look at its lights. You don't need to know what each one means. Just know where it is.
3. Decide where the family password list lives. A paper list in a drawer you control is fine.
4. Say the talk-box opening aloud once: "Who made this, who can see it, and who do we ask?" Then count to five in your head. That pause is half the lesson.

How to teach it this week

The Computing Hour shape holds: Unplug First, Predict, Build and Run, Bug Hunt, Save and Tell. This week most Hours need no screen at all. The internet is easiest to understand with paper, envelopes, and people walking between rooms.

A worked Hour (illustration)

Unplug First (5 min). “Today we’re going to be the internet. First, no screens.” Write a short message together, six to ten words, on a strip of paper. At 5–7, “WE HAVE PANCAKES FOR LUNCH TODAY” is perfect. At 8–10, let them write it.

Predict (3 min). Cut the message into numbered pieces. “If we send these pieces to Dad in the bedroom, one at a time, by different paths, what will he need to do to read it?” Wait. “What if one piece gets lost?” Wait again.

Build and Run (10 min). Your child carries the pieces, one per trip, by different routes: through the kitchen, around the hall. Dad (or a sibling, or a teddy on a chair) lays them out and reads the message.

Bug Hunt (5 min). Quietly keep one piece in your pocket. When the message reads “WE HAVE FOR LUNCH TODAY,” ask: “What did you expect it to say, and what does it say instead? Which piece is missing? What should Dad do?” The answer you’re waiting for is “Ask for piece 3 again.” That’s exactly what computers do.

Save and Tell (3 min). Photograph the pieces laid out in order (pieces only, no faces). Your child says one sentence to someone: “The internet cuts messages into pieces and puts them back together.”

Exact wording you can say

- To open: “Today you’re the internet. Your job is to carry pieces of a message.”
- Before running: “What do you think will happen when they arrive in a different order?”
- At the bug: “Interesting. Something’s missing. Which piece? How do you know?”
- To fade help: “What would a computer do if a piece didn’t come?”
- To close: “Photo of the pieces, screen stays off. Now tell Grandpa one thing you learned.”

When to stop talking

When your child is laying pieces in order, say nothing. Look at the strips, not at them. If they ask for the answer, ask which number is missing. Counting is their job.

In-home — NET-01 Message in Pieces

- **Time:** 15 minutes.
- **Materials:** paper strips, a pencil, scissors (adult cuts at 5–7), two rooms.
- **Safety/privacy:** clear the path between rooms; walking, not running, in a busy house. The message is about lunch, not about anyone’s address.
- **The fun:** being the internet; finding the missing piece.
- **The skill:** messages travel in numbered pieces and are reassembled; lost pieces are asked for again.
- **What to save:** a photo of the strips in order, and the exit crumb.
- **5–7:** five or six big pieces, numbers drawn large. They carry; you help lay out.

- **8–10:** ten pieces, two “messengers” taking different paths, and one deliberately delayed piece. Ask them to explain why video calls freeze.

A week of this chapter

You don’t need every card. Here is one honest week:

- **Day 1:** NET-01 Message in Pieces. Start a Bug Jar token for the missing piece found (BUG-08).
- **Day 2:** SAFE-01 Private or Okay to Share, then SAFE-02 Passwords Are Like Toothbrushes.
- **Day 3:** SAFE-03 Stop, Tell, Close, then SAFE-05 Pop-Up Practice.
- **Day 4:** NET-02 Every Device Has an Address (5–7) or NET-05 Who Made This Page? (8–10, parent at the keyboard).
- **Day 5:** SAFE-06 Family Device Agreement, signed by everyone, and the “done enough” list.
- **Once this week, out of the house:** OUT-03 Library Computing Visit.

A second week can add NET-03, NET-04, SAFE-04, and SAFE-07.

Age-band moves

Ages 5–7. Keep it physical: pieces they can carry, cards they can sort, three words they can shout. Passwords are simply “a grown-up keeps them.” Packets are “pieces with numbers.” The router is “the box that passes the pieces along.” The screen stays off for almost every Hour.

Ages 8–10. Add “who made this page?” with you at the keyboard, ads, who you are versus what you may do, the string network, and the kind-words rewrite. They can write pop-up tricks for you to spot, and explain packets to a younger sibling, which is the best test of all.

Practice that actually builds learning

Blocked practice for a new move: three days of private-or-okay sorting with new cards each day, until it’s quick. **Mixed practice** once it’s known: a stack of paper pop-ups and picture cards shuffled together, where the child must decide each time whether to sort, close, or ask.

The library visit, or an errand where you notice a store’s Wi-Fi sign, is practice too. It doesn’t replace the predict-run-debug move. It gives it somewhere to live.

Talk box (this chapter)

Opening (exact): “Who made this, who can see it, and who do we ask?”

Use it on a web page, a pop-up, a photo, a drawing your child wants to send to a cousin, or a game on the library computer.

Follow-ups (pick three to five):

1. *Prediction*: “If we put this online, who do you think could see it?”
2. *Evidence*: “What do you see on the page that tells you who made it?”
3. *Location*: “Point to the part that’s trying to sell us something.”
4. *Change*: “Is that private or okay to share? What’s one thing we could change to make it okay?”
5. *Revise*: “You can change your answer. Let’s look again.”

How to wait. After you ask, count to five in your head. That’s a convention drawn from classroom wait-time studies, used here as pause craft.⁵ After your child answers, wait again; the second thought is often better. If the silence feels long, look at the screen or the card, not at your child.

What stuck silence usually means.

- The question was too big. Ask just one part: “Who made this?”
- There was no wait. Ask again and count.
- Your child thinks there’s a right answer they’ll be scolded for missing. Say, “There’s no wrong guess here. We’re detectives.”
- They’re worried about being in trouble for something they clicked earlier. Say, “Nobody’s in trouble. Telling me is always the right move.”
- The screen is more interesting than the question. Turn it face-down for a moment.

The stuck-silence move: offer a choice of two. “Do you think a library made this, or a toy company?” Or ask them to point.

In-home — NET-02 Every Device Has an Address

- **Time:** 10 minutes.
- **Materials:** envelopes, a pencil, small notes; a sticky label for each “device” (the tablet, the TV, the printer, a phone).
- **Safety/privacy:** use made-up addresses on the envelopes (“Kitchen Tablet, 1 Shelf Street”), never your real one.
- **The fun:** playing mail carrier between gadgets.
- **The skill:** every device has an address so messages can find it.
- **What to save:** one addressed envelope, photographed.
- **8–10:** add a “from” address so the reply can come back. “What if two devices had the same address?”

In-home — NET-03 Router Look

- **Time:** 5 minutes.
- **Materials:** your home router, wherever it lives.
- **Safety/privacy:** look, don’t touch. No unplugging, no buttons, no settings, no reading out the network password on the sticker.
- **The fun:** finding the “magic box” and watching its lights.
- **The skill:** a router passes pieces along; your home’s internet goes through it.
- **What to save:** a drawing of the box and its lights, with one guess about what a blinking light might mean.

- **8–10:** “Which lights blink faster when someone watches a video?” Test it, with you starting the video.

In-home — NET-04 String Network

- **Time:** 15 minutes.
- **Materials:** a ball of yarn, paper notes, clothespins or tape; three to five people or teddies.
- **Safety/privacy:** yarn at ankle height is a tripping hazard. Keep it at table height, and nobody runs.
- **The fun:** passing notes along a web of string, then “cutting” one string.
- **The skill:** messages can take more than one path; if one link breaks, another path can carry the message.
- **What to save:** a photo or drawing of the network.
- **5–7:** three people in a triangle; send a note two ways. **8–10:** five points; cut one string and find another route (pairs well with AS-02 Show Me Two Ways).

In-home — NET-05 Who Made This Page? (8–10)

- **Time:** 10 minutes.
- **Materials:** your device, your account, and a safe page you’ve chosen and checked first (a library, museum, or science-center page works well).
- **Safety/privacy:** you hold the device and do the typing and scrolling. Your child points and talks.
- **The fun:** being a detective on a real page.
- **The skill:** every page has a maker; some parts are ads.
- **What to save:** one clue written down (“At the bottom it says the museum made it”).
- **The move:** look at the top and bottom of the page for a name. Find the “About” link together. Spot one ad or “sponsored” box, if there is one. Ask the talk-box opening.

In-home — SAFE-01 Private or Okay to Share

- **Time:** 10 minutes.
- **Materials:** picture or word cards: a house, a school, a phone, a face photo, a birthday cake, a favorite color, a pet’s name, a drawing of a dragon.
- **Safety/privacy:** use pictures, not your child’s real details.
- **The fun:** sorting fast; arguing the tricky ones (a pet’s name is often okay, but some families use pets’ names in passwords).
- **The skill:** telling private from okay-to-share.
- **What to save:** a photo of the two piles, or one private thing named aloud.
- **5–7:** two piles, picture cards only.
- **8–10:** add “it depends” as a third pile, and ask why.

In-home — SAFE-02 Passwords Are Like Toothbrushes

- **Time:** 5 minutes. Teach this in week one if your child already uses devices.

- **Materials:** a toothbrush (clean!) and conversation.
- **Safety/privacy:** use a pretend password (“banana-rocket”) in the game, never a real one.
- **The fun:** the silliness of sharing a toothbrush.
- **The skill:** passwords are personal; a grown-up holds the list.
- **What to save:** the answer to “Who holds the passwords?” said aloud: “Mom and Dad.”
- **8–10:** add the house key: a password proves who you are; permission is what you’re allowed to do.

In-home — SAFE-03 Stop, Tell, Close

- **Time:** 5 minutes. Teach this in week one if your child already uses devices.
- **Materials:** a sheet of paper folded like a laptop, with a drawn “screen.”
- **Safety/privacy:** pretend screens only. No real scary content, ever.
- **The fun:** acting it out, big and dramatic; the grown-up plays the pop-up.
- **The skill:** a single routine for anything weird, scary, or pushy online.
- **What to save:** the three words, said aloud at Save and Tell.
- **The move:** hold up a drawn pop-up (“Type your name!”). Your child freezes their hands (*stop*), calls you (*tell*), and folds the paper laptop shut (*close*). Swap roles. Practice until it’s boring. Boring is the goal.

In-home — SAFE-04 Kind Words Rewrite (8–10)

- **Time:** 10 minutes.
- **Materials:** a few short, mildly unkind comments you write on cards about made-up things (“Your drawing is ugly,” “That game is for babies”).
- **Safety/privacy:** comments about pretend objects, never about real people your child knows.
- **The fun:** turning a frown into a fix.
- **The skill:** online words land on real people; you can say true things kindly.
- **What to save:** the kind version, in your child’s words.
- **The move:** read the card in a flat voice. Ask, “How would that feel to read?” Then, “How could you say what you mean, kindly?”

In-home — SAFE-05 Pop-Up Practice

- **Time:** 10 minutes.
- **Materials:** six to eight paper “pop-ups” you draw: “You won!”, “Your tablet is sick!”, “Type your name,” “Chat with new friends,” “Do you want to save your picture?”, “Time for a break.”
- **Safety/privacy:** paper only.
- **The fun:** spotting the tricks.
- **The skill:** noticing pushy pop-ups; knowing that Stop, tell, close always works.
- **What to save:** one trick spotted and named.
- **5–7:** every pop-up gets Stop, tell, close. **8–10:** sort into “ask a grown-up” and “a normal message,” then design a trick pop-up for you to catch.

In-home — SAFE-06 Family Device Agreement

- **Time:** 20 minutes.
- **Materials:** a big sheet of paper, markers, everyone in the family.
- **Safety/privacy:** first names only on the poster if it hangs where visitors see it.
- **The fun:** making the rules together, and seeing the grown-ups sign too.
- **The skill:** shared, written routines.
- **What to save:** a photo of the signed poster.
- **What goes on it** (your family's words, not these exactly): where devices live (a shared room); who holds passwords (the grown-ups); Stop, tell, close; first names only in projects; photos of people need permission; screens off about an hour before bed; and one kind-words promise. The Computing Hour explains the screen-time and bedtime guidance once, with sources.^{9,8}

In-home — SAFE-07 Photo Permission Rule

- **Time:** 10 minutes.
- **Materials:** a few printed or drawn pictures: a sibling's face, a sunset, a dog, a card program.
- **Safety/privacy:** use drawings or photos of objects, not new photos of your children.
- **The fun:** deciding together which ones are “ask first.”
- **The skill:** photos of people need permission; faces don't go into apps; the parent decides.
- **What to save:** the two words “Ask first,” said or written.
- **The move:** sort the pictures into “ask first” and “fine to keep.” Then crop a photo of a card program together, so only the work shows. That's the habit your portfolio will use (Chapter 9).

Out-of-home — OUT-03 Library Computing Visit

- **Time:** about 30 minutes.
- **Materials:** a library card in your name; a short list of questions.
- **Safety/privacy:** you stay beside your child at any public computer, and you do any logging in. No personal details typed. Log out and close the browser when you leave.
- **The fun:** visiting the “public computers” and finding a coding or internet picture book to borrow.
- **The skill:** noticing who runs a computer and what rules come with it.
- **What to save:** the title of the book chosen, and one rule your child spotted.
- **The move:** read the posted computer rules together and ask the talk-box opening. (The library made the rules; other people may see the screen; we ask the librarian.) Ask a librarian how the computers connect to the internet.
- **8–10:** compare the posted rules with your family agreement. What's the same?

One incorrect example to diagnose (illustration)

Picture an eight-year-old playing a free game on the family tablet. A box appears: “Great job! Type your name and town to join the winners' board!” She types her full name and the town, taps OK, and keeps playing. That evening she mentions she's “on the winners' board.”

What went wrong: the pop-up asked for private information, and she gave it. She didn't stop, didn't tell, and didn't close.

What to do, calmly: thank her for telling you, even late. That's the part to praise. Look together at what was typed and where, close the game, and check the app's settings yourself. Then run SAFE-05 with a paper copy of that exact box, and let her catch it. If a service aimed at children collected her details without your consent, the FTC's consumer page explains how to ask for deletion and how to report it.⁶

What she learns: telling is never trouble, even after a slip.

For the student

The internet is millions of computers passing notes to each other, very fast.

When you send a picture to Grandma, the computer cuts it into tiny pieces. Each piece gets a number and an address, like a postcard. The pieces travel through lots of machines and get put back together on Grandma's screen. If a piece goes missing, her computer says, "Please send that one again." That's why a video sometimes freezes and then catches up. It's not broken. It's waiting for a piece.

A tiny example. Write "I LIKE RED APPLES" on paper. Cut it into four pieces and number them. Mix them up. Now put them back in order using the numbers. You just did what the internet does all day.

People made every page and every game you'll ever see. Some are kind and true. Some are tricks that want your name or want you to tap. You don't have to sort them out alone. That's what grown-ups are for.

Here are your three words for anything weird, scary, or pushy on a screen:

Stop. Take your hands off. **Tell.** Get your grown-up. **Close.** Shut it, or let your grown-up shut it.

You will never be in trouble for telling.

Passwords are like toothbrushes. You don't share them. Your grown-up keeps the list.

Try it today. Ask your grown-up to draw a pretend pop-up. Do Stop, tell, close. Then swap: you draw one for them.

Explain it back. Tell someone in your family why a video call sometimes freezes.

Challenge. Write a short message, cut it into pieces, and hide one piece. Can your family work out which one is missing, and what to do about it?

If it isn't clicking

“I’m scared of the internet now.”

What it usually means: something in the safety talk landed too heavily, or your child heard a frightening story somewhere else.

Next move: reassure with the plan, not more warnings. “You have three words that always work, and I’m right here.” Go back to the fun half for a few days (NET-01, NET-04). If a fear persists or seems tied to something your child saw, talk about it gently and privately, outside the lesson.

“This is boring. I already know.”

What it usually means: your child may know the words but hasn’t practiced the routine, or the activities feel babyish.

Next move: hand over the clipboard. Let them design pop-ups for you to catch (SAFE-05), or give an 8–10 NET-05 as a real detective job. Then test with a surprise paper pop-up in a different Hour. Knowing is doing Stop, tell, close without a reminder.

“But my older brother has an account.” / “My friend shares her password.”

What it usually means: the child is noticing that rules differ, and is checking whether yours are real.

Next move: keep it calm and short. “In our family, grown-ups hold the passwords and accounts, and the poster applies to everyone in this house.” Make sure the older sibling’s habits match the poster too. Another family’s choices are theirs; no need to criticize them.

When to slow down, go ahead, or step back

- **Slow down** if Stop, tell, close isn’t automatic yet; practice it once in every Hour for a week.
- **Go ahead** when private-or-okay sorting is quick and packets make sense on paper.
- **Step back to unplugged** whenever a screen activity turns into clicking around.

The rescue order is the same as everywhere in this book: ask, wait, hint, a short demonstration on *your own* paper strips or cards, then hand it back.

Tools, including AI

Everything in this chapter runs on paper, yarn, envelopes, and your own device for NET-05.

- **Paper and scissors** for messages in pieces and paper pop-ups.
- **Yarn** for the string network.
- **Envelopes and sticky labels** for addresses.
- **Your device and your account** for NET-05, with a page you’ve checked first.

- **Optional:** family controls on your devices, if you find them useful. They’re a helpful extra, not a substitute for a shared room and a grown-up nearby.

No accounts for your child, and no AI tools in this chapter. A chatbot is no place for private information either, which makes a good one-line reminder when you reach Chapter 8. For what you may use AI for as the parent, see the AI box in **The Computing Hour**.

What “done enough” looks like

Move on when most of these are true. You’re looking for skills, not birthdays.

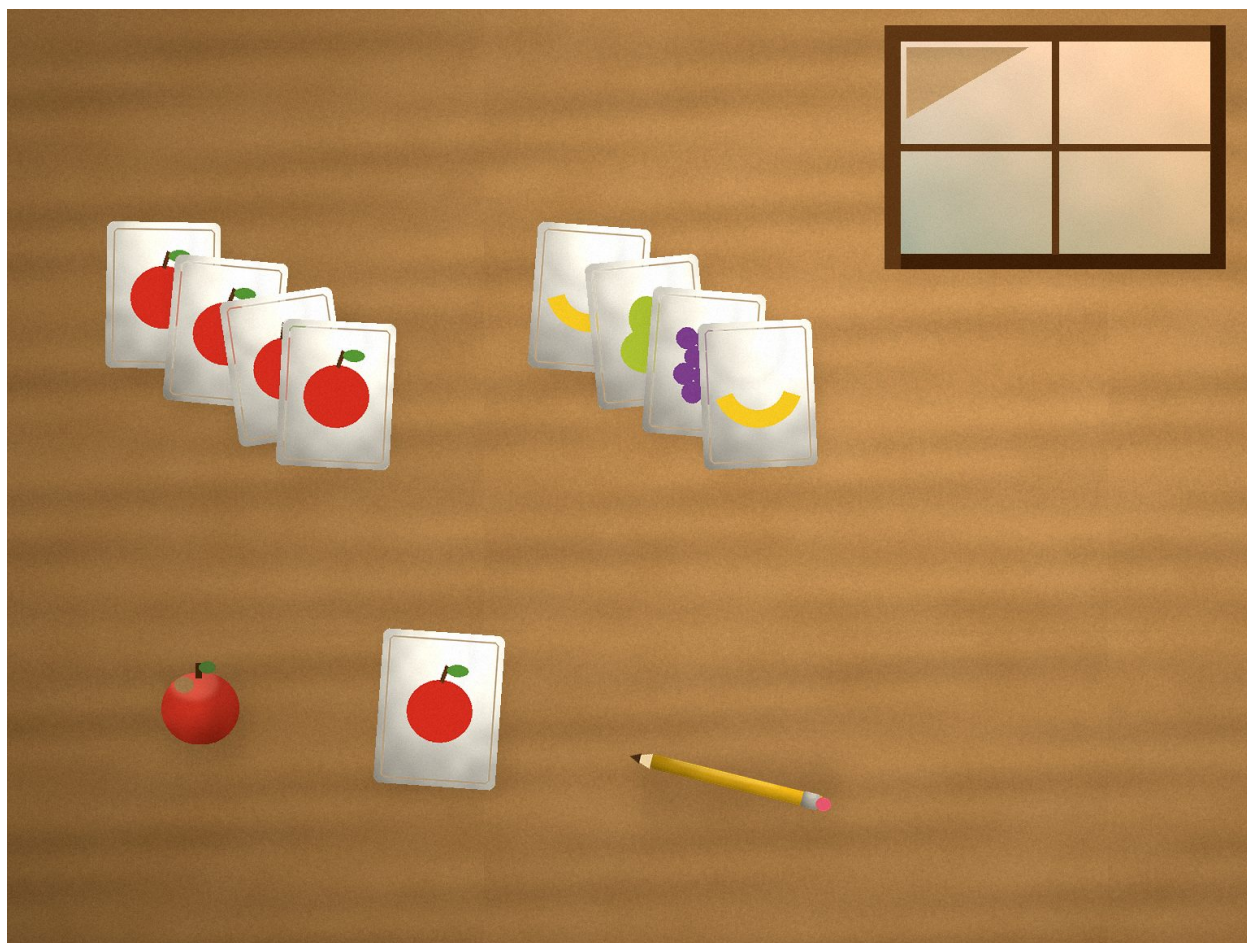
- ☐ Your child says **Stop, tell, close** and acts it out without prompting, including when you surprise them with a paper pop-up during another Hour.
- ☐ They sort **private** from **okay to share** with new cards, and give a reason for at least one.
- ☐ They can tell you **who holds the passwords** (you do).
- ☐ They answer the talk-box opening about one page, photo, or drawing: who made it, who can see it, who to ask.
- ☐ **5–7:** they put a cut-up message back in order and notice when a piece is missing.
- ☐ **8–10:** they explain in a sentence or two how a message travels in pieces with addresses, and they’ve rewritten one unkind comment kindly.
- ☐ The **family device agreement** is signed and on the wall.

Short checks for the file (Chapter 9 gathers these):

- **AS-01 Exit Crumb Trio:** “I predicted... It did... Next I’d change...”
- **AS-02 Show Me Two Ways:** two paths for a note across the string network.
- **AS-03 Debug on the Spot:** hand them a cut-up message with one piece missing or two pieces swapped. Can they find it?
- **AS-04 Teach It Back:** your child teaches a sibling or a teddy Stop, tell, close. Write down one sentence they say.
- **AS-05 Portfolio Pick:** they choose one thing for the file, perhaps the photo of the signed agreement, and say why.
- **BUG-02 Expected/Actual** card from the NET-01 Bug Hunt, and a **BUG-08 Bug Jar** token for each missing piece found.

Nobody can make the internet perfectly safe. You have given your child a plan, three words that always work, and a clear picture of who to ask. That’s a great deal.

Chapter 8



AI Explained, with the Grown-Up at the Keyboard

Why this matters

Picture a family at the kitchen table on a rainy afternoon. (This is an illustration, not a real family.) The older sister is a “sorting machine.” She closes her eyes while her little brother lays out three pictures of red apples, each one labeled APPLE. She opens her eyes, studies them, and nods: she

has learned. Then he holds up a green apple.

“Not apple,” says the machine.

Everyone laughs. The little brother protests that it *is* an apple. The sister shrugs in her best robot voice: “I was only shown red ones.”

That family has just learned the most important idea about artificial intelligence. It fits on one line: **AI learns from the examples people give it, and it guesses wrong when the examples are wrong or too few.**

Children this age hear about AI constantly. A voice assistant answers questions in a friendly voice. A tablet suggests the next video. A grown-up mentions a chatbot at dinner. Without a true picture, children fill the gap with whatever seems likely: that the voice in the speaker is a person, that it knows everything, that it is their friend.

This chapter gives them a true picture, at their size:

- **AI** is a computer program that learned patterns from lots of examples.
- Bad or narrow examples make bad guesses.
- AI can sound like a person. It isn’t one. It has no feelings, and it doesn’t know you.
- Text tools work by guessing the next word, over and over. That’s why they can sound sure and still be wrong.
- Pictures can be made up by a machine, and you can learn to spot clues.
- When a tool makes something, it’s “made by a tool,” not made by you.

The CSTA standards put these ideas in elementary school as things children **examine**, not tools they **use**. A grade 2 standard asks children to “examine how data is used to train a machine learning model.” A grade 4 standard asks them to “distinguish between human learning and machine learning processes.”^{25,26} Every card in this chapter’s bank does exactly that, and almost all of them run on paper.

The chapter title carries the rule. **The grown-up is at the keyboard.** At 5–10, your child learns *about* AI. Your child doesn’t chat with it alone. The Computing Hour holds the full AI box once: what you may use a tool for, what to skip, and why. This chapter points back to it and spends its time on teaching.

There’s a link to the focus line here, too. When the sorting machine guesses wrong, the bug isn’t in the machine’s “brain.” It’s in the examples. Finding the examples that caused the wrong guess, and changing them, is debugging. Predict before you run; debug before you start over. It works on machines that learn, too.

This week you can learn to explain, in two sentences, how a machine learns from examples and why it guesses wrong. **Today your child can try** being the sorting machine, or training one.

For the parent: understand it yourself

You don’t need to understand how any particular AI product is built. You do need one clear picture, a few honest limits, and a calm way to answer when your child says, “Is it alive?”

The everyday picture

Think back to Chapter 3 and the sock-sorting machine. You gave it a rule: *if it's striped, put it here; if not, put it there*. That's an ordinary program. A person wrote the rule, and the machine follows it exactly.

Now imagine you don't give the machine a rule. Instead, you show it a big pile of socks, each one already labeled "striped" or "plain," and let it work out the pattern for itself. Then you hand it a new sock and ask for its guess. That's **machine learning**, the idea behind most of what people call AI today.

The machine's guess can only be as good as the pile. If every striped sock it saw was red, it may decide that "red" means "striped." A plain red sock fools it. A blue striped one fools it too. The machine wasn't silly. It learned exactly what the examples showed.

The precise picture

An ordinary program follows instructions a person wrote: steps, loops, if-then. A **machine-learning** program is given many examples, usually labeled by people, and adjusts itself to match the patterns in those examples. The examples are called **training data**. After training, it makes guesses about new things it has never seen.

Three things follow, and each becomes a card in this chapter:

1. **It learns only from what it was shown.** Narrow or mistaken examples lead to wrong guesses (AI-01, AI-02). A grade 4 CSTA standard asks children to "analyze relationships between the properties of training data and a machine learning model's output," which is the grown-up way of saying the same thing.^{25,26}
2. **Text tools guess the next word.** The chatbots you may have used were trained on enormous amounts of writing. When you type a question, the tool produces a likely next word, then another, then another. That's why the answers sound smooth. It's also why they can be confidently wrong, and why they sometimes invent things that look real, including book titles and studies that don't exist (AI-04).¹⁴
3. **Picture tools make new pictures from patterns.** Those pictures can look real and still contain odd mistakes: a cat with five legs, a clock with too many numbers, writing that isn't quite letters (AI-05).

You may come across a tidy summary of the whole field called the "Five Big Ideas in AI," from AI4K12, a project of two professional groups (AAAI and CSTA): perception, representation and reasoning, learning, natural interaction, and societal impact.⁵⁶ This chapter lives mostly in "learning" and "natural interaction." It's there if you'd like the bigger map. Chapter 10 lists it with other parent references.

A program, not a person

Voice assistants and chatbots are built to sound friendly. They use "I." They say "Great question!" A five-year-old reasonably concludes there is someone in there.

The honest answer is short: it sounds like a person, and it isn't one. It doesn't have feelings. It doesn't know who you are the way a friend does. It isn't your friend, even when it's polite. Even Google's own help page for parents says its protections for children try to limit inappropriate content, "but they're not perfect." That's a fair reminder that the tool is a program people built, with the limits programs have.¹⁷

Sensor or learned?

Not every machine that "decides" is AI. Go back to OUT-05, the automatic door from Chapter 1. Most automatic doors follow an ordinary rule: *if the sensor sees movement, then open*. A person wrote that rule. A machine that learned from examples is different: someone trained it on many pictures of, say, faces or fruit, and it guesses from the patterns. At 8–10, telling those two apart is a real skill, and it's a great errand game.

Whose work is it?

If a tool writes a story, makes a picture, or builds a game, the tool made it. It isn't your child's work, even if your child typed the request or chose the colors. CSTA says plainly that computer science "is not" using tools like "word processors, slide decks, or generative AI tools."¹ This book keeps one simple label for anything a machine made: **made by a tool**. Chapter 9 shows how that label appears in your records.

Age terms, in plain language

You'll hear about chatbots and "kids' versions" from friends. Here's where things stand as of this book's checking date.

Most of the big consumer AI chat services set **13** as their minimum age, and ask for a parent's permission for teenagers. One major service, Anthropic's, sets **18**.¹⁶ Google is the notable exception: a parent can turn on its Gemini app for a child under 13 through a supervised Google account.¹⁷

This book names that option so the picture is honest, and doesn't recommend it for ages 5–10. The reason isn't one company's settings. It's the shape of the Hour: at this age the child does the thinking, predicting, building, and debugging, and the grown-up's hands are the safety feature. Independent testing groups have reported "kids modes" and talking AI toys giving children things no parent would choose.^{11,12} A chatbot, companion, voice "friend," or AI toy isn't something a 5–10 child uses alone, kids mode or not.

Terms change, sometimes several times a year. Before you rely on any of this, check the current terms yourself.

The Hour of AI

You may hear about the **Hour of AI**. It's a set of short activities organized by CodeAI (formerly Code.org) and CSforALL, and it has replaced the older Hour of Code.^{19,21} The activities run year-round and tie in with Computer Science Education Week each December. They're designed for

everyone “from pre-readers to adults,” and some are unplugged.²⁰ Some activities include an AI assistant.

The move is simple: **you preview first.** Open the catalog on your own, filter for your child’s age, and pick an unplugged activity or one with no chat. If an activity has a built-in AI helper, keep it off or run it yourself. The Hour of AI is a set of activities, not a chatbot for children, and that’s how it fits this book.

Wrong answers you should be able to hear

“It’s alive.” / “The speaker likes me.” *What it usually means:* the friendly voice did its job; the child reasonably assumes a person. *What you say:* “It sounds like a person. Let’s find out what it’s made of.” Then run AI-03 Person or Program?

“The AI said it, so it’s true.” *What it usually means:* the child hasn’t yet learned that a guess can sound sure. *What you say:* “It’s very good at sounding sure. Let’s check it in a book.”

“I made this!” (about a picture or story a tool made) *What it usually means:* the child chose the request and feels ownership. That’s understandable. *What you say:* “You had the idea. A tool made the picture. What did you make?” Then label it together (AI-07).

“The automatic door is AI.” / “Everything that moves by itself is AI.” *What it usually means:* the child is lumping all automatic machines together. *What you say:* “Did a person write a rule for it, or did it learn from examples? Let’s look for the sensor.”

“It got it wrong, so it’s stupid.” *What it usually means:* the child expects the machine to “know.” *What you say:* “It didn’t know. It guessed from what it was shown. What examples would have helped?”

Five-minute parent warm-up

1. Cut out or draw six fruit pictures: four red apples, one green apple, one red tomato. That’s tomorrow’s sorting machine.
2. Say aloud, as if to your child: “AI is a computer program that learned patterns from lots of examples. If the examples are wrong, the guesses are wrong.” Say it until it sounds like you.
3. Practice the talk-box opening: “What examples did it learn from — and where might it guess wrong?” Count to five in your head.
4. If you plan to use an AI tool yourself this week, open the AI box in **The Computing Hour** and reread the “you may” list.

How to teach it this week

The Computing Hour shape holds: Unplug First, Predict, Build and Run, Bug Hunt, Save and Tell. Almost every Hour this week is screen-free. That isn’t a compromise. The sorting games teach the idea more clearly than any screen could.

A worked Hour (illustration)

Unplug First (5 min). “Today you’re going to teach a machine. I’m the machine. I don’t know anything yet.” Close your eyes dramatically.

Predict (3 min). Your child chooses four pictures to “teach” you: apples labeled APPLE, other fruit labeled NOT APPLE. Before you look at the test picture, ask: “When I see this green apple, what do you think I’ll say?”

Build and Run (8 min). Open your eyes. Study the training pictures, out loud: “Apples are... red. And round.” Then guess the test picture, honestly, using only what you were shown. If you saw only red apples, the green one is “not apple.” If you saw a red tomato labeled NOT APPLE but it looks like the apples, wobble.

Bug Hunt (5 min). “What did you expect me to say, and what did I say instead? Which examples taught me that?” Let your child change the training pictures: add a green apple, add a yellow one. Run it again. You’ve just debugged the data.

Save and Tell (3 min). Photograph the two piles of training pictures. Your child says one sentence to someone: “The machine only knew red apples, so it got the green one wrong.”

Exact wording you can say

- To open: “I’m a machine that learns from examples. Teach me.”
- Before the test: “What will I say about this one? Why?”
- At the wrong guess: “Interesting. Where did I learn that?”
- To fade help: “What examples would fix my mistake?”
- When your child asks to try a real chatbot: “That’s a grown-up tool, like the oven. Sometimes I use it, and when we look together, my hands are on the keyboard. Your ideas stay yours.”

In-home — AI-01 Teach the Sorting Machine

- **Time:** 15 minutes.
- **Materials:** 8–12 picture cards of fruit, toys, or animals (drawn, printed, or cut from a catalog); two labels, such as APPLE and NOT APPLE.
- **Safety/privacy:** pictures of things, never photos of people.
- **The fun:** a sibling or parent playing a very literal machine.
- **The skill:** a machine that learns finds patterns in the examples it’s given, then guesses about new ones.
- **What to save:** a photo of the training piles, and one sentence: “It learned that...”
- **5–7:** you play the machine; your child is the teacher. Keep it to one idea (color, shape, size).
- **8–10:** your child plays the machine for a younger sibling, or designs a “tricky test” picture on purpose.

When to stop talking

When your child is choosing training pictures, let them choose, even badly. A narrow set is a gift: it guarantees a wrong guess and a good Bug Hunt. When the machine guesses wrong, wait. Let your child say why before you do.

A week of this chapter

- **Day 1:** AI-01 Teach the Sorting Machine.
- **Day 2:** AI-03 Person or Program?
- **Day 3:** AI-04 Guess the Next Word.
- **Day 4:** AI-07 Whose Work Is It?
- **For 8–10, add:** AI-02 Bad Examples, Bad Guesses; AI-05 Spot the Made-Up Picture; AI-06 Check It in a Book.
- **Once this week, out of the house:** OUT-05 Automatic Door Detective, revisited.
- **Friday:** the “done enough” list, plus AS-04 Teach It Back.

Age-band moves

Ages 5–7. The sorting machine, person or program, guess the next word with nursery-rhyme starters, and whose work is it. Keep to unplugged games. No AI chat for the child, even with you sitting beside them. That’s the AI box’s line for this age, and it’s an easy one to keep: the games are more fun.

Ages 8–10. Add bad examples and bad guesses, spotting a made-up picture that *you* made of an object or animal, and checking a question in a book. For AI-06, your child may sit beside you and say a question aloud while you type it into your own account. Then you both check the answer against a library book. Your account, your hands, no account of their own.

Practice that actually builds learning

Blocked practice for the new idea: three days of sorting-machine games with new materials each time (fruit, then socks, then toy animals) until your child can predict the machine’s mistake before it happens. **Mixed practice** once it’s known: a quick round of “person or program?” cards with a “rule or learned?” question added for each machine.

Errands count. A self-checkout that asks for help, a suggested-video list, a supermarket door: each one is a chance to ask, “Rule, or learned from examples?”

Talk box (this chapter)

Opening (exact): “What examples did it learn from — and where might it guess wrong?”

Use it after the sorting machine, after a next-word round, or when your child sees a machine “decide” something in the world.

Follow-ups (pick three to five):

1. *Prediction*: “What do you think it will say about this new one?”
2. *Evidence*: “What did you see in the examples that makes you say that?”
3. *Location*: “Point to the example that taught it the wrong thing.”
4. *Change*: “What’s one example we could add or take away?”
5. *Revise*: “You can change your guess. Shall we run it again?”

And two that belong especially to this chapter: “Is it a person?” and “Who made this: you, or a tool?”

How to wait. After you ask, count to five in your head, the same pause craft the whole book uses.⁵ After your child answers, wait again. If they’re rearranging the training pictures, let them finish before you ask anything.

What stuck silence usually means.

- The question was too big. “Where might it guess wrong?” can be hard; try “Which picture will fool it?”
- There was no wait.
- Your child thinks AI is “smart,” so it can’t be wrong, and doesn’t want to say otherwise. Say, “Machines guess wrong all the time. Finding where is the fun part.”
- They skipped the prediction, so there’s nothing to compare. Go back: “Before we test, what will it say?”
- A real device is nearby and talking. Put it in another room.

The stuck-silence move: offer two choices (“Will it say apple or not apple?”) or ask them to point at the picture that will cause trouble.

In-home — AI-02 Bad Examples, Bad Guesses (8–10)

- **Time:** 15 minutes.
- **Materials:** picture cards with a deliberately narrow training set: only red apples, only big dogs, only striped cats.
- **Safety/privacy:** pictures of things or animals only.
- **The fun:** setting a trap for the machine.
- **The skill:** narrow examples cause wrong guesses; better examples fix them.
- **What to save:** “It guessed wrong because it only saw...”, written by your child.
- **The move:** your child trains the “machine” (you or a sibling) on the narrow set, predicts the mistake, tests it, then fixes the training set. Ask: “If a real machine only saw pictures of one kind of thing, who might it get wrong?” Leave the conversation there. A ten-year-old understands fairness without a lecture.

In-home — AI-03 Person or Program?

- **Time:** 10 minutes.
- **Materials:** picture cards: a cat, a friend, a calculator, a talking toy, a voice assistant speaker, a grandparent on a video call, a robot vacuum.

- **Safety/privacy:** drawings or catalog pictures.
- **The fun:** arguing about the tricky ones. (Grandma on a video call is a person, even though she's on a screen.)
- **The skill:** telling people from programs; noticing that “sounds like a person” isn't “is a person.”
- **What to save:** one reason, in your child's words: “It talks, but it doesn't have feelings.”
- **5–7:** two piles, person and program. **8–10:** add a third question for each program: “Did a person write its rules, or did it learn from examples?”

In-home — AI-04 Guess the Next Word

- **Time:** 10 minutes.
- **Materials:** a few sentence starters: “Twinkle, twinkle, little...”, “Once upon a...”, “For breakfast I like...”, “The cat sat on the...”
- **Safety/privacy:** none needed.
- **The fun:** everyone guessing at once, then comparing.
- **The skill:** text tools work by guessing a likely next word, again and again.
- **What to save:** one starter and the family's guesses, written down, with your child's answer to “Why did we guess different words?”
- **The move:** read a starter and pause. Everyone says the next word. Easy ones (“star”) show how guessing from patterns works. Open ones (“For breakfast I like...”) show that there's no single answer, and the guess depends on what you've heard before. At 8–10, say: “That's how a chatbot writes. It guesses the next word, then the next. It can sound right and still be wrong.”

In-home — AI-05 Spot the Made-Up Picture (8–10)

- **Time:** 10 minutes.
- **Materials:** one or two pictures of an object or animal that **you** made beforehand with an image tool on your own account, plus a real photo of something similar.
- **Safety/privacy:** objects and animals only. Never a person, never a child, never your child's photo as a starting point.
- **The fun:** detective work.
- **The skill:** made-up pictures can look real; there are often clues.
- **What to save:** two clues your child spotted (“The dog has too many toes”; “The sign has squiggles instead of letters”). Label the picture “made by a tool” before it goes anywhere near the folder.
- **The move:** show both. Ask which is made up and how they know. Count fingers, toes, legs, and letters. If they can't tell, that's an honest lesson too: some made-up pictures are very hard to spot, which is why we ask who made a picture.

In-home — AI-06 Check It in a Book (8–10)

- **Time:** 15 minutes.

- **Materials:** your device and your own account; a library book on a topic your child cares about (sharks, volcanoes, planets).
- **Safety/privacy:** you type; your child speaks. No names, photos, or personal details in the question.
- **The fun:** a contest between a book and a machine.
- **The skill:** checking a claim against a trusted source; noticing that a fluent answer can be wrong.
- **What to save:** the question, the tool's answer, the book's answer, and "Did they agree?"
- **The move:** your child asks a factual question aloud ("How many teeth does a great white shark have?"). You type it. Read the answer aloud. Then find the same fact in the book together. If the tool cites a source, open it before trusting it.¹⁴ Agreement is a good result. Disagreement is a better lesson.

In-home — AI-07 Whose Work Is It?

- **Time:** 10 minutes.
- **Materials:** three things on the table: a drawing your child made, a picture a tool made (an object or animal, made by you), and a photo you took.
- **Safety/privacy:** the tool-made picture shows no people.
- **The fun:** sticking honest labels on everything.
- **The skill:** naming who made something.
- **What to save:** the three labels: "made by me," "made by a tool," "made by Mom."
- **The move:** ask, for each thing, "Who made this?" Then ask, "Which one goes in your Computing folder as your work?" The answer is the drawing. The tool's picture can go in only with its label. Chapter 9 explains why.

Out-of-home — OUT-05 Automatic Door Detective, revisited

- **Time:** an ordinary errand.
- **Materials:** a store with automatic doors, a hand dryer, a self-checkout, perhaps an elevator.
- **Safety/privacy:** stay out of doorways and away from moving parts; hold hands in parking lots; no photos of other shoppers.
- **The fun:** spying on machines.
- **The skill:** telling a rule-following machine from one that learned from examples.
- **What to save:** two "if ___ then ___" rules your child spotted, and one machine they think might have learned from examples (and why).
- **The move:** in Chapter 1 your child found the sensor and said the rule. Now ask the new question for each machine: "Did a person write a rule for this, or did it learn from examples?" Most doors and dryers follow rules. A machine that suggests things, or recognizes faces or voices, may have learned. Your child doesn't need the right answer for every machine. Asking the question is the skill.
- **5–7:** just the rule: "If I walk up, then it opens." **8–10:** rule or learned, with a reason.

One incorrect example to diagnose (illustration)

Picture a nine-year-old at a cousin's house. The cousin's parent types "make a game about a dragon collecting coins" into a tool, and a working game appears. Your child comes home thrilled and asks to put "my dragon game" in the Computing folder.

What went wrong: a tool made the game. Your child had the idea and watched. It isn't your child's program.

What to do, kindly: share the excitement first; it's a cool thing to have seen. Then run AI-07 with the game as the third object. Ask, "Who made it?" Let your child land on "a tool." Then turn it into a real project: "Shall we plan a dragon game on paper and build it in Scratch, block by block?" That's DECOMP-04 and SCR-05 from Chapters 2 and 5. The idea was theirs. Now the program can be too.

For the student

AI stands for **artificial intelligence**. It's a computer program that learned patterns from lots of examples.

Here's how it works. People show the computer thousands of pictures labeled "cat" and thousands labeled "dog." The computer looks for patterns. Then it guesses about a new picture it has never seen.

A tiny example. You show a "machine" three red apples and say, "Apple." Then you show it a green apple. The machine says, "Not apple!" Why? It only ever saw red ones. It learned exactly what you taught it, and no more.

So AI can be useful, and it can also be confidently wrong. When it's wrong, the problem is usually the examples it learned from.

Some AI talks. It might say "I" and sound friendly. But it isn't a person. It doesn't have feelings, and it doesn't know you. It's a program people made.

When a tool makes a picture or a story, a tool made it. You can say so: "made by a tool." Your own drawings, programs, and ideas are yours.

AI tools are grown-up tools, like the oven. When your family looks at one together, your grown-up's hands are on the keyboard.

Try it today. Be the sorting machine. Close your eyes. Let someone teach you with picture cards. Then guess the new one using only what you were shown.

Explain it back. Tell someone why the machine said a green apple wasn't an apple.

Challenge. Choose training pictures that will trick the machine on purpose. Then fix them so it guesses right.

If it isn't clicking

“It IS alive. It talks to me.”

What it usually means: the voice is convincing, and your child has a warm feeling about it. That's normal at this age.

Next move: don't argue; investigate. “It sounds like a person. Let's find out what it's made of.” Run AI-03, and ask what a person can do that the speaker can't: feel sad, give a hug, remember your birthday party, choose to be kind. Come back to it more than once. This idea grows slowly, and that's fine.

“I want my own AI.” / “My friend uses one.”

What it usually means: your child sees something exciting and wants in.

Next move: say yes to the idea and no to the account. “It's a grown-up tool, the way the stove is. You can learn how it works right now, and when we look at one, we look together.” Then give them something real to own: designing a sorting-machine game for a younger sibling, or an AI-06 question you'll type together on Friday. Keep the line the same every time. Consistency does more than explanation.

“Robots are going to take over.” / A worried child

What it usually means: your child has overheard something frightening, or something hyped, about AI.

Next move: keep it small and true. “AI is a program people make, from examples people choose. People decide what it does and when to switch it off.” Go back to AI-01. A child who has *been* the machine, and watched it fail on a green apple, finds it hard to stay scared of it.

When to slow down, go ahead, or step back

- **Slow down** if your child still thinks the machine “knows” things. Repeat AI-01 with new materials until they can predict its mistake.
- **Go ahead** when they can explain a wrong guess in terms of the examples.
- **Step back to unplugged** any time a real tool becomes the center of attention. The cards teach the ideas better.

The rescue order holds: ask, wait, hint, a short demonstration with *your own* picture cards, then hand it back.

Tools, including AI

Picture cards, a few labels, a library book, sentence starters on slips of paper, and an errand. That's the whole kit for this chapter.

If you use an AI tool yourself, the AI box in **The Computing Hour** is your guide, and it's worth rereading before AI-05 or AI-06. The short version: your account, your hands, previewed first; nothing of your child's face, voice, or name goes in; and anything a tool makes is labeled "made by a tool." The Hour of AI fits this chapter if you preview it and choose unplugged or non-chat activities.

What "done enough" looks like

Move on when most of these are true. Skill, not birthday.

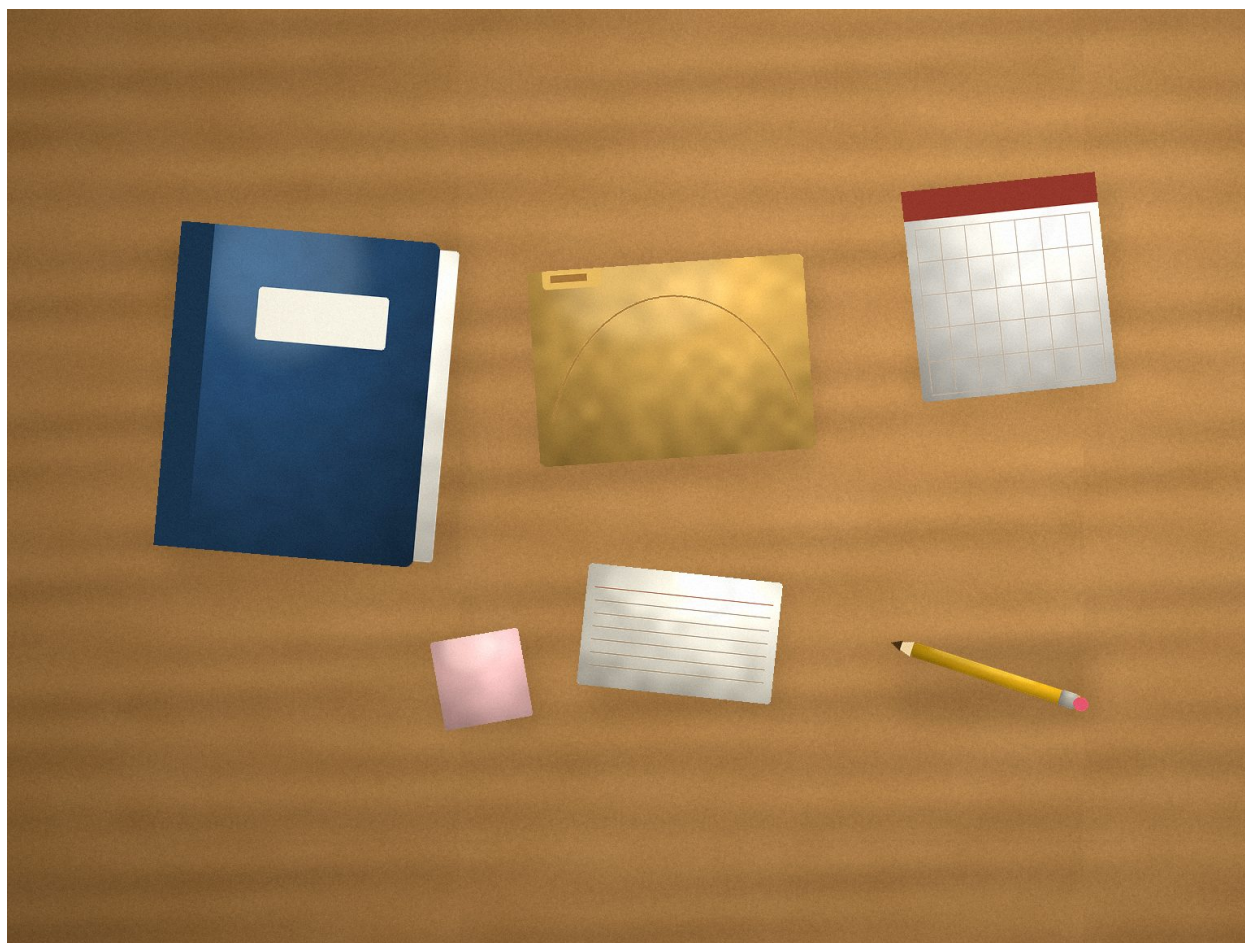
- ☐ Your child can say, in their own words, that **AI learns from examples**.
- ☐ They can give **one reason a machine guessed wrong** ("it only saw red apples").
- ☐ They sort **person** from **program** and give a reason for at least one tricky card.
- ☐ They label things **made by me** / **made by a tool** / **made by a grown-up**, and know which ones go in the folder as their work.
- ☐ **5–7:** they've played the sorting machine as both teacher and machine, and they say "It's a grown-up tool" without fuss.
- ☐ **8–10:** they've fixed a training set to correct a wrong guess, spotted at least one clue in a made-up picture, and checked one answer in a book.
- ☐ On the errand, they've asked "rule or learned?" about at least one machine.

Short checks for the file (Chapter 9 gathers these):

- **AS-01 Exit Crumb Trio:** "I predicted... It did... Next I'd change..."
- **AS-02 Show Me Two Ways:** two different training sets that both teach the machine correctly.
- **AS-03 Debug on the Spot:** you lay out a training set with one misleading example. Can your child find it?
- **AS-04 Teach It Back:** your child teaches a sibling or a teddy how the sorting machine learns. Write down one sentence.
- **AS-05 Portfolio Pick:** they choose one thing for the file (perhaps the photo of their training piles) and say what it shows.
- **BUG-02 Expected/Actual** card from a sorting-machine Bug Hunt, and a **BUG-08 Bug Jar** token for each bad example found.

A child who finishes this chapter won't know how any particular AI product is built. They'll know something more useful: machines learn from examples, examples can be wrong, a friendly voice isn't a person, and their own work is theirs.

Chapter 9



Records

A year of Computing Hours leaves a trail: card strips on the floor, a Bug Jar full of tokens, a cat that finally jumps at the right moment, a signed family device agreement on the wall. Most of it disappears by Tuesday. A few minutes a week can keep the best of it.

This chapter shows you how to keep records that a stranger could read: a grandparent, an evaluator if your state uses one, a teacher if your child ever moves to a school, or your own future self trying

to remember what happened in second grade. The records stay small. They're a folder, not a filing cabinet.

This week you can learn the one-line entry: date, card code, minutes, one sentence. **Today your child can try** choosing one thing for the folder and saying why.

Why keep records at 5–10

There are three good reasons, in this order.

Memory. You'll forget. By spring, October's Robot Grown-Up will be a blur. A dated photo and one sentence bring it back, and show you how far your child has come.

A reader outside the house, if there is one. Some states ask for a portfolio, an evaluation, or a list of subjects. Some ask for nothing at this age. Most states don't require computer science for young children at all. When this series checked five states with very different rules (Texas, North Carolina, Pennsylvania, Virginia, and New York), none of them named computer science or technology among required elementary subjects.⁵⁷ That doesn't mean computing doesn't count. It means it's your choice to teach it, and your records show what happened. This isn't legal advice. Check your own state's current law, on the state's own page.

Your child's pride. A folder they helped fill, with things they chose, says "I can do this" more clearly than any certificate.

You're also doing something many homeschooling families haven't. In a 2016 federal survey (the most recent found that asked), about one in six homeschooled students (16 percent) had been taught computer science that year; in the grades 3–5 band it was 18 percent.⁵⁸ Whatever you keep, keep it honestly. That's the whole point.

What goes in the file

Seven kinds of thing, and you don't need all of them every week.

Dated photos, cropped to the work

Most of this book is unplugged, so most of your evidence will be photos: an arrow-card path on the tape grid, a cup tower beside its code, a message laid out in numbered pieces, a sorting machine's training piles.

Crop every photo to the work. Cards, cups, paper, a screen of blocks. No faces. Under the children's privacy law, a photo of a child counts as personal information,⁶ and in this family's habits, faces don't go into apps or shared folders. A hand pointing at the first wrong card is fine. A face isn't needed.

Name the photo with the date first, then the card code, then a word or two: 2026-10-14-STEP-03-arrow-path.jpg. That's the same rule your child learned in Chapter 6 (FILE-02 Name It So Grandma Can Find It). Date-first names sort themselves into order.

Saved files with names a stranger can read

For screen work, save the project itself with the same kind of name: 2026-11-03-cat-jump, 2027-02-18-maze-game-v2. “Untitled,” “Project 7,” and “asdfgh” are the three most common names in any child’s folder. They’re also the three hardest to find.

At 5–7, you’ll usually do the naming while your child watches and says what the project is. At 8–10, your child does it, and checks they can find it again (FILE-03 Save, Close, Find).

Expected/Actual cards

The Expected/Actual card (BUG-02) is the heart of this book on an index card: what I thought would happen, what actually happened. Keep a few each month. A five-year-old’s card may be two drawings. A ten-year-old’s may be two sentences and an arrow. Either way, it shows your child predicting before running, which is half the focus line.

A Bug Log page (8–10)

One page a month is plenty. Each line has a date, what went wrong, how they found it, and what they changed (BUG-06). A page with three honest bugs on it shows more about your child’s learning than a page of finished projects. It shows debugging, the other half of the focus line.

A typing sample (only if typing was a strand)

If your 8–10 child practiced home-row typing in short, regular bursts (Chapter 6), keep one sample a month: the date, the practice, and an accuracy note. If typing was just something that happened now and then, skip this. That’s fine too.

“Made by a tool” notes

Whenever you used an AI tool, or any generator, for something that ended up near the folder, write it down plainly. “AI-05: made-up dog picture, made by a tool (parent’s account), used for the spot-the-clues game.” “Puzzle sheet made by a tool; child solved it.”

Anything a tool made is labeled “made by a tool” and never filed as your child’s work. The label isn’t a confession. It’s simply accurate, and it protects the value of everything else in the folder.

A one-page year summary

At the end of the year, write one page: what you studied, roughly how often, what’s in the portfolio, and two or three sentences about what your child can now do. There’s an example later in this chapter.

A five-minute Friday routine

Records stay small when they happen at the same time each week. Here's one shape that works.

1. **Gather.** Pull this week's photos off the phone into the month's folder. Drop any Expected/Actual cards and Bug Log lines into the month's envelope.
2. **Name.** Rename anything called "IMG_4471" with the date-first rule. At 8–10, your child does their own project files.
3. **Pick.** Run AS-05 Portfolio Pick. One item, one caption in your child's words.
4. **Log.** Make sure each Hour this week has its one line in the hours log.
5. **Copy.** Once a month, copy the folder to its second place.

That's it. If a week was rough and you only managed step 4, the week still counts.

At 5–7, a typical month in the folder might hold six to ten cropped photos, three or four Expected/Actual cards (drawings are fine), four captions from portfolio picks, and the hours log. **At 8–10**, add a Bug Log page, a few saved projects with readable names, and, if typing was a strand, one typing note.

Where records live

Keep records somewhere you control: a family drive, a computer you own, or a paper folder in a box. Better still, keep them in **two places**, so one spilled drink or one lost laptop doesn't take the year with it. That's the Two Places Rule from Chapter 6 (FILE-04).

A chatbot isn't a filing cabinet. And if you use Grok Bot, remember that all of your bots share one computer, so keep your child's records and your family's photos on a family drive or on paper instead.¹⁸

A paper folder is a perfectly good record. A box with twelve labeled envelopes, one per month, is a complete system.

The check set

There's no test chapter in this book, and no test battery. Each teaching chapter ends with a short "What 'done enough' looks like" list and a handful of short checks. This section gathers those checks in one place so you can see them as a set.

The checks

Each one takes two to five minutes. Use one or two a week, usually on a Friday, never all of them at once.

- **AS-01 Exit Crumb Trio.** At Save and Tell: "I predicted... It did... Next I'd change..." Spoken at 5–7 (you write it down); written at 8–10. Keep a few per month.
- **BUG-02 Expected/Actual card.** Already in your folder from the Bug Hunts.

- **AS-02 Show Me Two Ways.** The same goal reached by two different programs or paths. Two card strips to the same cheese; two string-network routes; two training sets that both teach the sorting machine. Photograph both.
- **AS-03 Debug on the Spot.** You lay out a short program, four to six steps, with one bug. Can your child find the first wrong step? Write “found” or “not yet,” and what you’ll try next week. A “not yet” is useful information, not a grade.
- **AS-04 Teach It Back.** Your child teaches a sibling, a grandparent, or a teddy today’s idea. You write down one sentence they say, word for word.
- **AS-05 Portfolio Pick.** Your child chooses one thing for the folder and says why. You write their reason as a caption.

Seen, growing, not yet

Once a semester, or at the end of the year, look across the folder and mark each big idea of the book. Three words are enough.

Big idea (chapter)	What “seen” looks like at 5–7	What “seen” looks like at 8–10	Seen / growing / not yet
Steps and algorithms (2)	Orders a short routine; builds a card path to a goal; fixes a missing step	Writes steps someone else can follow; compares two ways	
Patterns, loops, if-then, events (3)	Extends a pattern; uses a repeat card; follows an if-then rule	Writes a loop with a stop condition; writes if-then-else; names events	
Debugging (4)	Points to the first wrong card; one change before rerunning	Logs a bug and fix; changes one thing at a time	
Block programs (5)	ScratchJr script with a start block and three or more blocks	Tours, remixes one thing, builds a small paper-planned project	
Computers up close (6)	Names input and output in a household device; finds own name on the keys	Makes a number in binary; saves, closes, and finds a file	
Internet, safe and kind (7)	Says Stop, tell, close; sorts private from okay to share	Explains a message traveling in pieces; rewrites an unkind comment kindly	

Big idea (chapter)	What “seen” looks like at 5–7	What “seen” looks like at 8–10	Seen / growing / not yet
AI ideas (8)	Says AI learns from examples; sorts person from program	Explains a wrong guess from its examples; labels “made by a tool”	

These descriptions come from the “done enough” lists in each chapter. If a line says “not yet,” that’s a placement note, not a verdict. It tells you where next week starts.

There are no grades at 5–7. At 8–10, a short narrative line for each big idea is plenty (“Debugs one change at a time; still rushes predictions on screen days”). If your state or a later school asks for a grade, you can give one, but it’s never the point, and it’s never a percentile.

What doesn’t go in the check set

Program badges, completion pages, and scores from outside activities can go in the folder as artifacts, labeled with what they are. They aren’t the check set. An optional puzzle event such as Bebras, which runs once a year and is free to enter, is a fun outing, not a score for your records.⁵⁹ And nothing a tool generated is ever evidence of your child’s skill.

Writing the narrative line

At 8–10, and in any year summary, a sentence or two about each big idea does more than a grade. The trick is to describe what you *saw*, not how you felt about it.

- Vague: “Did great with loops!” Specific: “Replaces repeated steps with a repeat block without being reminded.”
- Vague: “Struggles with debugging.” Specific: “Finds the first wrong step on cards; on screen, still changes several things at once.”
- Vague: “Loves AI.” Specific: “Explains why the sorting machine guessed wrong, in terms of its examples.”

Specific lines are kinder, too. “Still changes several things at once” tells everyone, including your child, exactly what to practice next. It doesn’t say anything about who your child is.

A portfolio-pick talk (optional)

Once a week, at Save and Tell, try this two-line exchange. It takes under a minute.

You: “Which one should go in the file — and what does it show you can do?” **Your child:** “The arrow cards, because I found the wrong one by myself.”

Write the child’s answer as the caption, in their words. Then stop. If they shrug, offer a choice of two (“this photo or this card?”) and ask again. The caption is the record. The choosing is the learning.

The hours log

Some states ask for hours or days. Even where nobody does, a log helps you see the rhythm of your year.

Keep it simple: one line per Hour.

2026-10-14 · 25 min · STEP-03, BUG-01 · Toy reached the cheese on the second run.

Date, minutes, the card codes you used, and one sentence. Five-minute typing bursts count as minutes. Library visits and errands where you ran an out-of-home try-it count too.

The log is not proof of learning. The portfolio is. The log only shows that you showed up, which is no small thing.

Titles a stranger can read

At 5–10 there’s often no transcript at all. If you need a name for the year, for a state form, an evaluator, a school, or your own summary, use one a stranger would recognize.

- **Computer Science.** A year with algorithms, debugging, and programming. Unplugged card programs and block coding both count. Most families using this book will write only this line.
- **Keyboarding.** Only if typing was a separate, real strand, practiced regularly.
- **Digital Literacy.** Only if the year was mostly safety, files, and device use rather than programming.

Other subjects keep their plain names: Reading or Language Arts, Mathematics, Science, History or Social Studies, Art, Music, Physical Education.

Never a brand or a book title in the title column. Not “Little Thinkers Computing I.” Not “ScratchJr Level 2,” a CodeAI (formerly Code.org) course letter such as “Course C,” “Bebras,” or “Hour of AI.” Those are tools and events. If you want to name them, put them in the description: “Computer Science (unplugged card programs; ScratchJr; CodeAI CS Fundamentals Course B).”

No credits at 5–10. Credits are a high-school idea.

No certificates from this book. There’s nothing to award. If a program issues a completion page, keep it in the folder as an artifact. It isn’t a credit, and it isn’t a grade.

Combining tools doesn’t create a new subject. Library books, cards on the floor, ScratchJr, and one CodeAI course in the same year is still **Computer Science**.

A “Course C” or a “grade 2 book” is a publisher’s scope, not your child’s grade and not a transcript line.

Two entries, as illustrations

These are illustrations of the format, not real families.

A 5–7 portfolio entry:

2026-10-14 · STEP-03 Arrow Card Path · 15 min · Toy reached the cheese on the second run; child found the missing “turn right” card. Photo: card strip (work only). Caption (child’s words): “I fixed the turn.”

An 8–10 year summary:

Computer Science (Grade 4). Computing Hours three to four times a week, September to May (about 90 sessions of 20–40 minutes). Unplugged algorithms, loops, conditionals, and debugging; Scratch projects planned on paper (maze game, score keeper); binary and parity card activities; internet safety and AI ideas, taught unplugged. Portfolio: 12 dated program photos, 6 saved projects, Bug Log (9 pages), Expected/Actual cards. Narrative: debugs one change at a time; explains events and loops clearly; still learning to write predictions before screen work.

Keyboarding (Grade 4). Home-row practice in 5-minute bursts, three times a week; accuracy noted monthly.

Use whatever grade label your family or state already uses. The label describes the year of schooling, not a level your child has to reach.

What an honest entry isn’t

Each of these sounds official and tells a reader nothing true:

- “Little Thinkers Computing I” (a book title, not a subject)
- “ScratchJr Level 2” (a tool, not a course)
- “Certified Junior Coder” (a certificate nobody issued)
- “AI Studies: designed a game with a chatbot” (a tool’s work filed as the child’s)
- “1 credit Computer Science, Grade 2” (credits don’t exist at this age)

The honest version is always shorter and plainer, and it’s always more useful to the next reader.

Special situations

A year with no device, or very little. If your child did algorithms, debugging, and programming with cards, cups, tape, and the occasional library computer, the title is still **Computer Science**. Card programs are programs. Your photos and Expected/Actual cards are the evidence. Note the library visits in the hours log.

Siblings and mixed ages. Each child gets their own folder and their own captions, even when they worked as a pair. In a driver-and-navigator session, a line like “Navigator: read the plan and spotted the bug; driver: built it” keeps both children’s work honest. The older child isn’t recorded as the younger one’s teacher. Both were programmers.

Co-ops and groups. Photograph only the work, never the group. For a shared project, write down what *your* child did: “Planned the maze on paper; built the score keeper.”

When you operated the device for your child. Some children, especially those with fine-motor challenges, tell you each block while you place it. That’s real programming: your child decided every step. Record it plainly: “Child designed and debugged the program aloud; parent operated the tablet on the child’s instructions.” Honest and complete.

If your child moves to a school

A receiving school is a stranger with very little time. Send the one-page year summary and three to five samples: a cropped photo of a card program, an Expected/Actual card, a Bug Log page, and one saved project with its name and a two-line description. Keep the originals; send copies.

Expect the school to place your child by what they see, perhaps with a short task of their own. That’s normal, and it’s the same principle this book uses: place by skill, not birthday. A plain, honest folder makes that conversation easy.

Place by skill, not birthday

Your records also tell you where to go next. If the folder shows a nine-year-old who still needs help finding the first wrong step, the next weeks belong in Chapter 4, whatever the birthday says. If a six-year-old is writing steps for a sibling to follow, let them borrow 8–10 cards. The “seen / growing / not yet” table is a map for next semester, not a report card.

Looking ahead

When your child is ready for longer programs, text-based code, and the records that come with middle school, *Computing for Young Minds* (ages 11–14), the next book in this series, picks up from here. Bring the folder. The habits in it (predict, debug, save with a name a stranger can read, say who made what) carry straight on.

Chapter 10



Resources

This is a short shelf, not a shopping list. Nothing on it is required. A full year of this book can run on paper, tape, cups, a library card, and whatever device you already have.

The programs here are arranged by **fit**, not ranked. The right one for your family depends on your child, your devices, and how much of the teaching you want to carry. Each description says what the program says about itself (who it's for, whether it's free) as checked when this book was written.

Nothing here is a promise that one program teaches better than another.

Two plain rules run through the chapter. “Free” appears only where the program’s own page says so. And there are no prices. Hardware prices move, so check the current price and age guidance yourself before you buy anything.

This week you can learn which one program fits your child next, and why. **Today your child can try** a library trip to choose a book about computers.

How to read this list

Four questions do most of the work.

Device or none? Can this run on paper and cards? Does it need a tablet, a laptop, or a robot?

Structure or open play? Does it give you a lesson-by-lesson course, or a set of tools and ideas you arrange yourself? Both are good. Many families mix them. Younger children often do well with more structure and something they can touch; older children enjoy more open projects.

Reader or pre-reader? Some tools use pictures and icons. Others need your child to read words on blocks or instructions.

Parent load and accounts. How much setting up, previewing, and sitting beside does it need? Does anyone need an account, and if so, whose? In this book, the grown-up creates and holds any account, and most of what follows needs none.

Whatever you pick, the record still says **Computer Science** (Chapter 9). Combining programs doesn’t create a new subject.

The library first

Start here. A library card in your name costs nothing in most places, and it opens up:

- **Picture books and chapter books about computers, coding, and the internet.** Ask a librarian for books that explain how computers work or tell stories about programming. Reading one aloud is a fine Unplug First.
- **Public computers,** with you sitting beside your child and doing any logging in (OUT-03 Library Computing Visit, Chapter 7).
- **A librarian,** who is often the most useful person in your town for “where can we try a robot?” or “do you know a homeschool group that does coding?”
- **Things to borrow.** Some libraries lend tablets, robots, or kits. Ask yours. Borrowing for a few weeks tells you more than any review.

One picture-book world worth knowing is **Hello Ruby**, by Linda Liukas: a series of books about a girl exploring computers and programming, with free printable activities on its website. It describes itself as suited for children aged 5 and older.⁶⁰ Look for the books at your library first.

Free, and no device needed

CS Unplugged

What it is: a collection of free teaching material from the University of Canterbury in New Zealand that “teaches Computer Science through engaging games and puzzles that use cards, string, crayons and lots of running around.”³⁴

Fit: no device; you read a short lesson plan and set it up. Several topics are marked for ages 5 to 10: **Kidbots** (a person acts as the robot, which matches Chapters 2 and 5), **binary numbers** and **image representation** (Chapter 6), and **error detection and correction** (the parity card trick in Chapter 6). It’s a set of topics rather than a full year’s course, which makes it a good companion to this book.

CodeAI (formerly Code.org) Unplugged lessons

What it is: CodeAI offers a set of unplugged activities for kindergarten through grade 5 inside its free CS Fundamentals curriculum, for learning computer science without computers.³³

Fit: no device for the activities themselves; a device to read the lesson plans. Written for classrooms, so expect to shrink group games down to a family of two or three. More structured than CS Unplugged.

Hello Ruby printables

What it is: the free activities that go with the Hello Ruby books (above): printable games about what’s inside a computer, how the internet works, and how machines learn.⁶⁰

Fit: no device beyond a printer; very low parent load; best for 5–8 and for children who love stories.

Bebras (optional, once a year)

What it is: an international puzzle challenge about computing and logic. In the United States, the 2026 challenge runs **9–20 November**. It’s free, needs no preparation, and gives students 45 minutes. The youngest groups are **Kits** (ages 6–8) and **Castors** (ages 8–10). Students take part through a registered coordinator, usually a teacher. A parent can register as the coordinator for a homeschool; the registration page asks for a distinctive homeschool name, and approval takes a few business days.⁵⁹

Fit: a one-off event, fun for a puzzle-loving 7- to 10-year-old. It’s not a placement test, and it’s not a score for your records. If your child takes part, file the participation page as an artifact (Chapter 9).

Free, on a screen

ScratchJr (5–7)

What it is: a free app for tablets in which “young children (ages 5–7) can program their own interactive stories and games” by snapping together picture blocks. It’s made by the DevTech Research Group at Boston College and the Scratch Foundation, and is available in more than 20 languages. There’s no account to set up for the child.^{4,40}

Fit: a tablet; pre-readers welcome; open play, with structure available. Chapter 5 is built around it.

Structure if you want it: **Coding as Another Language (CAL)** is ScratchJr’s official curriculum, free from the ScratchJr site. It covers four grade levels from pre-kindergarten to second grade, each with 24 lessons of about 45 minutes, and it teaches programming alongside storytelling and literacy.⁶¹ At home, split a lesson across two Computing Hours.

Scratch (8–10)

What it is: a free block-based programming environment from the Scratch Foundation, with word blocks, for building games, stories, and animations.^{42,43} It runs in a web browser or as an offline app.

Fit: a laptop, desktop, or (with the app) some tablets; your child needs to read the blocks; open project work. Chapter 5 shows the tour-before-you-touch, remix-one-thing, and paper-plan-first routine.

Accounts: this book’s default is the **offline app** (no account), or an online account that **you** confirm, with public sharing left off. Since November 2024, anyone under 16 who registers a new Scratch account needs a parent’s permission by email; unverified accounts can still make and save projects but can’t comment or share.^{44,45} If you do create an account, choose a username that doesn’t reveal your child’s real name.

CodeAI CS Fundamentals (5–10)

What it is: six courses, A to F, one for each elementary grade from kindergarten to grade 5. Each has 13 to 17 lessons “designed for 45-minute periods.” There are also self-paced **Pre-reader Express** and **Express** courses. The organization states that its “curriculum and platform are available at no cost for anyone, anywhere, to teach.”³³ The organization renamed itself from Code.org to CodeAI on 2 June 2026.²¹

Fit: a computer or tablet; structured, lesson by lesson; pre-readers can use Pre-reader Express. A good choice if you’d like someone else to sequence the year.

How to use it at home: you set up any student logins yourself through a teacher section, and you keep the passwords. Split each 45-minute lesson across two Hours. Preview each lesson before your child sees it. If an AI helper appears anywhere in the course, keep it off or run it yourself (see the AI box in **The Computing Hour**). A course letter such as “Course C” is the program’s scope, not your child’s grade, and not a transcript line.

The Hour of AI (previewed)

What it is: a catalog of short activities from CodeAI and CSforALL, which has replaced the Hour of Code. Activities are available year-round, connect with Computer Science Education Week each December, and are designed “from pre-readers to adults,” including unplugged options.^{19,20} Some activities include an AI assistant.

Fit: a one-off or occasional treat. **You preview first**, then choose an unplugged activity or one with no chat for your child. Chapter 8 explains how it fits.

BBC Dance Mat Typing (8–10)

What it is: a free typing game on the BBC’s Bitesize site, in four levels, that teaches the home row.⁵⁵

Fit: a computer with a keyboard; for 8–10 only, in five-minute bursts, accuracy before speed (Chapter 6). At 5–7, finding letters and typing a first name is plenty; this can wait.

Fit at a glance

This table sorts by need, not by quality. Read across to find a starting point.

If your family needs...	Consider	Device	Reader?	Account?
No device at all	This book’s unplugged cards; CS Unplugged; CodeAI Unplugged; Hello Ruby printables; library books	None	Not needed	None
A first screen tool for 5–7	ScratchJr (CAL if you want lessons)	Tablet	Not needed	None
A structured course, someone else sequencing	CodeAI CS Fundamentals (Pre-reader Express for non-readers)	Computer or tablet	Optional (pre-reader path)	Parent-created student login
Open projects for 8–10	Scratch, offline app or parent-confirmed account	Computer (some tablets)	Yes	None offline; parent-confirmed online

If your family needs...	Consider	Device	Reader?	Account?
Something to touch	A button floor robot (5–7) or a micro:bit (8–10), borrowed first	Robot or board	Not for a floor robot	Usually none for the child
Typing practice for 8–10	BBC Dance Mat Typing	Computer with keyboard	Yes	None to start
A once-a-year puzzle outing	Bebras	Computer, through a coordinator	Yes	Through the coordinator
An occasional AI-literacy activity	Hour of AI, previewed, unplugged or no chat	Varies	Varies	Parent previews

“Account” here means the child’s. You may need your own account to download an app or read a lesson plan; that’s fine and expected.

Optional hardware

Every activity in this book has a no-robot version. A person on a masking-tape grid (BOT-01 Kidbot) teaches the same ideas as a floor robot, and costs nothing. If you’d like one piece of hardware, one is plenty. Borrow before you buy, from a library, a co-op, or a friend.

A button floor robot (5–7)

What it is: a small robot you program by pressing arrow buttons on its back, then pressing go. Several makers sell them for preschool classrooms.

Fit: pre-readers; tangible; good with a paper mat and the arrow cards from Chapter 2. **Check the age label:** small parts are a choking risk at 5–7. Check the current price; prices vary widely.

A micro:bit (8–10)

What it is: a small programmable board with buttons, lights, and sensors, from the non-profit Micro:bit Educational Foundation. The foundation says it’s designed for young people ages 8 and up, and it’s programmed with Microsoft MakeCode, a free block editor. It also works with Scratch.^{50,52}

Fit: 8–10, with you setting it up and handling batteries; lovely for input and output (Chapters 5 and 6). Use first names or nicknames only in projects like the Light-Up Name Badge (BOT-05). Check the current price and what’s included before you buy.

Three honest years, as illustrations

These are illustrations of how families might combine the shelf, not recommendations and not real families.

A no-device year, ages 5–7. The Computing Hour three or four times a week with this book’s unplugged cards; CS Unplugged’s Kidbots and binary lessons for extra games; a Hello Ruby book from the library each month, with one of its printables; and a library visit every few weeks to try ScratchJr on a borrowed tablet or a library computer. The record says Computer Science.

A tablet year, ages 5–7. Unplug First every Hour, then ten to fifteen minutes of ScratchJr most days; a few CAL lessons when you’d like a plan; a floor robot borrowed from a friend for a month in the spring; the Chapter 7 safety cards in week one, because the tablet is already part of family life. The record says Computer Science.

A laptop year, ages 8–10. CodeAI CS Fundamentals as a spine, split across two Hours per lesson, with any AI helper off; Scratch offline for one paper-planned project each semester; Dance Mat Typing in five-minute bursts three times a week; Bebras in November for fun. The record says Computer Science, plus Keyboarding if typing ran all year as its own strand.

None of these is the “right” year. Each one keeps the same habits at the center: predict before you run, debug before you start over, and keep the child’s program in the child’s hands.

For the parent

These are reference shelves for you, not lessons for your child.

The CSTA standards

The **2026 CSTA PK–12 Computer Science Standards**, from the Computer Science Teachers Association, are free to read and shared under a Creative Commons license (CC BY-NC-SA 4.0).¹ They describe what students might know and be able to do: a combined pre-kindergarten and kindergarten band, then one grade at a time from 1 to 5. Use them as a map for “is this age-reasonable?”, not a checklist to finish. Homeschool families aren’t required to follow them.

The overview page also says, usefully, what computer science **is not**: “using technology tools like word processors, slide decks, or generative AI tools.”¹

How to read a standard code

Every standard has a code like **EK-PRO-TR-07**. Here’s how to read it.

- **The first part is the band or grade.** EK is the pre-kindergarten and kindergarten band. E2 is grade 2. E4 is grade 4.
- **The second part is the concept.** ALG is Algorithms & Design. PRO is Programming. DAT is Data & Analysis. SYS is Systems & Security. SOC is Computing & Society.
- **The next two letters name a smaller topic** inside that concept.

- **The number** puts the standard in order within the band.

So **EK-PRO-TR-07** is a kindergarten-band Programming standard: “Identify a step in a sequence of commands that does not work as expected.”² That’s debugging, at age five, which is why Chapter 4 exists. You never need to write these codes in your records. They’re just a way to find the line you’re looking for.

AI4K12: Five Big Ideas in AI

A project of two professional groups (AAAI and CSTA) that organizes AI education around five ideas: perception; representation and reasoning; learning; natural interaction; and societal impact. It offers a printable poster and guidelines by grade band.⁵⁶ Chapter 8 lives mostly in “learning” and “natural interaction.”

COPPA, from the FTC

The Federal Trade Commission’s plain-language page, “Protecting Your Child’s Privacy Online,” explains the children’s privacy law in a few minutes: what counts as a child’s personal information, what companies must ask you first, and how to ask for information to be deleted.⁶ Chapter 7 gives the two-sentence version.

The AAP Family Media Plan

The American Academy of Pediatrics offers a free online Family Media Plan, a tool for writing down your family’s own choices about screens, places, and times.⁶² It pairs well with the Family Device Agreement in Chapter 7 (SAFE-06). The Computing Hour summarizes the current screen-time guidance once.

CDC milestone pages

The Centers for Disease Control and Prevention’s “Learn the Signs. Act Early.” pages describe what most children can do at each age, with parent tips. The age-5 page is where this book’s note about five-to-ten-minute attention spans comes from.⁹ They’re not a screening tool, and they’re not about computing, but they’re a calm reminder of what’s ordinary.

The CAL curriculum

If you’re using ScratchJr and want to see how a full curriculum sequences it, CAL (above) is free and readable even if you only borrow a few lessons.⁶¹

Low-device and no-device homes

Most American households have some kind of computer: in 2021, 95 percent had at least one, and 90 percent had broadband. Not all have a laptop or a tablet, though, and smartphone-only

households were more likely to have lower incomes.⁶³ If that's your house, you're in good company, and this book works for you.

- **A full year can run unplugged.** About two-thirds of the activity cards in this book need no device at all. Card programs are real programs, and the record still says Computer Science.
- **Your child should still meet a real program sometimes.** A monthly library visit, a borrowed tablet with ScratchJr for a few weeks, or a friend's laptop for an afternoon is enough.
- **A phone alone is hard for coding.** ScratchJr is made for tablets, and Scratch for larger screens. Lean on unplugged cards and the library.
- **One shared device is plenty** for siblings, with a driver and a navigator swapping on a timer.

Mixed ages and co-ops

A co-op or a group of families is a lovely place for this book's games: one tape grid, a few Kidbots, one robot for everyone, or a shared message-in-pieces relay across a church hall.

- **Pairs, not crowds.** Driver and navigator, with a timer. Neither child is the other's teacher.
- **One device or robot for the group** is enough if the unplugged warm-ups include everyone.
- **Photos crop to the work.** No group photos of children in shared folders.
- **No group chats or shared accounts for the children.** Grown-ups coordinate; children program.
- **Each family keeps its own records**, with each child's own captions.

The series, and the neighbors

Computing for Little Thinkers is the first book in the SMF WisdomForge Computing series. When your child is ready for longer programs, text-based code, and middle-school records, *Computing for Young Minds* (ages 11–14) picks up where this book stops. After that come *Computing for Emerging Adults* (ages 15–18) and *Computing Refresh for Adults*, for the grown-ups.

The other Little Thinkers books share the same calm Hour shape for other subjects. *Critical Thinking Skills for Little Thinkers* is the natural neighbor for the predicting, sorting, and reasoning habits here. *Math for Little Thinkers* sits beside the counting, patterns, and binary work. *Art for Little Thinkers*, and the History, Science, and English titles, each take one subject at a time. You don't need any of them to use this book.

A short before-you-buy check

Before you spend money on anything in this chapter, ask:

1. Have we used the library this month?
2. Could a card, a cup, or a person on a tape grid teach this idea first?
3. Does my child need an account for it? If so, who holds it?
4. Have I previewed it, including any AI helper inside?

5. Will the record still say Computer Science?

If the answers point to the library and the floor, you may not need to buy anything this year. If you do buy, buy for fit, and borrow first if you can.

A Note on Sources

Standards, studies, government pages, program pages and platform terms named in the chapters are listed in Notes, in one series at the back, so the teaching pages can keep a warm, plain voice. When the same source comes up in several chapters, it has one note, and every mark that cites it carries that note's number.

Sources were re-read on 1 October 2026. A few were read only at abstract level, and their notes say so: the second-grade study by del Olmo-Muñoz and colleagues, the kindergarten robotics study by Bers and colleagues, and the review of learning strategies by Donker and colleagues. Two pages blocked automated reading that day, so their wording was confirmed through search-engine records of the pages rather than a fresh read: the CDC's page on screen time for young children and Meta's AI terms. Meta's effective date could not be confirmed, so the note gives none. The high-school math study in the AI box comes from a different age and subject, and the book says so where it uses it.

A few things rest on the best available source rather than a perfect one, and the book says so instead of filling the gap. Scratch's "about 8 and up" rests on the Scratch Foundation's own age range ("8-13+") and its help page ("ages 8 to 16"). ScratchJr's "no account" rests on its privacy policy and FAQ, which describe projects stored on the device and ask only a few setup questions. The share of homeschooled students taught computer science comes from a 2016 federal survey, the most recent found that asked. Whether CodeAI's elementary courses show an AI helper is based on CodeAI's AI Tutor page, which lists a pilot for older students only. Common Sense Media and U.S. PIRG are advocacy and testing groups, not regulators, and their findings are independent testing. The five-step Hour, the wait-time pause, the dose suggestions, the try-its and the other practice rules are this book's own suggestions, not research findings, and the families in the examples are illustrations, not real people.

Program pages, platform terms, age rules, state rules and dates change; the dates in each note say when they were read. Nothing here is legal advice. Check your own state's homeschool rules with the state itself. Access date for web sources: 1 October 2026.

Notes

1. Computer Science Teachers Association (CSTA), *2026 CSTA PK–12 Computer Science Standards*, overview page, CSTA, 2026, <https://csteachers.org/pk12standards/>, accessed 1 October 2026. The page names five concepts (Algorithms & Design; Programming; Data & Analysis; Systems & Security; Computing & Society), spreads AI-related content across them rather than making it a separate concept, and says the Foundational Standards “begin with a combined pre-kindergarten and kindergarten (PK/K) band, followed by grade-level expectations for grades 1–5.” Under “Computer Science Is Not,” it lists “using technology tools like word processors, slide decks, or generative AI tools” and “being replaced by AI. CS is what powers AI.” The materials are licensed CC BY-NC-SA 4.0. The standards are a reference map of what is age-reasonable, not a homeschool requirement.
2. Computer Science Teachers Association (CSTA), “2026 CSTA Foundational Standards: PK–Kindergarten (Ages 4–6),” CSTA, 2026, <https://csteachers.org/pkk-foundational-standards/>, accessed 1 October 2026. Standards quoted in this book: EK-ALG-PS-01 “Carry out algorithms in daily activities”; EK-ALG-ML-02 “Recognize attributes of objects to notice patterns and make decisions”; EK-PRO-PD-04 “Create a sequence of commands to solve a problem or express an idea”; EK-PRO-VD-05 “Identify everyday gestures and symbols that represent information people use to make choices”; EK-PRO-TR-07 “Identify a step in a sequence of commands that does not work as expected”; EK-SYS-SE-12 “Differentiate between public and private information.” How to read a code: the first part names the band (EK = PK/K, E2 = grade 2, E4 = grade 4); the middle names the concept (ALG = Algorithms & Design, PRO = Programming, SYS = Systems & Security, SOC = Computing & Society); then come a two-letter topic tag and the standard’s number.
3. National Center for Education Statistics, *Parent and Family Involvement in Education: 2023* (NCES 2024-113), U.S. Department of Education, 2024, <https://nces.ed.gov/pubs2024/2024113.pdf>, accessed 1 October 2026. Among students aged 5 through 17 in kindergarten through grade 12, “some 3.4 percent were reported as being homeschooled” in 2022–23, about 1,765,000 students. The report’s separate 5.2 percent figure counts homeschooling and full-time virtual school together; it is not the homeschool share.
4. DevTech Research Group at Boston College and the Scratch Foundation, *ScratchJr*, home page, last updated June 2026, <https://www.scratchjr.org/>, accessed 1 October 2026: “ScratchJr is available as a free app”; “young children (ages 5-7) can program their own interactive stories and games”; downloads from the App Store, Google Play, and the Amazon Appstore; “20+ Languages.” The page names Coding as Another Language (CAL) as “the official curriculum of ScratchJr.”

5. Mary Budd Rowe, “Wait-Time and Rewards as Instructional Variables: Their Influence on Language, Logic, and Fate Control,” paper presented to the National Association for Research in Science Teaching, April 1972, ERIC ED061103, <https://eric.ed.gov/?id=ED061103>; and Robert J. Stahl, “Using ‘Think-Time’ and ‘Wait-Time’ Skillfully in the Classroom,” ERIC Digest, May 1994, ERIC ED370885, <https://eric.ed.gov/?id=ED370885>; both accessed 1 October 2026. Rowe studied elementary science classrooms and found typical waits near one second; longer waits of about three to five seconds changed how students talked. These are classroom studies. The book’s “count to five in your head” is a pause habit drawn from them, not a measured rule for kitchen tables.

6. Federal Trade Commission, “Protecting Your Child’s Privacy Online,” FTC Consumer Advice, May 2026, <https://consumer.ftc.gov/articles/protecting-your-childs-privacy-online>, accessed 1 October 2026. The COPPA Rule covers websites and online services, including apps, “directed to children under 13,” and general-audience sites that know they are collecting personal information from children that age. Such services must notify parents and get their approval first. Personal information includes a child’s full name, address, phone number or email address, physical location, “photos, videos and audio recordings of the child,” biometric identifiers, and persistent identifiers. Parents may have collected information deleted and can report problems at [ReportFraud.ftc.gov](https://www.ftc.gov/identitytheft).

7. Federal Trade Commission, “Children’s Online Privacy Protection Rule,” final rule, *Federal Register* 90 FR 16918, FR Doc. 2025-05904, 22 April 2025, <https://www.federalregister.gov/documents/2025/04/22/2025-05904/childrens-online-privacy-protection-rule>, accessed 1 October 2026. “The amended Rule is effective June 23, 2025”; with a few listed exceptions, “regulated entities have until April 22, 2026 to comply.” COPPA governs how companies collect children’s data. It does not rate content or certify a site as safe.

8. Tiffany Munzer et al., Council on Communications and Media, American Academy of Pediatrics, “Digital Ecosystems, Children, and Adolescents: Policy Statement,” *Pediatrics* 157, no. 2 (February 2026): e2025075320, published online 20 January 2026, <https://publications.aap.org/pediatrics/article/157/2/e2025075320/206129/>, accessed 1 October 2026. Quoted from the statement: “Time limits might range from <1 hour/d for toddlers and preschoolers to 1 to 2 hours/d or more of entertainment (not school-related) media for school-aged children and teens. The most important considerations are high-quality content and prioritizing healthy activities”; “Establish phone-free zones during mealtimes, in bedrooms, an hour before bed, and during homework.” See also [HealthyChildren.org](https://www.healthychildren.org/English/news/Pages/creating-a-child-friendly-digital-world-AAP-releases-new-media-recommendations.aspx), “A Child-Friendly Digital World: AAP Releases New Media Recommendations,” 2026, <https://www.healthychildren.org/English/news/Pages/creating-a-child-friendly-digital-world-AAP-releases-new-media-recommendations.aspx>, accessed 1 October 2026, which says the statement and its technical report “replace prior AAP guidance on communications and media.”

9. Centers for Disease Control and Prevention, “Milestones by 5 Years,” *Learn the Signs. Act Early.*, page updated 16 May 2026, <https://www.cdc.gov/act-early/milestones/5-years.html>, accessed 1 October 2026. The page lists “Pays attention for 5 to 10 minutes during activities. For example, during story time or making arts and crafts (screen time does not count),” and, among five-year-old milestones, following rules or taking turns in games and telling a story with at least two events. Its parent tips suggest no more than 1 hour of screen time a day, no screens for 1 to 2 hours before bed, and no screens in the bedroom. Milestones describe what most children (75 percent or more) do by an age; the checklist is not a screening test.

10. World Health Organization, *WHO Guidelines on Physical Activity and Sedentary Behaviour*, WHO, 25 November 2020, <https://www.who.int/publications/i/item/9789240015128>, accessed 1 October 2026. For ages 5–17: “an average of 60 minutes per day of moderate- to vigorous-intensity” physical activity across the week, and “Children and adolescents should limit the amount of time spent being sedentary, particularly the amount of recreational screen time.” The guidelines give no numeric screen limit.
11. Common Sense Media, “Grok AI Chatbot Not Safe for Teens, Common Sense Media Report Finds,” press release, 27 January 2026, <https://www.commonsensemedia.org/press-releases/grok-ai-chatbot-not-safe-for-teens-common-sense-media-report-finds>, accessed 1 October 2026. The risk assessment, run with teen test accounts, reported inadequate identification of users under 18, weak guardrails, and that “Kids Mode doesn’t work.” Common Sense Media is an advocacy and testing organization, not a regulator; its findings are independent testing, not official findings.
12. U.S. PIRG Education Fund, “The risks of AI toys for kids,” 11 December 2025, updated 7 January 2026, with the full report *AI Comes to Playtime: Artificial Companions, Real Risks and Trouble in Toyland 2025* (13 November 2025), <https://pirg.org/edfund/resources/ai-toys/>, accessed 1 October 2026. PIRG’s testers found that the toys’ guardrails “vary in effectiveness – and can even break down entirely,” that some “started to break down over the course of longer interactions,” and that “AI toys record children’s conversations.” PIRG is an advocacy and testing organization, not a regulator.
13. Weixin Liang, Mert Yuksekgonul, Yining Mao, Eric Wu, and James Zou, “GPT Detectors Are Biased against Non-Native English Writers,” *Patterns* 4, no. 7 (2023): 100779, <https://doi.org/10.1016/j.patter.2023.100779>, accessed 1 October 2026. Seven widely used detectors labeled more than half of 91 essays written by non-native English speakers as AI-generated (average false-positive rate 61.3 percent) while classifying U.S. eighth-grade essays accurately. The authors caution against using detectors in educational settings.
14. William H. Walters and Esther Isabelle Wilder, “Fabrication and Errors in the Bibliographic Citations Generated by ChatGPT,” *Scientific Reports* 13 (2023): 14045, <https://doi.org/10.1038/s41598-023-41032-5>, accessed 1 October 2026. Many citations produced by the chatbot versions tested did not exist, and real ones often contained errors. The study concerns adult research use; it is cited here for the parent’s habit of opening a source before trusting it.
15. Hamsa Bastani, Osbert Bastani, Alp Sungu, Haosen Ge, Özge Kabakçı, and Rei Mariman, “Generative AI without Guardrails Can Harm Learning: Evidence from High School Mathematics,” *Proceedings of the National Academy of Sciences* 122, no. 26 (2025): e2422633122, <https://doi.org/10.1073/pnas.2422633122>, accessed 1 October 2026. A field experiment with about 1,000 high-school math students in Turkey; a guarded tutor version largely removed the harm. Off-age evidence for this book: not ages 5–10 and not computing. It is used once, in the AI box in *The Computing Hour*.
16. Consumer AI terms, all accessed 1 October 2026; terms change, so check the current versions. OpenAI, “Terms of Use,” effective 1 January 2026, <https://openai.com/policies/terms-of-use/> (“You must be at least 13 years old or the minimum age required in your country to consent to use the Services. If you are under 18 you must have your parent or legal guardian’s permission”). SpaceXAI, “Terms of Service – Consumer,” last updated 11 September 2026,

<https://x.ai/legal/terms-of-service> (at least 13; ages 13 to 17 need a parent or legal guardian's permission). Microsoft, "Microsoft Copilot Supplemental Terms of Use," effective 18 August 2026, <https://www.microsoft.com/en-us/microsoft-copilot/for-individuals/terms-of-use> (you must have a Microsoft account and be "at least 13, but sometimes older, depending on your country's laws"). Meta, "Meta AIs Terms of Service," <https://www.facebook.com/legal/ai-terms> ("you cannot access AIs if you are under the age of 13 (or such greater age required in your country or territory)"). Anthropic, "Consumer Terms of Service," effective 8 October 2025, <https://www.anthropic.com/legal/consumer-terms> ("at least 18 years old or the minimum age required to consent to use the Services in your location, whichever is higher").

17. Google For Families Help, "Guide your child's Gemini Apps experience," Google, 2026, <https://support.google.com/families/answer/16109150>, accessed 1 October 2026. "A parent must enable access before their child under 13 (or the applicable age in your country) can use Gemini Apps with a supervised account"; the parent gets an email when the child first uses it; the filters "try to limit inappropriate content from appearing, but they're not perfect"; and Gemini Apps are not available for supervised accounts in the European Economic Area, Switzerland, and the United Kingdom. This book names the option and does not recommend it for ages 5–10.

18. SpaceXAI, "Frequently Asked Questions," Grok Bot documentation, SpaceXAI Docs, last updated 29 September 2026, <https://docs.x.ai/grok-bot/faq>, accessed 1 October 2026: "Every Bot on your account uses one persistent cloud computer. They share its files, browser sessions, and logins"; "The computer is assigned per user, not per Bot. Do not use separate Bots as a security boundary"; and, after a bot is deleted, "files and logins on that computer may remain." The FAQ describes no education product or child account.

19. CodeAI (formerly Code.org), "The Hour of AI," CodeAI, 2026, <https://code.org/en-US/hour-of-ai>, accessed 1 October 2026: "The Hour of Code is now the Hour of AI." The catalog includes at least one activity (for grades 8–12) that works "with AI Tutor as an AI Assistant."

20. CSforALL, "Hour of AI FAQ," 2026, <https://csforall.org/cs/hour-of-ai/faq>, accessed 1 October 2026. Activities are "available year-round" and connect with Computer Science Education Week each December; they are designed "from pre-readers to adults"; and families with limited computer access can "Choose activities in the catalog marked as unplugged."

21. CodeAI, "Code.org is now CodeAI," 2026, <https://code.org/en-US/codeai>, accessed 1 October 2026. The organization announced its new name on 2 June 2026 and describes CodeAI as "the same organization, with the same commitment to every student."

22. Chiara Montuori, Filippo Gambarota, Gianmarco Altoè, and Barbara Arfé, "The Cognitive Effects of Computational Thinking: A Systematic Review and Meta-Analytic Study," *Computers & Education* 210 (2024): 104961, <https://doi.org/10.1016/j.compedu.2023.104961>, accessed 1 October 2026. The review names four component skills (decomposition, algorithmic thinking, debugging, and abstraction), covers 19 studies with 1,523 participants aged 4–16 (11 meta-analyzed), and reports "beneficial effects of structured virtual and tangible coding (educational robotics) activities for preschoolers and first graders, and significant effects of more unstructured virtual coding activities (e.g., Scratch-based) for older students." The pool is small and mostly from schools and labs.

23. Jeannette M. Wing, “Computational Thinking,” *Communications of the ACM* 49, no. 3 (2006): 33–35, <https://doi.org/10.1145/1118178.1118215>, accessed 1 October 2026. The essay that popularized the term.

24. Karen Brennan and Mitchel Resnick, “New Frameworks for Studying and Assessing the Development of Computational Thinking,” paper presented at the annual meeting of the American Educational Research Association, 2012, copy at <https://people.cs.vt.edu/~kafura/CS6604/Papers/Framework-Assessing-CT.pdf>, accessed 1 October 2026. From young people’s Scratch projects the authors describe four sets of practices: “being incremental and iterative, testing and debugging, reusing and remixing, and abstracting and modularizing.”

25. Computer Science Teachers Association (CSTA), “2026 CSTA Foundational Standards: Grade 2 (Ages 7–8),” CSTA, 2026, <https://csteachers.org/grade-2-foundational-standards/>, accessed 1 October 2026. Standards quoted in this book: E2-ALG-PS-01 “Create an algorithm that includes sequence, events, and iteration to solve a problem or express an idea”; E2-ALG-ML-02 “Examine how data is used to train a machine learning model”; E2-PRO-TR-07 “Debug a program that includes sequence, events, and iteration”; E2-SYS-HW-11 “Explain how the basic hardware components of a computing system work together to perform input and output (I/O) operations”; E2-SYS-SE-12 “Explain how online actions have real-world consequences.”

26. Computer Science Teachers Association (CSTA), “2026 CSTA Foundational Standards: Grade 4 (Ages 9–10),” CSTA, 2026, <https://csteachers.org/grade-4-foundational-standards/>, accessed 1 October 2026. Standards quoted in this book: E4-ALG-PS-01 “Create a written representation of an algorithm that includes a combination of sequence, events, iteration, and selection to solve a problem or express an idea”; E4-ALG-ML-02 “Analyze relationships between the properties of training data and a machine learning model’s output”; E4-PRO-PD-04 “Compare different programming solutions to the same problem based on correctness and clarity”; E4-PRO-VD-06 “Trace how data flows and changes variable values in a program”; E4-PRO-TR-08 “Debug a program incrementally and repeatedly throughout the development process”; E4-SYS-HW-12 “Apply a basic troubleshooting process to identify and fix common hardware and software issues”; E4-SYS-SE-13 “Distinguish between authentication and authorization in protecting devices and private information”; E4-SYS-NT-14 “Compare wired and wireless methods that computing devices use to connect to the internet”; E4-SOC-HU-18 “Distinguish between human learning and machine learning processes.” The activities matched to these standards are this book’s own teaching moves.

27. Javier del Olmo-Muñoz, Ramón Cózar-Gutiérrez, and José Antonio González-Calero, “Computational Thinking through Unplugged Activities in Early Years of Primary Education,” *Computers & Education* 150 (2020): 103832, <https://doi.org/10.1016/j.compedu.2020.103832>, accessed 1 October 2026. A quasi-experimental classroom study of 84 second graders using activities from Code.org courses: one group began with unplugged activities and the other with plugged-in ones, then both worked on screen. The authors conclude that including unplugged activities “seems beneficial taking into account CT, motivation and gender.” Read at abstract level; one classroom study.

28. Scratch Foundation, “Tools,” 2026, <https://www.scratchfoundation.org/tools>, accessed 1 October 2026. The page lists Scratch with “Age range: 8-13+” and ScratchJr with “Age range: 5-7,” and describes ScratchJr’s icon-based blocks as “Designed with pre-readers in mind.” The Scratch Help page “About Scratch,” <https://scratch.mit.edu/help/about/>, describes Scratch as “designed es-

pecially for ages 8 to 16, but is used by people of all ages.”

29. DevTech Research Group, Boston College, “Enhancing Computer Science Education for K-2 Students: Insights from a Randomized Controlled Trial,” paper for the American Educational Research Association annual meeting, 2024, <https://sites.bc.edu/devtech/wp-content/uploads/sites/181/2024/04/AERA-CAL-efficacy-paper.pdf>, accessed 1 October 2026. With 1,057 students in 26 public elementary schools, “the CAL-ScratchJr curriculum was effective in improving students’ programming skills but no significant differences were detected for students’ computational thinking skills.” The trial was run by the curriculum’s developers.

30. Zhanxia Yang, Patricia Moore Shaffer, Courtney Hagan, Parastu Dubash, and Marina Bers, *Impact Study of the Coding as Another Language Curriculum: Study B*, grantee submission, June 2023, ERIC ED628951, <https://eric.ed.gov/?id=ED628951>, accessed 1 October 2026. Second graders in 20 randomly assigned schools: a significant positive effect on coding, no significant difference in computational thinking, and comparable standardized literacy scores. Run by the curriculum’s developers.

31. Ronny Scherer, Fazilat Siddiq, and Bárbara Sánchez Viveros, “The Cognitive Benefits of Learning Computer Programming: A Meta-Analysis of Transfer Effects,” *Journal of Educational Psychology* 111, no. 5 (2019): 764–792, <https://doi.org/10.1037/edu0000314>, accessed 1 October 2026. 105 studies across mixed ages; an overall transfer effect of $g = 0.49$, larger against untreated than against active control groups; “School achievement and literacy, however, benefited the least.”

32. Roy D. Pea and D. Midian Kurland, “On the Cognitive Effects of Learning Computer Programming,” *New Ideas in Psychology* 2, no. 2 (1984): 137–168, [https://doi.org/10.1016/0732-118X\(84\)90018-7](https://doi.org/10.1016/0732-118X(84)90018-7), accessed 1 October 2026. A review from the Logo era that found no reliable evidence that planning skills transferred from programming on their own. Later meta-analyses find some transfer, smaller when the comparison group also does something worthwhile.

33. CodeAI (formerly Code.org), “Computer Science Fundamentals,” CodeAI, 2026, <https://code.org/en-US/curriculum/computer-science-fundamentals>, accessed 1 October 2026. “Six comprehensive courses, one for each elementary grade” (Courses A–F, grades K–5), plus self-paced Pre-reader Express and Express courses and “CS Fundamentals Unplugged” for grades K–5 (“Learn computer science without computers”; unplugged unit at <https://studio.code.org/courses/k5-unplugged/units/1>). “Each CSF course includes 13-17 lessons designed for 45-minute periods”; Course F “teaches students about variables”; “Our curriculum and platform are available at no cost for anyone, anywhere, to teach!” The statement that the courses are aligned with CSTA standards is the publisher’s own description.

34. CS Unplugged, University of Canterbury, home page, “Topics,” and “Kidbots,” 2026, <https://www.csunplugged.org/en/>, <https://www.csunplugged.org/en/topics/>, and <https://www.csunplugged.org/en/t/>, accessed 1 October 2026. CS Unplugged describes itself as “a collection of free teaching material that teaches Computer Science through engaging games and puzzles that use cards, string, crayons and lots of running around.” Topics marked “Ages 5 to 10” include Binary numbers, Error detection and correction, Image representation, and Kidbots.

35. TTS Group, “Bee-Bot Programmable Floor Robot,” product page, 2026, <https://www.tts-group.co.uk/bee-bot-programmable-floor-robot/1015268.html>, accessed 1 October 2026. Named

only as an example of a button-programmed floor robot sold for preschool classrooms, not as a recommendation. Prices are left out on purpose.

36. Marina Umaschi Bers, Louise Flannery, Elizabeth R. Kazakoff, and Amanda Sullivan, “Computational Thinking and Tinkering: Exploration of an Early Childhood Robotics Curriculum,” *Computers & Education* 72 (2014): 145–157, <https://doi.org/10.1016/j.compedu.2013.10.020>, accessed 1 October 2026. Fifty-three kindergartners in the TangibleK robotics program were able to learn programming and robotics concepts, with mastery varying by how hard the concept was. Read at abstract level; the study was run by the curriculum’s developers.

37. ScratchJr, “Block Descriptions,” <https://www.scratchjr.org/learn/blocks>, accessed 1 October 2026. The block list includes Start on Green Flag, Start on Tap, Start on Bump, Start on Message, Send Message, Move Right, Hop, Say, Go Home, Play Recorded Sound, Repeat, Repeat Forever, Stop, and Go to Page; it has no “if” block.

38. Scratch Wiki, “Control Blocks” and “Events Blocks,” https://en.scratch-wiki.info/wiki/Control_Blocks and https://en.scratch-wiki.info/wiki/Events_Blocks, accessed 1 October 2026. In Scratch 3.0, Control blocks include repeat, forever, if-then, if-then-else, and repeat until; Events blocks include “when green flag clicked,” “when [key] pressed,” and “when this sprite clicked.” The Scratch Wiki is a community-written reference, not an official Scratch Foundation page; check the blocks in the app itself.

39. Andrew Hunt and David Thomas, *The Pragmatic Programmer* (Addison-Wesley), section “Debugging,” subsection “Rubber Ducking,” excerpt at <https://courses.cs.duke.edu/fall21/compsci307d/readings/prag-programmer-debugging.pdf>, accessed 1 October 2026. The excerpt says that “the simple act of explaining, step by step, what the code is supposed to do often causes the problem to leap off the screen and announce itself,” tells how a colleague kept a small rubber duck on his terminal while coding, and adds that if no person is around, “a rubber duck, or teddy bear, or potted plant will do.”

40. Scratch Foundation, “ScratchJr Privacy Policy,” last updated April 2022, <https://www.scratchjr.org/privacy>, accessed 1 October 2026. Projects, “including pictures or sound recordings,” are “only stored locally on your device and [are] not transmitted to or accessible by Scratch”; “You can delete your projects by deleting the App from your device.” The app asks only whether it will be used at home or at school and in which language, and describes no account or sign-in; it does collect some device and usage information for analytics.

41. ScratchJr, “Frequently Asked Questions,” last updated June 2026, <https://www.scratchjr.org/about/faq>, accessed 1 October 2026: “ScratchJr saves project data in an internal database stored locally on your device. ScratchJr is not currently set up for shared installation, so all users of a device share the same project database.” Projects can be shared from tablet to tablet by email (and AirDrop on iPads). See also Scratch Help Center, “My ScratchJr projects disappeared from my school device,” <https://mitscratch.freshdesk.com/en/support/solutions/articles/4000216685-my-scratchjr-projects-disappeared-from-my-school-device->, accessed 1 October 2026: when the app is deleted or offloaded, “so is all of the user-made content.”

42. Scratch Foundation, home page, 2026, <https://www.scratchfoundation.org/> (the older <https://scratch.mit.edu/about> address now leads here), accessed 1 October 2026: “Scratch is free

for everyone”; “Scratch was developed at the MIT Media Lab in 2007 by a team of educational experts and researchers led by Professor Mitchel Resnick. In 2019, Scratch was transferred from MIT to the Scratch Foundation”; and the online community is “safe, moderated.”

43. Scratch Help Center, “What are the system requirements for Scratch?,” modified 11 September 2026, <https://mitscratch.freshdesk.com/en/support/solutions/articles/4000156845-what-are-the-system-requirements-for-scratch->, accessed 1 October 2026. The offline Scratch app runs on macOS 12 and higher, Windows 10 and higher, and “Android 8.0 and higher, on tablets and Chromebooks”; Scratch “does not currently support creating or editing projects on phones”; and on tablets there is “currently no way to use ‘key pressed’ blocks.” The download page is <https://scratch.mit.edu/download>. Platforms change; check before installing.

44. Scratch Team, “Under 16,” Scratch Help Center, <https://mitscratch.freshdesk.com/en/support/solutions/articles/under-16>, accessed 1 October 2026. Anyone under 16 who registers a new Scratch account needs permission from a parent or guardian, by an email link; unverified accounts “can still make and save Scratch projects, but they cannot comment, share, or participate in the online community.” The change was announced on the Scratch forums on 6 November 2024 (“Parental Permissions for Scratch Accounts,” <https://scratch.mit.edu/discuss/topic/787896/>).

45. Scratch Foundation, “Scratch Privacy Policy,” last updated 27 March 2026, <https://mitscratch.freshdesk.com/en/scratch-privacy-policy>, accessed 1 October 2026. “If you would like to build projects with Scratch without submitting any Personal Information to us, you can download and use the Scratch app offline. Projects created in the Scratch app are not accessible by Scratch.” When creating an account, users are asked to “select a username that does not disclose your real name.”

46. CodeAI (formerly Code.org), “AI Tutor,” 2026, <https://code.org/en-US/tools/ai-tutor>, accessed 1 October 2026. The page offers AI Tutor as a pilot for grades 6–12 “For students age 13 +,” in select units of AI Foundations, CS Discoveries, and CS Principles, where programming units are “opt-in only”; a teacher turns it on. CS Fundamentals (grades K–5) is not listed. Course features change; preview any lesson before your child uses it.

47. Afke Donker and Pieter Reitsma, “Young Children’s Ability to Use a Computer Mouse,” *Computers & Education* 48, no. 4 (2007): 602–617, ERIC EJ747492, <https://eric.ed.gov/?id=EJ747492>, accessed 1 October 2026. 104 Dutch children in kindergarten and first grade; drag-and-drop was faster, with fewer errors, than click-move-click. Read at abstract level; an older lab-style study of mouse use, not of any current app.

48. Irene Lee, Fred Martin, Jill Denner, et al., “Computational Thinking for Youth in Practice,” *ACM Inroads* 2, no. 1 (2011): 32–37, <https://doi.org/10.1145/1929887.1929902>, accessed 1 October 2026. The Use–Modify–Create progression: learners use an existing program, then modify it, then create their own. It was developed with middle- and high-school youth programs and is adapted lightly here for 8–10.

49. Jean Salac, Cathy Thomas, Chloe Butler, Ashley Sanchez, and Diana Franklin, “TIPP&SEE: A Learning Strategy to Guide Students through Use→Modify Scratch Activities,” *Proceedings of the 51st ACM Technical Symposium on Computer Science Education (SIGCSE 2020)*, <https://doi.org/10.1145/3328778.3366821>, accessed 1 October 2026. Title, Instructions, Purpose, Play; then Sprites, Events, Explore. Studied in upper-elementary classrooms, with gains on harder

assessment items. “Tour Before You Touch” is this book’s home adaptation.

50. Micro:bit Educational Foundation, “What is the micro:bit?,” 2026, <https://microbit.org/get-started/what-is-the-microbit/>, accessed 1 October 2026. The foundation describes the BBC micro:bit as a small programmable device designed to help young people ages 8 and up learn computing, programmed with Microsoft MakeCode blocks or Python, and also usable with Scratch; its coding tools and classroom resources are free. Prices are left out on purpose.

51. Micro:bit Educational Foundation, “MakeCode,” micro:bit developer documentation, <https://tech.microbit.org/software/makecode/>, accessed 1 October 2026: “Microsoft MakeCode for micro:bit is a free, open source platform”; “A device simulator shows you what will happen on the physical device before you download your program.”

52. Microsoft MakeCode for micro:bit, editor home page, <https://makecode.microbit.org/>, accessed 1 October 2026 (built-in tutorials include “Name Tag” and “Dice”); and Microsoft, “MakeCode Error Helper” FAQ, <https://makecode.microbit.org/errorhelper/ai-faq>, accessed 1 October 2026: when a program throws an error, “students can click an ‘Explain with AI’ button in the problems window to get an AI-generated walkthrough of the issue.”

53. CS Unplugged, University of Canterbury, “Error Detection and Correction” (topic) and “Parity Magic” (lesson, ages 8 to 10), <https://www.csunplugged.org/en/topics/error-detection-and-correction/> and <https://www.csunplugged.org/en/topics/error-detection-and-correction/unit-plan/parity-magic/>, both accessed 1 October 2026. The lesson uses a set of 36 two-colored cards in a 5-by-5 grid plus one parity row and column, describes the cards as bits (“the way everything is represented on computers”), and notes that “this kind of method is applied to almost all data that is stored or transmitted on computers (although usually a more sophisticated method is used).”

54. National Governors Association Center for Best Practices and Council of Chief State School Officers, *Common Core State Standards for English Language Arts* (2010), Writing standard 6, grades 3–5 (W.3.6, W.4.6, W.5.6), https://learning.ccsso.org/wp-content/uploads/2022/11/ELA_Standards1.pdf, accessed 1 October 2026. Grade 3 uses technology “with guidance and support from adults”; grade 4 expects enough keyboarding “to type a minimum of one page in a single sitting”; grade 5, two pages. A public-school standards map, not homeschool law.

55. BBC Bitesize, “Dance Mat Typing – KS2 Computing,” BBC, 2026, <https://www.bbc.co.uk/bitesize/topics/zf2f>, accessed 1 October 2026. A free typing game in levels that starts with the home row; the page advises “use the correct fingers” and “there’s no need to rush!”

56. AI4K12, an initiative of the Association for the Advancement of Artificial Intelligence (AAAI) and CSTA, “Five Big Ideas in AI” and K–12 AI guidelines by grade band, <https://ai4k12.org/>, accessed 1 October 2026. The five ideas are Perception; Representation & Reasoning; Learning; Natural Interaction; and Societal Impact.

57. Five state examples, checked on official pages, accessed 1 October 2026 (laws change; read your own state’s current rules; this is not legal advice). Texas Education Agency, “Home Schooling” and “Alternative Schooling,” <https://tea.texas.gov/families-and-students/finding-school-your-child/home-schooling> and <https://tea.texas.gov/families-and-students/finding-school-your-child/alternative-schooling> (under *Leeper*, a bona fide written curriculum of “reading, spelling, grammar, math, and a course in good citizenship”). North Carolina Division of Non-Public

Education, “Requirements and Recommendations,” <https://www.doe.nc.gov/divisions/non-public-education/home-schools/requirements-recommendations> (an annual nationally standardized achievement test that “must measure English grammar, reading, spelling, and mathematics”). Pennsylvania Public School Code § 13-1327.1, <https://www.legis.state.pa.us/WU01/LI/LI/US/HTM/1949/0/0014.013.0> (the elementary course list names English, arithmetic, science, geography, history, civics, safety education, health and physiology, physical education, music, and art). Virginia Department of Education, “Home Instruction,” <https://www.doe.virginia.gov/parents-students/for-parents/home-instruction> (the parent submits “A list of the subjects to be studied for the coming year”). New York, 8 NYCRR § 100.10, via Cornell Legal Information Institute, <https://www.law.cornell.edu/regulations/new-york/8-NYCRR-100.10> (required subjects for grades one through six). None of the five names computer science or technology among required elementary subjects.

58. Jiashan Cui and Rachel Hanson, *Homeschooling in the United States: Results from the 2012 and 2016 Parent and Family Involvement Survey (PFI-NHES: 2012 and 2016)* (NCES 2020-001), National Center for Education Statistics, 2019, Table 9, “Computer science (e.g., computer programming),” <https://nces.ed.gov/pubs2020/2020001.pdf>, accessed 1 October 2026. Taught during home instruction in 2016: 16 percent overall; 18 percent at grades 3–5; 11 percent at K–2, which NCES flags to interpret with caution. The figure describes that one year, not later years; no later federal table on this subject was found.

59. USA Bebras Computing Challenge, home and “Registration” pages, <https://www.bebbraschallenge.org/> and <https://www.bebbraschallenge.org/registration/>, accessed 1 October 2026. “The Bebras Challenge USA 2026 is scheduled for November 9–20”; participation “is free” and needs no preparation; students aged 6 to 18 have 45 minutes; the youngest groups are Kits (age 6–8) and Castors (age 8–10). Students take part through a teacher registered as a coordinator, and the registration page tells homeschool teachers to “select a distinctive name for your homeschool (e.g., Home School [initials] ...)”; processing “may take 1-3 business days.”

60. Linda Liukas, *Hello Ruby*, home page and “Play,” Hello Ruby, 2026, <https://www.helloruby.com/> and <https://www.helloruby.com/play>, accessed 1 October 2026. The site says Hello Ruby is “suited for kids age 5 years and older” and offers “free, fun activities,” with a set of printable activities for each book.

61. ScratchJr, “Curriculum,” DevTech Research Group, Boston College, <https://www.scratchjr.org/teach/curriculum>, accessed 1 October 2026. DevTech offers free ScratchJr curricula, including Coding as Another Language (CAL), “the official curriculum of ScratchJr,” which “covers four grade levels, from Pre-Kindergarten to 2nd grade, each made up of 24 lessons of about 45 minutes.”

62. American Academy of Pediatrics, HealthyChildren.org, “Family Media Plan,” <https://www.healthychildren.org>, accessed 1 October 2026. A free online tool for writing down a family’s own choices about screens, places, and times.

63. U.S. Census Bureau, “Computer and Internet Use in the United States: 2021,” press release CB24-TPS.61, 18 June 2024, <https://www.census.gov/newsroom/press-releases/2024/computer-internet-use-2021.html>, accessed 1 October 2026. In 2021, 95 percent of households had at least one type of computer and 90 percent had a broadband subscription; “smartphone-only” households were more likely to have incomes of \$25,000 or less.